

Intelligent SFP module
ISM4120A, ISM4130A, ISM4140A, ISM4150A
ISM4100B
ISM4100J
ISM4100L
ISM4100N
ISM4100P
User Guide

Version 19, 2025

No part of this document may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior permission of the manufacturer.

Table of Contents

1. Safety measures	8
2. Introduction	9
2.1. Overview.....	9
2.2. Application.....	10
2.3. Main features of Smart SFP 1G copper and Smart SFP 1G fiber	12
2.3.1. Loopback	12
2.3.2. Passthrough	12
2.3.3. Statistics.....	12
2.3.4. Filter statistics	12
2.3.5. Filter.....	13
2.3.6. Loaned IP	13
2.3.7. ITU-T Y.1564.....	13
2.3.8. TWAMP Light.....	13
2.3.9. PTPv2 (IEEE 1588) and NTP.....	14
2.3.10. Traffic capturing and mirroring	14
2.3.11. Service activation test with interval statistics	15
2.3.12. Service of managing additional network stack settings.....	17
2.3.13. Timestamp insertion.....	17
2.3.14. Monitoring and control of Smart SFP signals	17
2.3.15. Remote management	17
2.4. Main features of Smart SFP 1G RS-232	17
2.4.1. IEC 60870-5-101 transfer over Ethernet/IP networks	17
2.4.2. RS-232 transfer over Ethernet/IP networks	18
2.4.3. RS-232 terminal server function.....	18
2.4.4. PTPv2 (IEEE 1588) and NTP.....	18
2.4.5. Remote management	18
2.5. Main features of Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232.....	18
2.5.1. Time synchronization from GNSS.....	18
2.5.2. PTPv2 (IEEE 1588) and NTP.....	19
2.5.3. Remote management	19
2.6. Main features of Smart SFP 1G USB	19
2.6.1. USB devices connection	19
2.6.2. PTPv2 (IEEE 1588) and NTP.....	19
2.6.3. Remote management	19
3. Technical specs.....	20

3.1. General	20
3.2. Module part numbers	21
3.3. Operating conditions	22
3.4. External Smart SFP view	23
3.5. Internal block diagram	25
3.5.1. Internal block diagram of Smart SFP 1G fiber and Smart SFP 1G copper	25
3.5.2. Internal block diagram of Smart SFP 1G RS-232	25
3.5.3. Internal block diagram of Smart SFP 1G GNSS	26
3.5.4. Internal block diagram of Smart SFP 1G GNSS-RS232	26
3.5.5. Internal block diagram of Smart SFP 1G USB	27
3.6. DIP switch	27
3.7. LED	28
4. Delivery set	29
5. Software structure	29
6. Installation and setup.....	30
6.1. Inserting Smart SFP into a Host Device	30
6.2. IP interfaces	31
6.3. Default accounts.....	31
6.4. Connect to Smart SFP.....	32
6.4.1. SSH.....	32
6.4.2. SSH connection using a key	33
7. Smart SFP connection schemes	34
7.1. In-line connection.....	34
7.1.1. Access through the line interface (port A)	34
7.1.2. Access via both ports (A and B).....	35
7.2. Out-of-line connection	36
8. Changing Smart SFP port settings.....	38
8.1. Changing the speed of port A and port B	38
8.1.1. Automatic speed change on Smart SFP ports	38
8.1.2. Changing the speed on the Smart SFP ports manually.....	39
8.2. Changing the MAC address of port A and port B	40
9. Packet transmission via Smart SFP 1G fiber and copper	42
9.1. Block diagram.....	42
9.2. Traffic priorities inside Smart SFP 1G fiber and copper	43
9.3. Package processing steps	43
9.3.1. Passthrough	43
9.3.2. Loopback	43
10. Configure device functions (CLI and API)	46
10.1. Overview.....	46

10.1.1. CLI	46
10.1.2. Profiles	50
10.1.3. REST API	50
Request example	50
Response example	50
Request format	51
Response format	51
10.2. Commands description.....	52
10.2.1. Loopback commands.....	52
10.2.2. Loopback API	57
10.2.3. Statistics commands in configuration mode	60
10.2.4. Statistics API	61
10.2.5. Commands in the Smart SFP optical transceiver configuration mode.....	63
10.2.6. Smart SFP optical transceiver API	63
10.2.7. Y.1564 commands	64
10.2.8. Y.1564 API.....	74
10.2.9. Filters commands	84
10.2.10. Filters API	89
10.2.11. Filter statistics commands	95
10.2.12. Filter statistics API.....	99
10.2.13. Time synchronization commands.....	104
10.2.14. Time synchronization API	109
10.2.15. Loaned IP function commands	115
10.2.16. Loaned IP function API	118
10.2.17. Traffic capturing and mirroring commands.....	122
10.2.18. Traffic capturing and mirroring API	131
10.2.19. Commands for the service activation test with interval statistics generator function.....	142
10.2.20. Service activation test with interval statistics generator function API	149
10.2.21. Commands for the service activation test with interval statistics analyzer function.....	155
10.2.22. Service activation test with interval statistics analyzer function API	163
10.2.23. Commands for the service of managing additional network stack settings	170
10.2.24. API for the service of managing additional network stack settings.....	171
10.2.25. Commands for the timestamp insertion “tsins” function	172
10.2.26. API for the timestamp insertion function “tsins”	176
10.2.27. Smart SFP signals monitoring and managing commands.....	179
10.2.28. Smart SFP signals monitoring and managing API	181
10.2.29. Show commands	182
10.2.1. Show commands API.....	184
11. Data transfer configuration for Smart SFP 1G RS-232	186

11.1. RS-232 data transfer over Ethernet/IP networks	186
11.2. IEC 60870-5-101 data transfer over Ethernet/IP networks	190
11.2.1. IEC-101 “point-to-two-points” data transfer configuration example	190
11.2.2. IEC-101 data transfer settings	194
11.2.3. IEC-101 data transmission service management.....	203
11.2.4. IEC-101 data transmission state monitoring using SNMP	212
11.2.5. Backup and restore the configuration of the module with IEC-101 data transmission function	213
12. Factory reset	220
13. Software upgrade	221
13.1. Software upgrade of Smart SFP with installed version 1.0.101 or higher	221
13.1.1. Necessary files.....	221
13.1.2. Keeping settings during software upgrade	221
13.1.3. Upgrade procedure	222
13.2. Upgrade procedure from software versions below 1.0.101 to version 1.0.104	223
13.2.1. Necessary files.....	223
13.2.2. Upgrade procedure	223
14. EEPROM memory reprogramming	226
14.1. Reprogramming the A0h (0x50) EEPROM page using a programmer.....	226
14.2. Reprogramming the A0h (0x50) EEPROM page without using a programmer	227
14.3. Reprogramming the A2h (0x51) EEPROM page	227
15. Maintenance	229
16. Examples of typical settings	229
16.1. Loopback	229
16.2. Loopback by filter	230
16.3. TWAMP Light (reflector)	231
16.4. Traffic capturing and mirroring	232
16.5. Loaned IP.....	233
16.5.1. Configuring loaned IP settings	234
16.5.2. Configuring network and system parameters and applying all changes	234
16.6. Time synchronization from NTP server	235
16.7. Time synchronization from PTP server.....	237
16.8. Filter statistics.....	238
16.9. Y.1564.....	239
16.10. Timestamp insertion	240
16.11. Configuring VLAN subinterfaces.....	242
16.12. Speed calculating at different levels	243
16.13. Configuring time synchronization from GNSS.....	244
16.14. PTP server configuring with synchronization from GNSS	247

16.15. NTP server configuring with synchronization from GNSS	248
Appendix 1. Pins of the electrical connector of SFP module	249
Appendix 2. Pins of the 8P8C connector of Smart SFP 1G RS-232.....	249
Appendix 3. Pins of the 8P8C connector of Smart SFP 1G GNSS	250
Appendix 4. Smart SFP Dimensions	251

1. Safety measures

Before using the device, carefully read this user manual.

- Get the device from the box and make the external inspection.
- Keep the device in normal environmental conditions for at least 2 hours (if the device has been previously kept in conditions distinct from normal).
- When using the device, it is necessary to comply with the general requirements of fire safety regulations.
- It is necessary to protect the device from shock, moisture, and dust, prolonged exposure to direct sunlight.

2. Introduction

2.1. Overview

Intelligent SFP module (hereinafter may be referred to as device, module or Smart SFP) has different models for different use cases.

Smart SFP 1G copper and Smart SFP 1G fiber are network performance monitoring and diagnostic devices in the SFP form factor. Smart SFP can be installed in any network device with an SFP slot and act simultaneously like a standard SFP module and a probe. Depending on the feature set, it can act like an IP SLA Probe, smart loopback, or network appliance. Smart SFP 1G copper and Smart SFP 1G fiber provide a possibility to run RFC 2544/Y.1564, TWAMP tests, turn up smart loopback on L2-L4, analyze traffic statistics and capture traffic. They can be used for continuous network performance monitoring and diagnostic or service activation tests (SAT).

Smart SFP 1G RS-232 is a converter for RS-232 data transmission over Ethernet/IP networks. It can be installed in any network device with an SFP slot. Depending on the feature set, it can be used as a converter for transferring IEC 60870-5-101 protocol data over Ethernet/IP networks, or as a RS-232 over Ethernet/IP converter with up to three RS-232 interfaces.

Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232 modules are precision time servers with a port for connecting a global navigation satellite system receiver. These modules can be installed in any network device with an SFP slot and can be used as a PTP or NTP precision time server in an Ethernet/IP network.

Smart SFP 1G USB functions as a universal network appliance with a USB port. This module can be installed in any network device with an SFP slot and can be used to connect any USB devices to an Ethernet/IP network.



Figure 2.1. Smart SFP appearance

2.2. Application

The figures below show a typical example of Smart SFP IP SLA Probe usage on mobile operator networks, ISP networks for performance monitoring, or enterprises for service assurance.

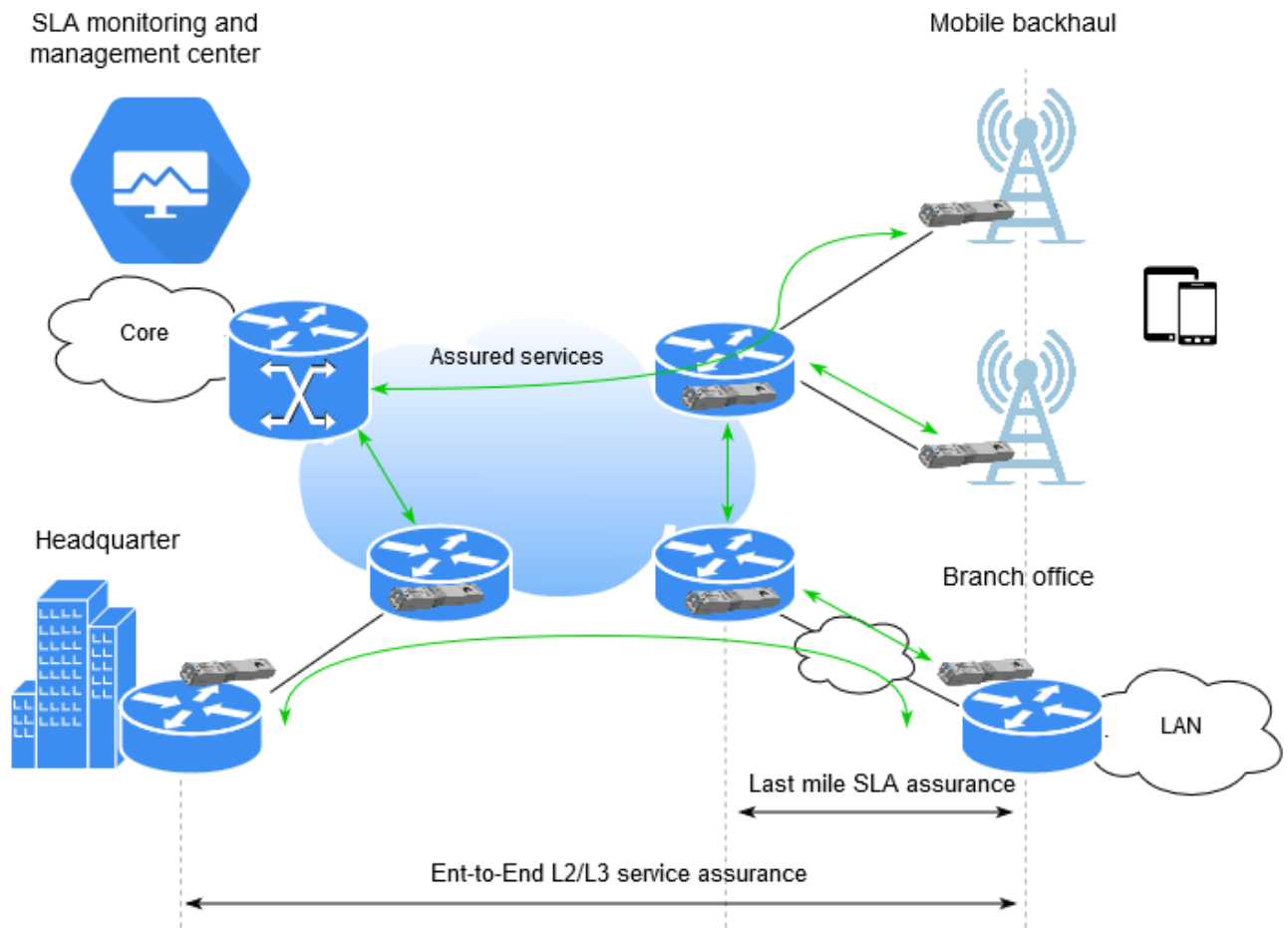


Figure 2.2. Using Smart SFP on the mobile operator network for performance monitoring and by enterprise for service assurance

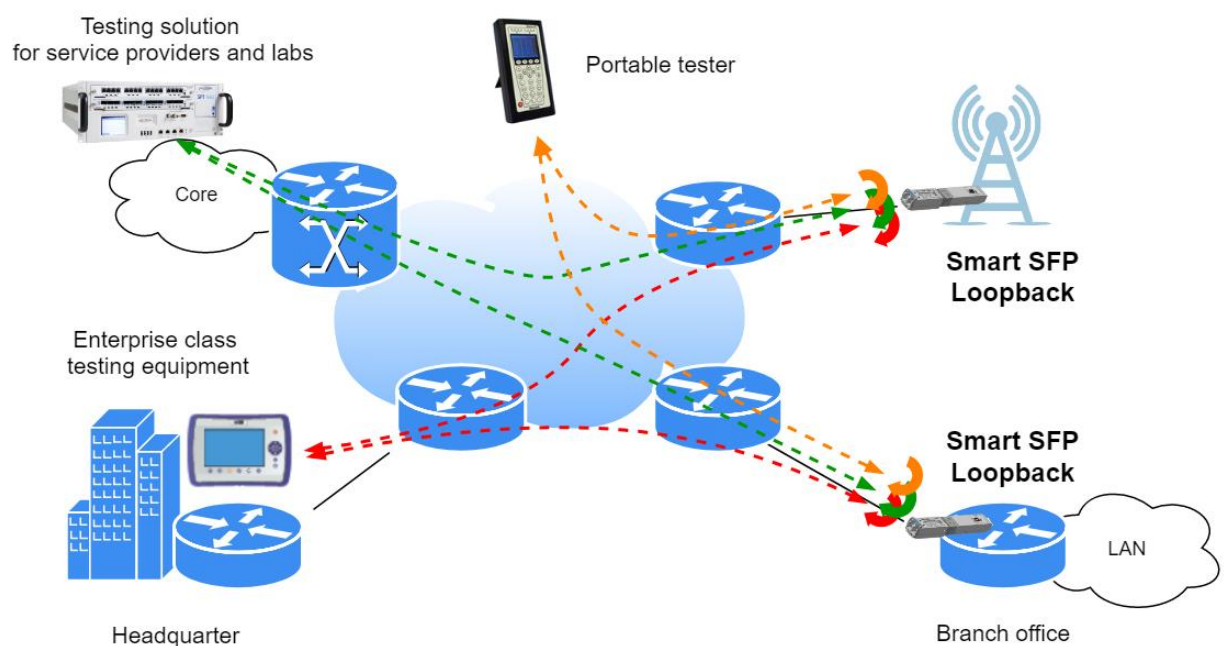


Figure 2.3. Using Smart SFP as a Smart Loopback

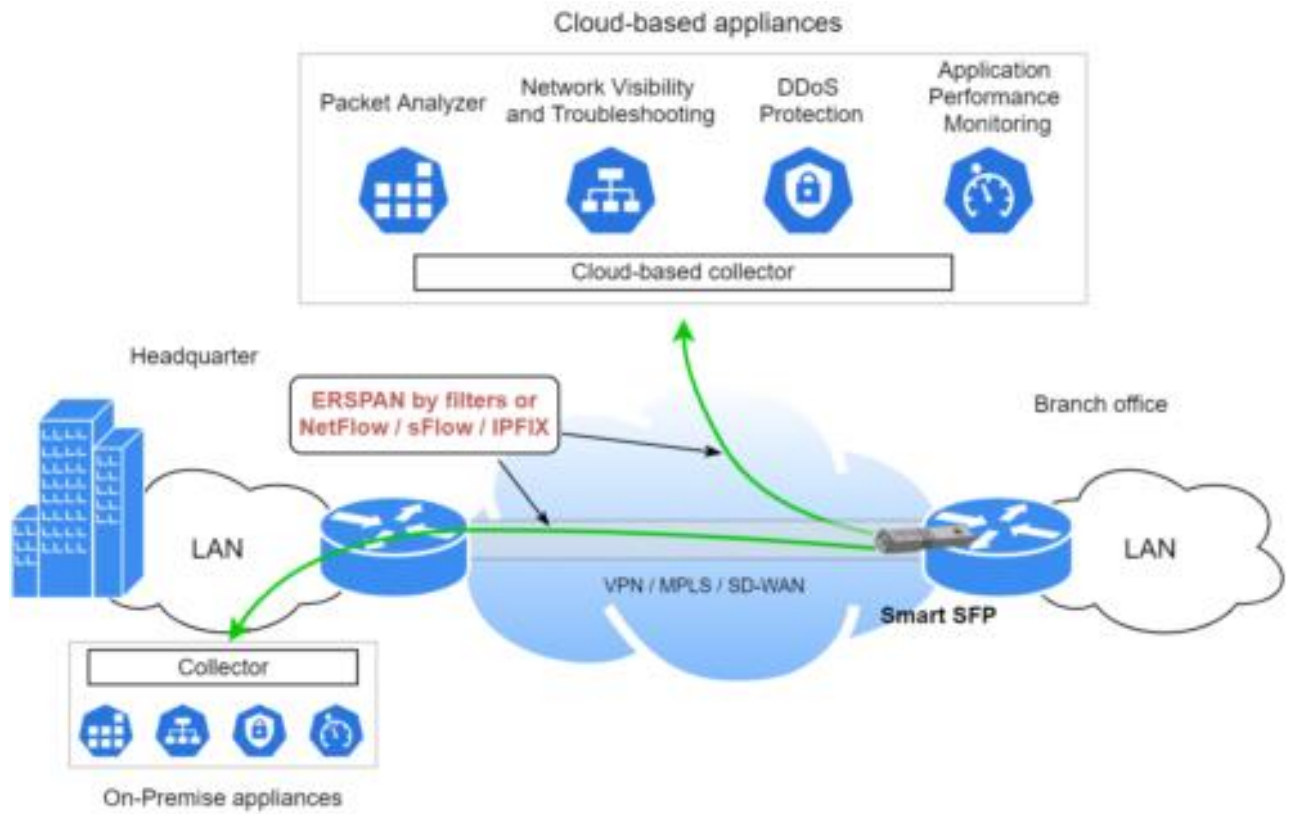


Figure 2.4. Using Smart SFP in a corporate network for remote packet capturing and mirroring

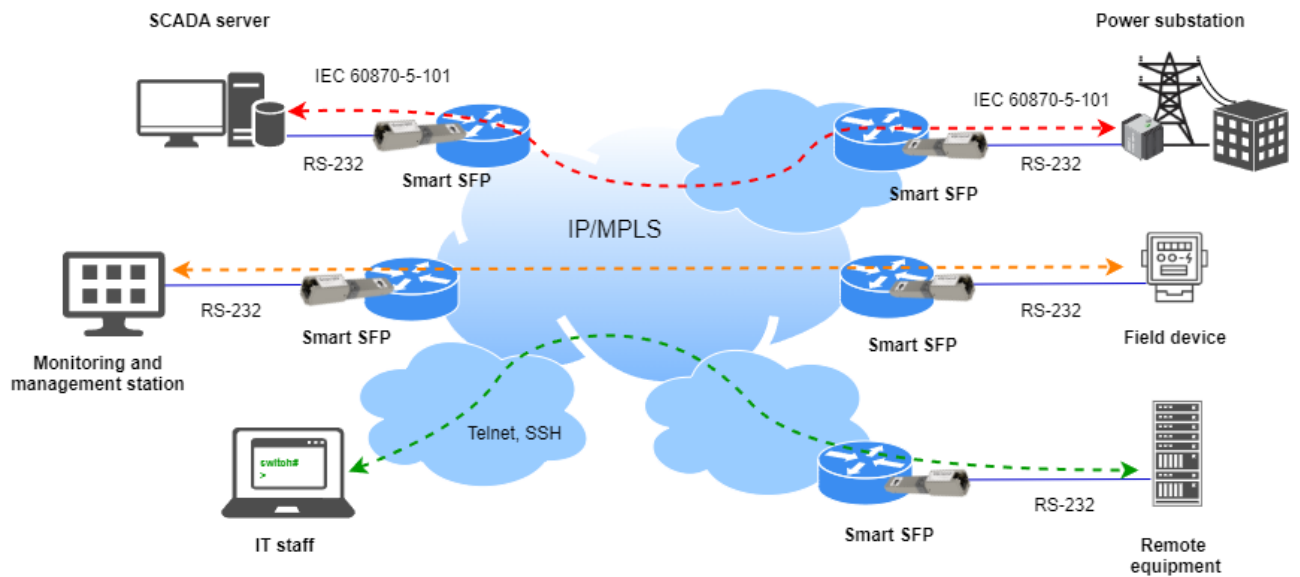


Figure 2.5. Using Smart SFP 1G RS-232 for IEC 60870-5-101 data transfer, RS-232 devices connection or terminal server function

2.3. Main features of Smart SFP 1G copper and Smart SFP 1G fiber

This User guide describes functions of Smart SFP 1G copper and Smart SFP 1G fiber running software version 1.0.104. These devices can have following feature sets:

Table 2.1. Smart SFP 1G copper and Smart SFP 1G fiber feature sets

Feature set number	01	02	03	04	05
Loopback	-	+	+	+	-
Passthrough	+	+	+	+	+
Statistics	-	+	+	+	+
Filter statistics	-	-	-	+	-
Filter	-	+	+	+	-
Loaned IP	+	+	+	+	+
ITU-T Y.1564	-	-	+	-	-
TWAMP Light (RFC 5357)	-	+	+	+	-
PTPv2 (IEEE 1588) and NTP	+	+	+	+	+
Packet capturing and mirroring (ERSPAN)	-	+	-	-	+
Service activation test with interval statistics	-	-	-	-	+
Timestamp insertion	-	-	-	+	-
Monitoring and control of line interface signals	+	+	+	+	+
Remote management	+	+	+	+	+

Note. To change the set of functions, you need to install the corresponding software (see section 13).

2.3.1. Loopback

The “Loopback” function allows the device to swap the contents of the received packets fields and send the changed frames back to the sender¹.

2.3.1.1. Loopback label support

Smart SFP supports layer 2, layer 3, and layer 4 loopback for streams with up to three VLAN labels and layer 3 and layer 4 loopback for streams with up to three MPLS labels.

2.3.2. Passthrough

When connecting to the tested channels to the break-in existing connection, the device operates in the software bypass mode with interfering of test traffic — network traffic is transmitted from port A (B) to port B (A) while running tests. In the current software version, passthrough mode is always on.

2.3.3. Statistics

The “Statistics” function allows to count all received and transmitted frames. Smart SFP supports collecting the following types of statistics: the number of received and transmitted frames, the number of received and transmitted bytes, and the data reception and transmission rate for the last second calculated at the L2 level.

2.3.4. Filter statistics

The “Filter statistics” function is used to gather the statistics of the transit traffic flows. The flow can be defined by the following packet’s fields:

- protocol type (UDP/TCP)

¹ Loopback layer 1 is not supported in current software version.

- source /destination TCP/UDP port number
- source/destination IP address
- source/destination subnet mask
- VLAN identifier
- VLAN priority
- Quality of Service type
- Precedence value
- DSCP value

There are three ways to aggregate statistics:

- 1) statistics will be aggregated for packets with the specified field value
- 2) statistics will be aggregated for all packets: with any field value or without it
- 3) statistics will be aggregated for packets with any field value

The current software version supports 36 filters for the statistics of transit traffic flows. Filters can be assigned to ports A and B in any combination. For every filter amount of bytes and packets counts.

2.3.5. Filter

The “Filter” function allows determining the route of packets that match the filtering rules. There are two types of filters: “selector” (12 filters) and “CPU” (2 filters).

Selector filters specify source/destination IP addresses, source/destination port numbers for UDP or TCP protocols, VLAN identifier. For selector filters, a route can be TWAMP, loopback, or none.

CPU filters allow specifying destination port numbers for UDP or TCP protocols. Packets containing the specified port numbers will not be redirected backward by loopback and will be sent to the CPU for further processing.

2.3.6. Loaned IP

The “Loaned IP” function saves address space by assigning the probe the IP address of the device to which it is connected.

2.3.7. ITU-T Y.1564

Recommendation ITU-T Y.1564 defines a test methodology that may be used in assessing the proper configuration and performance of an Ethernet network to deliver Ethernet-based services.

Smart SFP supports a traffic generator and analyzer and can test up to four services at a time. Y.1564 is only supported on the optical port (A).

2.3.8. TWAMP Light

The “TWAMP Light” function is implemented in accordance with RFC 5357 Appendix I - TWAMP Light. It implements the functionality of the TWAMP Light Session-reflector. The current software version does not support integrity protection, authentication, and encryption.

Regardless of the time synchronization protocol used, PTP or NTP, hardware insertion of timestamps in TWAMP test packets is supported.

Configuration of the “TWAMP Light” function is performed through the selector filter by setting the “route” parameter equal to “twamp”. Also, the “loopback” service must be enabled for the TWAMP Light Session-reflector to work.

2.3.9. PTPv2 (IEEE 1588) and NTP

PTPv2 synchronization is possible through both ports (A and B). Smart SFP can be a Boundary Clock (master and slave) and an Ordinary Clock (slave). Hardware support for PTP has been implemented.

NTP synchronization is supported and possible through both ports (A and B).

2.3.10. Traffic capturing and mirroring

The traffic capture and mirroring function allow to capture transit packets passing through Smart SFP ports and send these packets to a specified IP address using the ERSPAN protocol.

A PC with Wireshark or specialized software designed for in-depth traffic analysis can be used as the receiving host. This function can be used for remote diagnostics of network communication without the need for an engineer visit.

Traffic capture and mirroring function supports:

- Capture received and transmitted packets on ports A and B
- Select the direction of the captured traffic:
 - packets received from the network to the port (RX), with the ability to capturing by filter (see section 2.3.10.1)
 - packets transmitted to the network from the port (TX), capturing only all packets
- Capturing all packets, including packets with errors: invalid CRC, packet size less than 64 bytes, more than 9600 bytes
- Defining the duration of the packet capturing in seconds, by default, the capturing duration is 5 seconds
- Capturing traffic with a total speed up to 2 Gbps on one port (RX + TX) or two ports (RX + TX, RX + RX, TX + TX)
- Mirroring all captured packets, except for generated ERSPAN packets
- Mirrored packets are encapsulated and sent to a user-specified IP address using the ERSPAN Type III protocol (version 2)
- Captured packets are sent through one of the ports, A or B, regardless of which port was captured
- Hardware insertion of timestamps in the Timestamp field of the ERSPAN header. In the case of RX traffic, a label is inserted that corresponds to the time when the packet was received on the Smart SFP port; in the case of TX traffic, a label is inserted that corresponds to the time when the packet was transmitted from the port.
- The speed of captured packets sending is up to 1 Gbps if no other traffic is transmitted through the channel
- The setting of MTU for the ERSPAN package
- Reducing the size of the original package:
 - If the size of the generated ERSPAN packet, including the Ethernet, IP, GRE, and ERSPAN (minimum 54 bytes) headers and the encapsulated original packet, is larger than the specified MTU, then the packet size is reduced to the specified value. Reducing the size of the packet occurs by discarding the required number of bytes from the end of the original packet;
 - The maximum size of a mirrored packet including ERSPAN headers is 9600 bytes, excluding ERSPAN headers – 9538 bytes;
 - Packet size reducing option is enabled by default.

2.3.10.1. Filtering features of captured traffic

- It is possible to filter only packets received on the port (RX)
- Number of filters – 2
- List of fields in the checking condition for each filter:
 - VLAN ID
 - Protocol type
 - Destination IP address
 - Source IP address
 - TCP / UDP destination port
 - TCP / UDP source port
 - Enable/disable packets with errors capturing: packet size less than 64 bytes, more than 9600 bytes, invalid CRC
- If a packet does not contain one of the filter fields, this packet does not match the filter condition and will not be captured;
- Fields not specified in the filter will be ignored.

The traffic capture and mirroring function allow viewing statistics showing the actual number of captured and sent packets over the ERSPAN protocol.

2.3.11. Service activation test with interval statistics

This test is used to evaluate the parameters of the data transmission network. The test results are divided by time into equal segments – intervals. The following statistics are collected for each interval:

- On generator:
 - The number of transmitted frames
 - The number of transmitted bytes (L2)
 - Generation rate (L1-L4) – number of bits per interval or interval duration
- On analyzer:
 - The number of received frames
 - The number of received bytes (L2)
 - FTD (Frame Transfer Delay) – min/max/avg
 - Packet Jitter in accordance with RFC 3550 – min/max/avg
 - TOS cnt – number of frames with changed ToS during transmission
 - OOP – Out Of Order Packet
 - Number of frames with errors – frame with error is a frame with an invalid FCS that was identified by the L2-L4 headers
 - Number of lost frames
 - Rate (L1-L4) – number of bits per interval or interval duration

Apart from these statistics, additional information about each interval is available on the generator and analyzer. For more information, see the output of the commands “show wlt-tx results local profile (0 | 1)” and “show wlt-rx results local profile (0 | 1)”.

The interval size is set by the test settings; the default is 10 s. Within a single test run, the interval duration is always constant and is configured before the test starts. Integral statistics are also collected during one test session. Integral statistics – analyzer statistics for one test session, not tied to intervals. The integral statistics at each point in time is equal to the sum of the interval statistics for all past intervals for:

- bytes
- packets
- losses

The integral statistics can differ from the sum of the interval statistics for:

- packet jitter;
- ToS errors;
- OOP;
- frames with errors
- rate

Integral statistics:

- On generator:
 - The number of transmitted frames
 - The number of transmitted bytes (L2)
 - Generation rate (L1-L4)
- On analyzer:
 - The number of received frames – the sum of frames received for all intervals completed at reception
 - Total number of received bytes (L2) - the sum of bytes received for all intervals completed at reception
 - FTD (Frame Transfer Delay) – frame delay (min/max/avg) calculated separately from interval statistics. It may not be equal to the minimum, maximum, average value for all intervals, for example, due to a situation with receiving a frame later WAIT TIME seconds after the end of the interval.
 - Packet Jitter based on RFC 3550 – packet jitter (min/max/avg) calculated separately from interval statistics. It may not be equal to the minimum, maximum, average value for all intervals, for example, due to a situation with receiving a frame later WAIT TIME seconds after the end of the interval.
 - TOS cnt – number of frames with changed ToS during transmission. Calculated separately from interval statistics and can differ from the sum over all intervals.
 - Out Of Order Packet (OOP) – calculated separately from interval statistics and can differ from the sum over all intervals
 - Number of frames with errors – frames with CRC error that match the filters. Calculated separately from interval statistics and can differ from the sum over all intervals.
 - Number of lost frames – calculated as the sum of lost frames for all intervals
 - Rate (L1-L4)

Apart from these statistics, other additional information about the generator and analyzer is available. For more information, see the output of the commands “show wlt-tx results total profile (0 | 1)” and “show wlt-rx results total profile (0 | 1)”.

When testing, it should be noted that:

- NTP or PTP synchronization is required between generator and analyzer
- The channel under test should be entirely reserved for the interval load test
- Bidirectional test implemented as two independent unidirectional tests
- The maximum test duration is 1 hour
- Currently, the test can only be run on port A

2.3.12. Service of managing additional network stack settings

The service allows to manage filtering of multicast traffic passing through Smart SFP. The ability to enable/disable the transmission of multicast packets to the CPU is required for some services, such as PTP.

By default, transmission of all multicast traffic to the CPU is disabled.

2.3.13. Timestamp insertion

The timestamp insertion function allows to search for a specified sequence of 8 bytes in packets passing through the device and insert a timestamp in NTP Timestamp Format into the packets instead of this sequence. Depending on the settings, the timestamp may contain the time the packet was received or the time it was sent. After inserting the timestamp, the data is sent to the network.

2.3.14. Monitoring and control of Smart SFP signals

The function allows to view information about the TX_FAULT and RX_LOS signals' current state and masking the signals. By default, signal masking is enabled.

2.3.15. Remote management

Smart SFP supports remote management:

- REST API
- CLI via SSH
- VLAN-based

2.4. Main features of Smart SFP 1G RS-232

Smart SFP 1G RS-232 can have following feature sets:

Table 2.2. Smart SFP 1G RS-232 feature sets

Feature set number	01	02	03
One RS-232 port with DSR, DCD, DTR, CTS, RTS circuits support	+	+	-
Three RS-232 ports without DSR, DCD, DTR, CTS, RTS circuits support	-	-	+
IEC 60870-5-101 (IEC-101) transfer over Ethernet/IP networks	-	+	-
RS-232 transfer over Ethernet/IP networks	+	-	+
RS-232 terminal server function	+	-	+
PTPv2 (IEEE 1588) and NTP	+	+	+
Remote management	+	+	+

2.4.1. IEC 60870-5-101 transfer over Ethernet/IP networks

Smart SFP 1G RS-232 can transfer IEC 60870-5-101 (IEC-101) data over Ethernet/IP networks in Unbalanced mode. It is possible to set up “point-to-point” or “point-to-multipoint” operation.

The functionality of IEC 60870-5-101 transmitting over Ethernet/IP networks is described for software version 1.1.16.

2.4.2. RS-232 transfer over Ethernet/IP networks

Smart SFP 1G RS-232 can transfer RS-232 data over Ethernet/IP networks. It is possible to set up “point-to-point” or “point-to-multipoint” operation. You can use any available on Debian Linux utilities for this purpose, such as ser2net and socat.

2.4.3. RS-232 terminal server function

Smart SFP 1G RS-232 can operate as RS-232 terminal server. You can use any available on Debian Linux utilities for this purpose, such as minicom.

2.4.4. PTPv2 (IEEE 1588) and NTP

PTPv2 synchronization is possible through port B. Smart SFP can be a Boundary Clock (master and slave) and an Ordinary Clock (slave). Hardware support for PTP has been implemented.

NTP synchronization is supported and possible through port B.

2.4.5. Remote management

Smart SFP supports remote management:

- CLI via SSH
- VLAN-based

2.5. Main features of Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232

This User guide describes functions of Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232 running software version 1.1.6.

Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232 modules perform the following functions:

- Time synchronization from GNSS
- PTPv2 (IEEE 1588) and NTP precision time server
- Remote management

2.5.1. Time synchronization from GNSS

The module determines the precise time using the global navigation satellite system (GNSS).

When using the Smart SFP 1G GNSS, a GNSS antenna should be connected to the built-in GNSS receiver. The antenna connects to an SMA(f) connector. The 1PPS signal can be transmitted or received via an MMCX(f) connector.

When using the Smart SFP 1G GNSS-RS232 communication with the GNSS receiver is established using port A with an RS-232 interface. The RD signal is not required for the receiver to operate. It is only necessary when receivers operating mode needs to be reconfigured. The 1PPS signal line is used to receive 1PPS pulses from the GNSS receiver. Power for the GNSS receiver is supplied via the +5V and GND lines.

The minicom utility is used to check the connection to the receiver. To switch the GNSS receiver to GLONASS mode and check for satellite availability, utilities like gpsd, ubxtool, cgps, and gpstcl are used.

The ts2phc utility receives NMEA messages and the timestamp of the last 1PPS signal edge from a GNSS receiver. Using this data, ts2phc calculates once per second how much the PHC hardware clock lag/fast and adjusts it. The phc2sys utility synchronizes the system clock from the hardware clock.

2.5.2. PTPv2 (IEEE 1588) and NTP

Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232 modules function as a precision time server supporting PTPv2 (IEEE 1588) and NTP protocols. To do this, the module must receive the precise time using the global navigation satellite system (GNSS).

For PTP time server operation, utilities such as `ts2phc` and `ptp4l` are used. The `ts2phc` utility adjusts the PHC hardware clock. The PHC hardware clock is used by the network interface to determine the precise time for receiving and sending PTP packets. The `ptp4l` utility reads time from the PHC hardware clock and transmits it to the network via the PTP protocol, functioning as a time server.

For NTP time server operation, the utilities `ts2phc`, `phc2sys`, and `ntp` are used. The `ts2phc` utility adjusts the PHC hardware clock. The `phc2sys` utility synchronizes the system clock with the PHC hardware clock. The `ntp` utility uses the system clock to operate as an NTP server.

Publicly available documentation should be used for configuring these utilities.

2.5.3. Remote management

Smart SFP supports remote management:

- CLI via SSH
- VLAN-based

2.6. Main features of Smart SFP 1G USB

This User guide describes functions of Smart SFP 1G USB running software version 1.0.104.

Smart SFP 1G USB has following features:

- PTPv2 (IEEE 1588) and NTP precision time server
- Remote management

2.6.1. USB devices connection

Smart SFP 1G USB functions as a universal network appliance with a USB port. Various USB devices can be connected to it.

The module runs the Debian OS, which allows installing utilities and drivers to support different USB devices and solve a variety of tasks using them.

2.6.2. PTPv2 (IEEE 1588) and NTP

PTPv2 synchronization is possible through port B. Smart SFP can be a Boundary Clock (master and slave) and an Ordinary Clock (slave). Hardware support for PTP has been implemented.

NTP synchronization is supported and possible through port B.

2.6.3. Remote management

Smart SFP supports remote management:

- CLI via SSH
- VLAN-based

3. Technical specs

3.1. General

Table 3.1. Common specifications

Parameter	Description
Manual Switches	2 x DIP Switches
CPU	ARM, 1 core. 528 MHz
RAM	512 MB
Nonvolatile storage memory	4GB eMMC
OS	Debian Linux
Indication	1×LED
Weight	35 g
Power consumption	1.4-1.8 W, depending on use case

Table 3.2. Specification of Smart SFP 1G fiber

Parameter	Description
Interface	1000BASE-LX
Data Rate	1.25 Gb/s
Wavelength	1310, 1550, 1550/1310 or 1310/1550 nm (depending on modification)
Cable Type	SMF
Max Distance	20 km
Tx/Rx type	FP/PIN
Transmitter Power	-4.5 ~ -3.5 dBm
Receiver Sensitivity	< -19 dBm
Receive Overload	-3 dBm
Connector	LC
Digital Diagnostic Monitoring	Yes
Dimensions	W: 13.9 mm / H: 12.3 mm / L: 79.7 mm

Table 3.3. Specification of Smart SFP 1G copper

Parameter	Description
Interface	10/100/1000BASE-T
Data Rate	1 Gb/s
Cable Type	Cat. 5e, Cat. 6
Max Distance	100 m
Connector	RJ-45
Digital Diagnostic Monitoring	No
Dimensions	W: 13.9 mm / H: 15.8 mm / L: 79.7 mm

Table 3.4. Specification of Smart SFP 1G RS-232

Parameter	Description
Interface for connecting to network equipment	1000BASE-X
Interface for terminal equipment connection	RS-232
Data rate of the terminal equipment connection interface	110..115200 b/s
Max Distance	15 m
Connector	RJ-45
Digital Diagnostic Monitoring	Yes ²
Dimensions	W: 13.9 mm / H: 15.8 mm / L: 79.7 mm

² To improve compatibility with various network equipment, DDMI emulation is implemented: the module's actual voltage and temperature values are displayed, while the remaining parameters show fixed values to prevent false alarm messages.

Table 3.5. Specification of Smart SFP 1G GNSS

Parameter	Description
Interface for connecting to network equipment	1000BASE-X
Connector for GNSS antenna connection	SMA(f)
GNSS antenna supply voltage	+3.3 V
GNSS antenna maximum current	50 mA
1PPS input/output connector	MMCX(f)
1PPS signal output level	2.0-2.3 V
1PPS signal input level	3,6-5,0 V
Matched resistance of connectors	50 ohm
Digital Diagnostic Monitoring	Yes ²
Dimensions	W: 13.9 mm / H: 15.8 mm / L: 79.7 mm

Table 3.6. Specification of Smart SFP 1G GNSS-RS232

Parameter	Description
Interface for connecting to network equipment	1000BASE-X
Data interface for GNSS receiver connection	RS-232 without Flow Control support
Data rate of the interface for GNSS receiver connection	110..115200 b/s
Signals of the data interface for GNSS receiver connection	RD OUT, TD IN
Synchronization signal of the GNSS receiver connection	1PPS
Synchronization signal level	According to RS-232 standard
GNSS receiver power supply voltage	+5 V
Max Distance	15 m
Connector	RJ-45
Digital Diagnostic Monitoring	Yes ²
Dimensions	W: 13.9 mm / H: 15.8 mm / L: 79.7 mm

Table 3.7. Specification of Smart SFP 1G USB

Parameter	Description
Interface for connecting to network equipment	1000BASE-X
Interface for terminal equipment connection	USB 2.0
Connector	USB type A
Max Distance	5 m
Digital Diagnostic Monitoring	Yes
Dimensions	W: 13.9 mm / H: 15.8 mm / L: 79.7 mm

3.2. Bandwidth

This section contains data on the throughput of Smart SFP 1G fiber and copper in the bridge mode.

Table 3.1. Smart SFP 1G fiber bandwidth

Frame size	Number of frames per second	Number of bits per second at the 2nd OSI layer
64 bytes	1488094	761904128
1518 bytes	81274	986991456
9600 bytes	12993	997862400

Table 3.2. Smart SFP 1G copper bandwidth

Frame size	Number of frames per second	Number of bits per second at the 2nd OSI layer
64 bytes	1488095	761904640
1518 bytes	81274	986991456
9600 bytes	12993	997862400

3.3. Module part numbers

Table 3.8. Module order codes

Module type	Part number
Smart SFP 1G fiber 1310 nm, 20 km, 2xSMF, LC	ISM4120A
Smart SFP 1G fiber Tx: 1310 nm, Rx: 1550 nm, 20 km, SMF, LC	ISM4130A
Smart SFP 1G fiber Tx: 1550 nm, Rx: 1310 nm, 20 km, SMF, LC	ISM4140A
Smart SFP 1G fiber 1550 nm, 80 km, 2xSMF, LC	ISM4150A
Smart SFP 1G copper	ISM4100B
Smart SFP 1G RS-232	ISM4100J
Smart SFP 1G GNSS	ISM4100N
Smart SFP 1G GNSS-RS232	ISM4100L
Smart SFP 1G USB	ISM4100P

3.4. Operating conditions

Table 3.9. Operating conditions

Parameter	Description
Operating temperature	-25...+85 °C
Storage temperature	-40...+85 °C
Relative humidity	20-85%, non-condensing

Taking into account the fact that the module is not a self-sufficient device and is operated as part of other equipment, the specified operating temperature range of the module does not indicate the range of ambient temperatures at which the module operates correctly. This range indicates the temperature values at which the module components operate correctly.

The ambient temperature range is not specified, because the module temperature may differ significantly from the ambient temperature because the module temperature is affected by many factors, such as the type of cooling in the room, the type of equipment where the module is installed, the type and design of cooling in the equipment, the location of the equipment in the rack, the presence or absence of neighboring SFP modules, the current temperature of neighboring SFP modules, etc.

3.5. External Smart SFP view

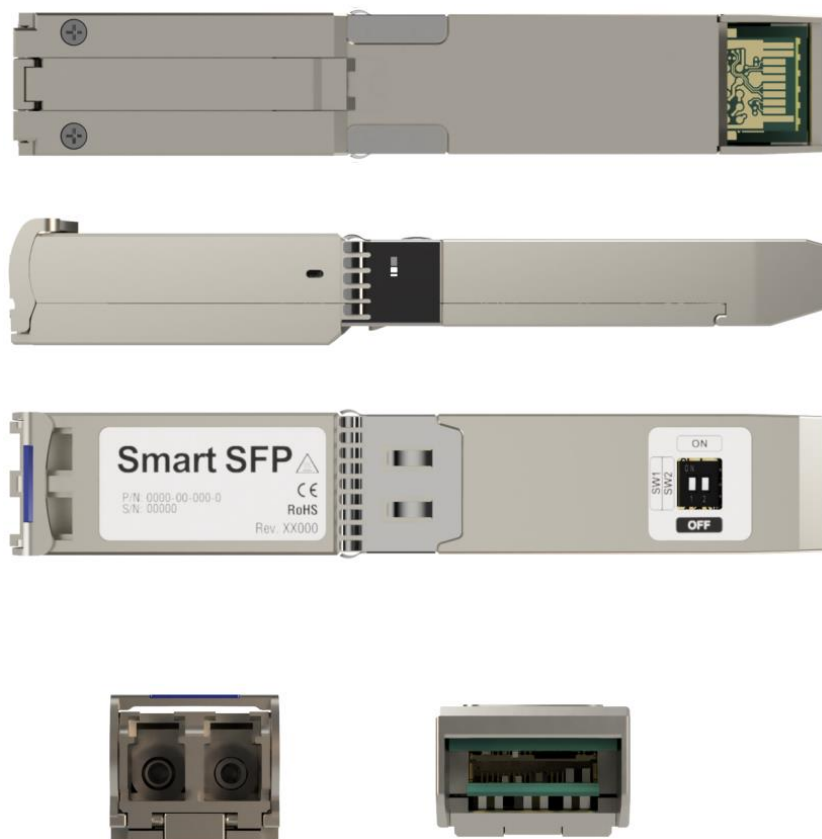


Figure 3.1. Smart SFP 1G fiber appearance

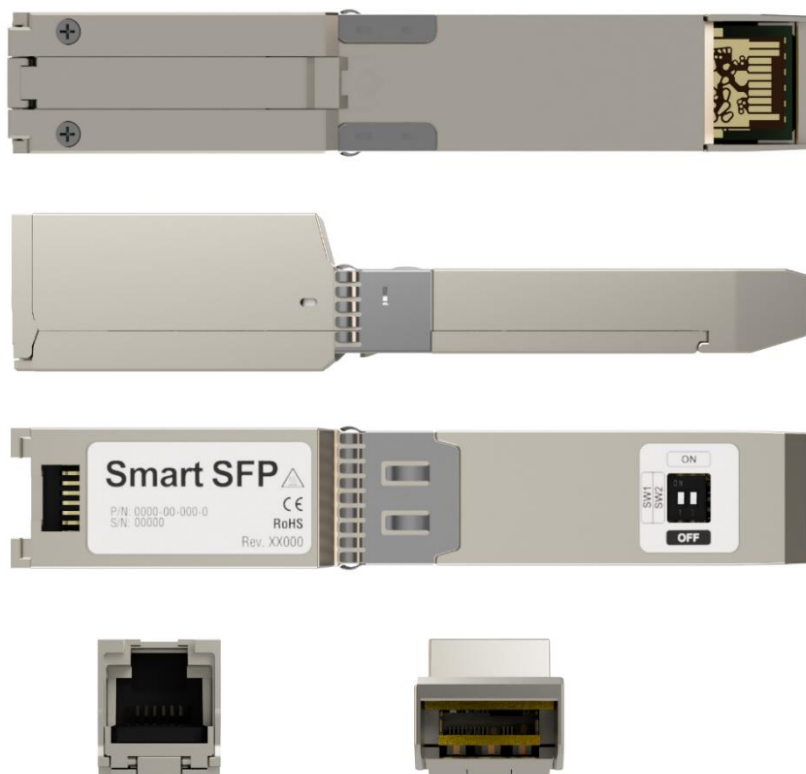


Figure 3.2. Smart SFP 1G copper, Smart SFP 1G RS-232 and Smart SFP 1G GNSS-RS232 appearance

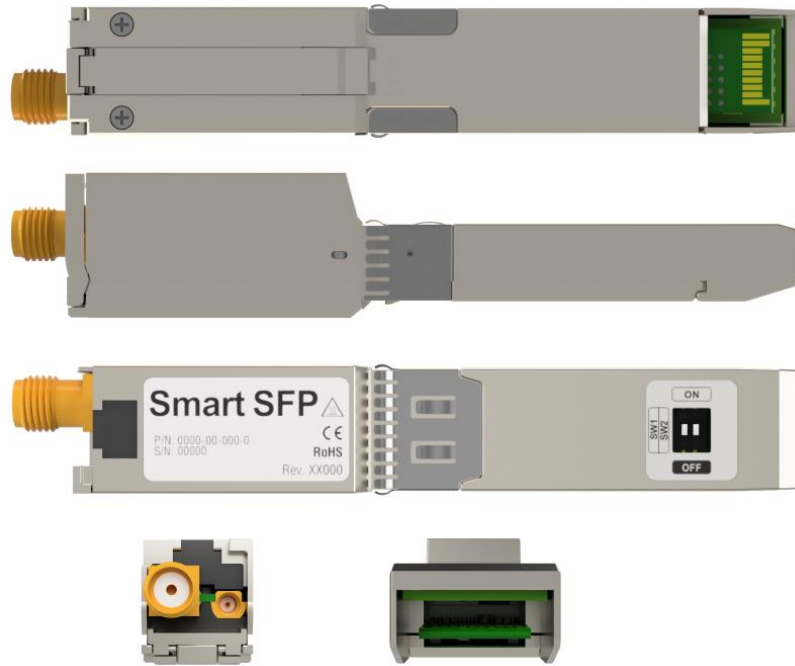


Figure 3.3. Smart SFP 1G GNSS appearance

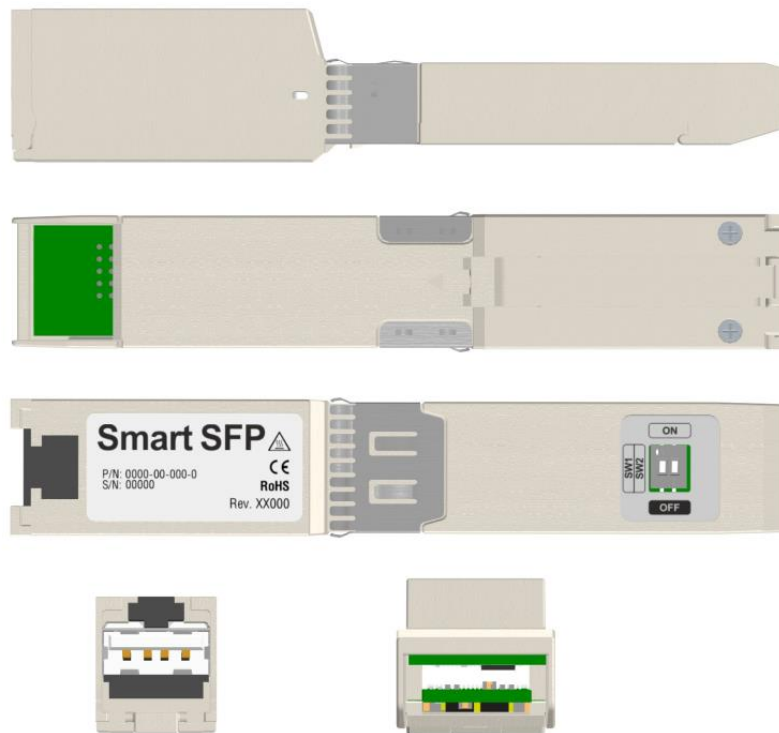


Figure 3.4. Smart SFP 1G USB appearance

3.6. Internal block diagram

3.6.1. Internal block diagram of Smart SFP 1G fiber and Smart SFP 1G copper

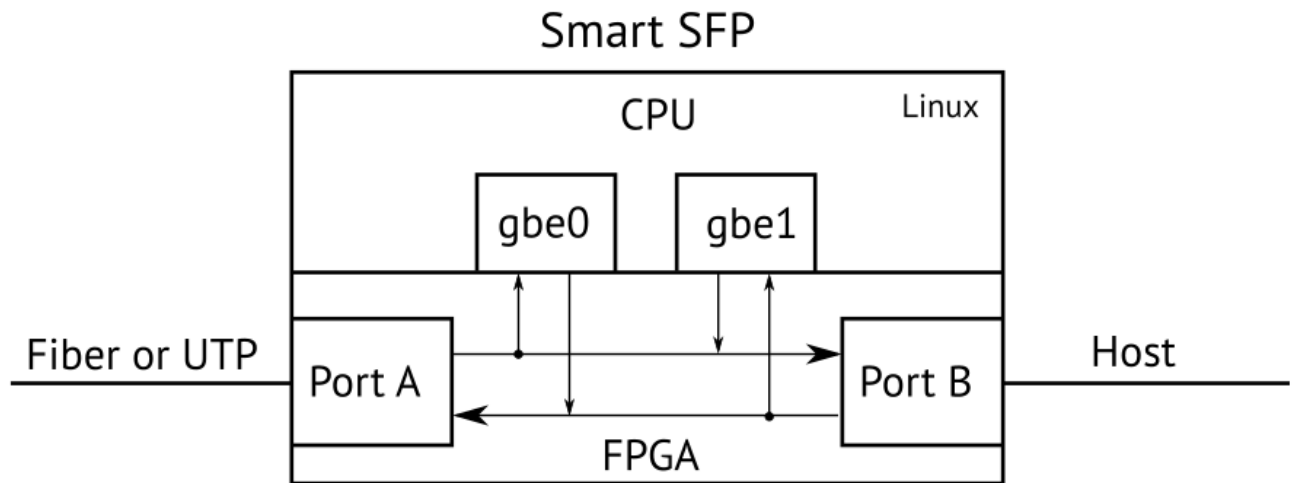


Figure 3.5. Smart SFP 1G fiber and Smart SFP 1G copper internal block diagram

3.6.2. Internal block diagram of Smart SFP 1G RS-232

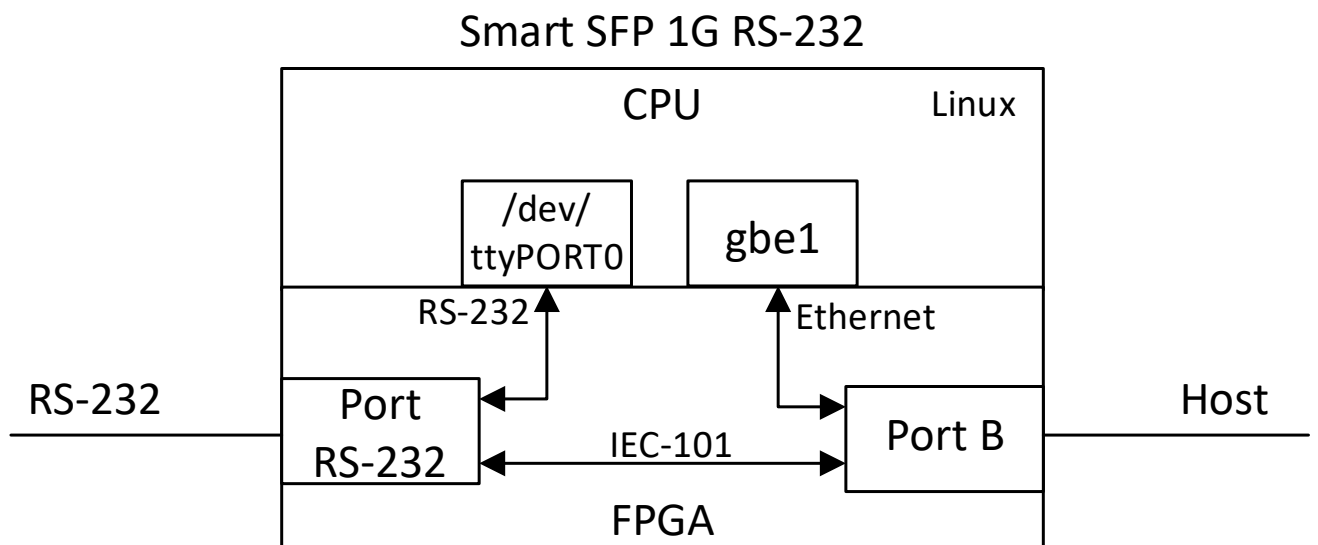


Figure 3.6. Smart SFP 1G RS-232 internal block diagram

3.6.3. Internal block diagram of Smart SFP 1G GNSS

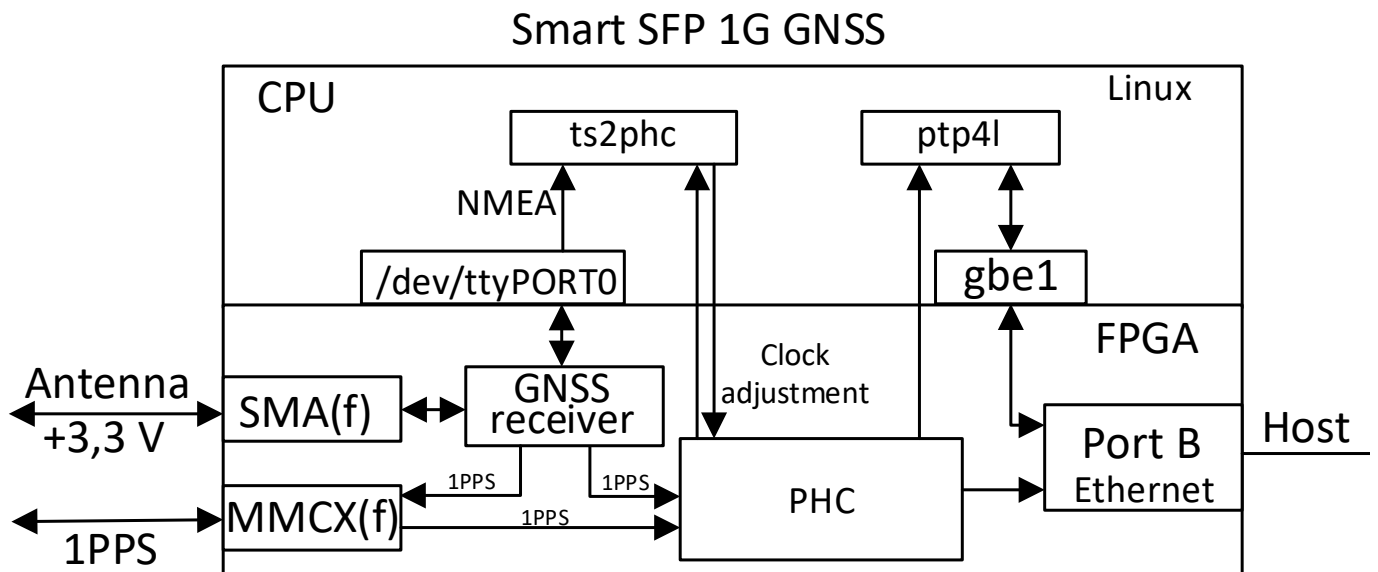


Figure 3.7. Smart SFP 1G GNSS internal block diagram

3.6.4. Internal block diagram of Smart SFP 1G GNSS-RS232

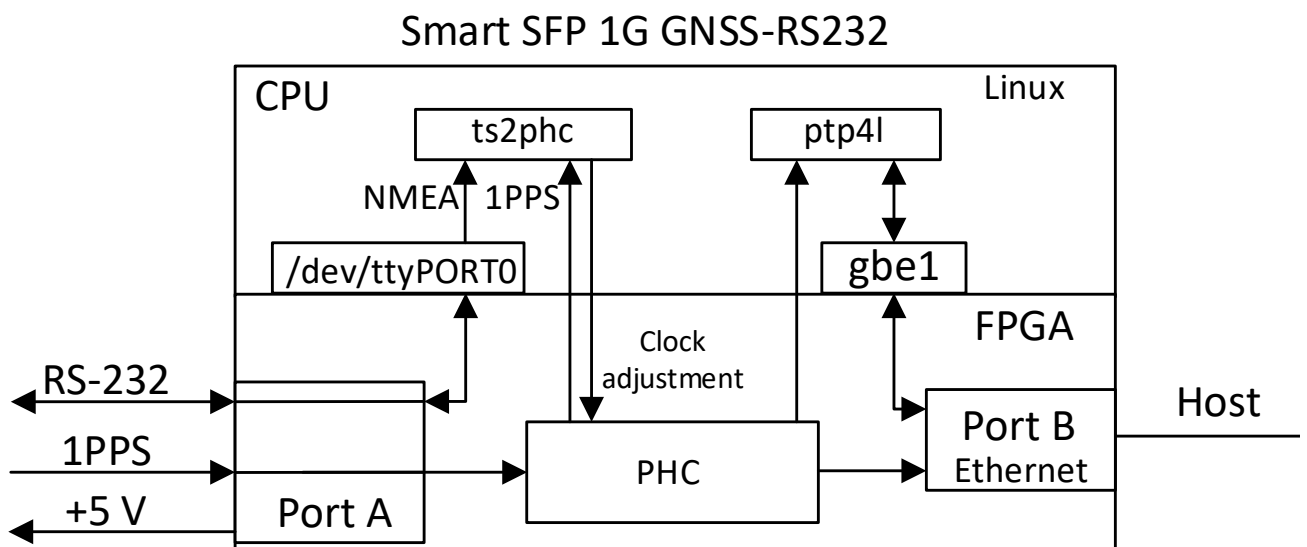


Figure 3.8. Smart SFP 1G GNSS-RS232 internal block diagram

3.6.5. Internal block diagram of Smart SFP 1G USB

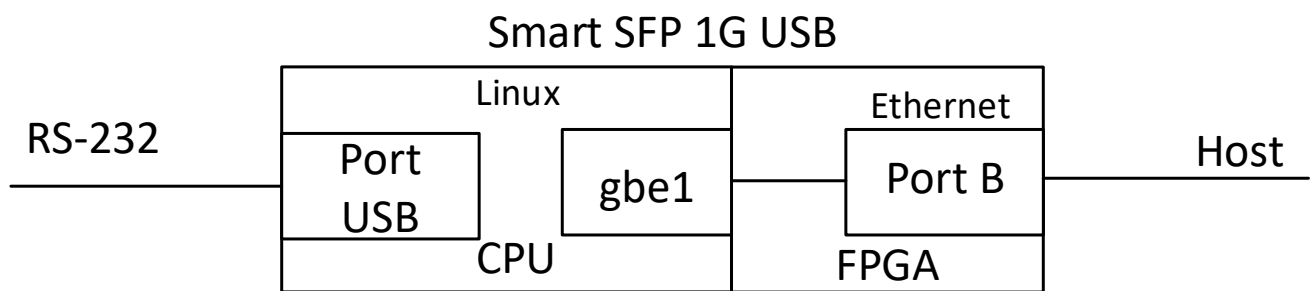


Figure 3.9. Smart SFP 1G USB internal block diagram

3.7. DIP switch

Smart SFP includes a two-part DIP switch on its upper part, used to select the device’s operating mode.

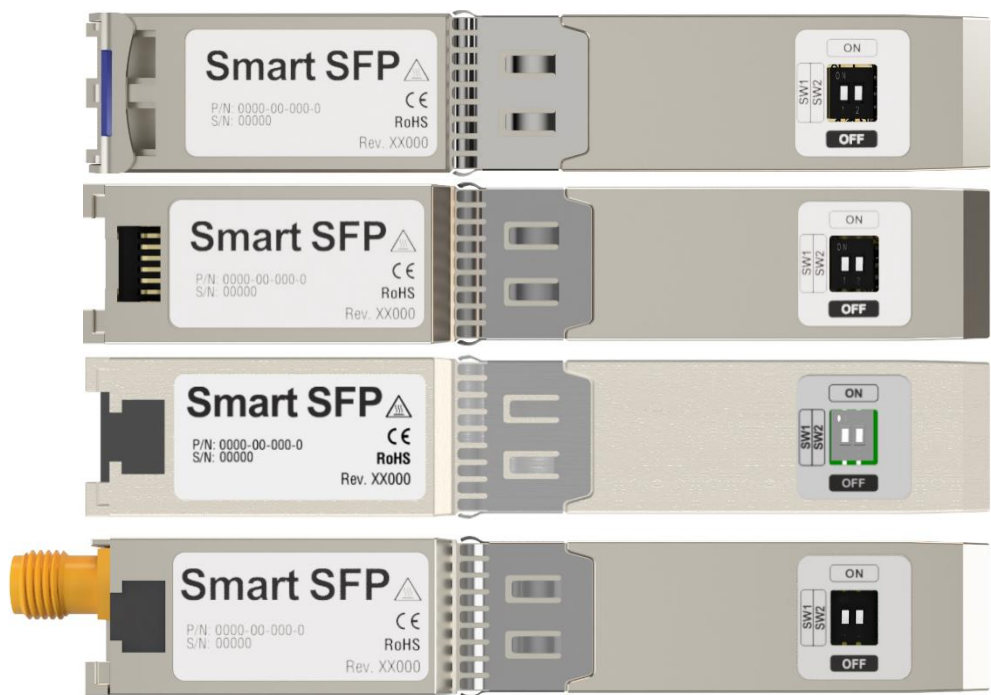


Figure 3.10. Smart SFP DIP switch

Table 3.10. DIP switch positions and operating modes

	SW1	SW2
Regular mode	ON	ON
Recovery mode	ON	OFF
Not used	OFF	ON
Not used	OFF	OFF

3.8. LED

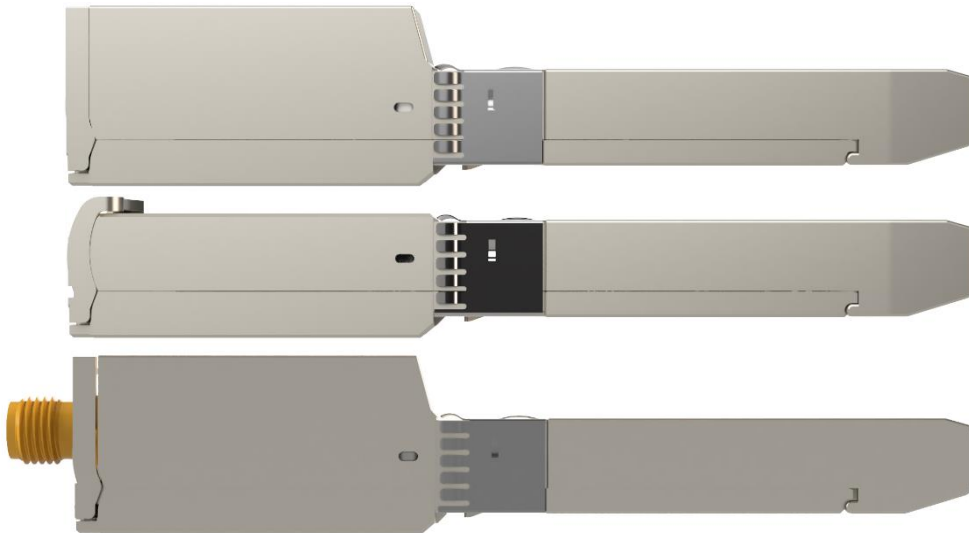


Figure 3.11. Smart SFP State LED

Smart SFP LED indicator State is located on the side. LED color depends on the operating mode and the device state:

1. Regular mode:
 - After Smart SFP is inserted into the host device, the LED indicator blinks during software initialization for about ten seconds
 - After software initialization, the indicator lights up green
 - If an error occurs, the indicator lights up red.
2. Recovery mode:
 - Starting service firmware: does not light for about 10 seconds, then blinks green
 - Writing software image to ROM: flashes red for about 15 minutes
 - Constant color – writing is completed, you can eject Smart SFP from the host equipment. “Green” indicates successful completion of the writing process, “red” indicates an error during the writing process.

4. Delivery set

Smart SFP delivery set includes:

- Smart SFP
- Quick start guide
- Packing box

Packing box size – 120×75×25 mm. Packing box weight with Smart SFP – 58 g.

5. Software structure

The device is running under Debian Linux. Standard OS commands and configuration files are used to configure network and system parameters. For example, network interfaces configuration is performed using the “ifconfig” command. If you want to save the settings after a reboot, change the file “/etc/network/interfaces.d/gbe”. Detailed information about commands usage can be obtained through the built-in help system or at www.debian.org.

“Loopback”, “Statistics”, “Filter Statistics”, “Y.1564” and other functions listed in section 2.3 “Main features of Smart SFP 1G copper and Smart SFP 1G fiber” are used to diagnose the network. To configure them, use the vendor-specific command-line interface or REST API. Changes made using vendor-specific CLI or REST API are automatically saved when the device is rebooted using the “reboot” command.

Also, Debian allows to install of standard Linux packages on the Smart SFP. For that:

1. Remount the Smart SFP file system with write permissions:

```
mount -o rw,remount /
```

2. Install standard software.

3. After the installation of the software is completed, the file system should be remounted with read-only rights:

```
mount -o ro,remount /
```

6. Installation and setup

6.1. Inserting Smart SFP into a Host Device

You need to insert Smart SFP into the host device to configure it via the host or operate it. Smart SFP is a hot-swappable device – you do not have to switch off the host unit when inserting or extracting it.

Note. Smart SFP is not recommended to be installed in adjacent SFP slots, as it may become very hot during operation.

To set in the in-line mode, it is necessary to perform all steps, to set in the out-of-line mode (end host) only the 1st:

1. Insert Smart SFP into an MSA-compatible SFP/SFP+ socket of the host equipment. Press Smart SFP firmly into the SFP socket until Smart SFP clicks into place.
2. Connect cable.
3. Smart SFP is ready to operate.

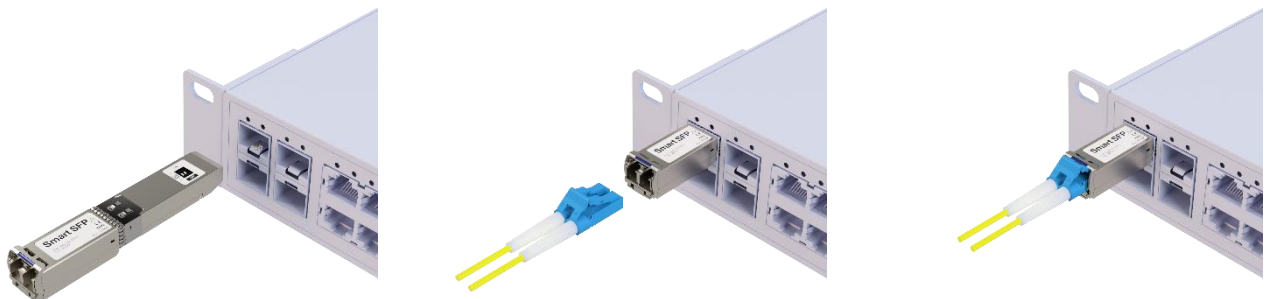


Figure 6.1. Inserting Smart SFP

To eject Smart SFP (see Figure 6.2):

1. Disconnect any cables attached to Smart SFP.
2. Push the release latch on Smart SFP's underside in the direction of the host device.
3. While continuing to keep pressure on the latch, pull Smart SFP gently out of the socket. Release the latch after you have completely removed Smart SFP from the socket.

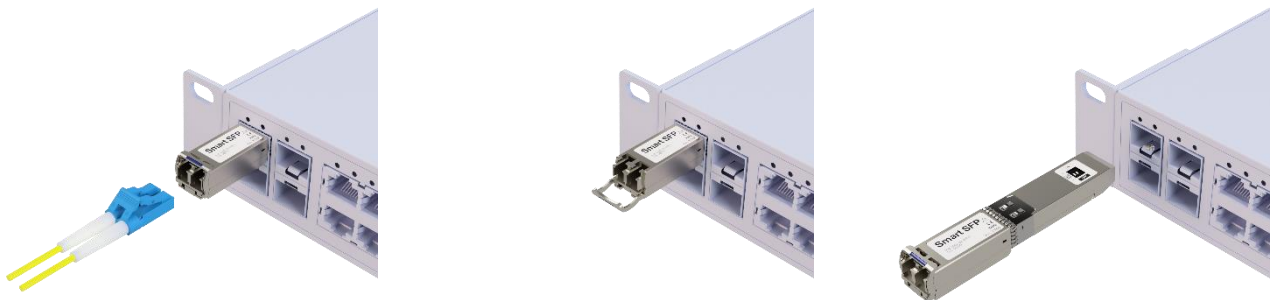


Figure 6.2. Ejecting Smart SFP

6.2. IP interfaces

- Users can get access to the device using the SSH protocol.
- USB OTG implements serial console (RS-232) and network interface:
 - the console is typically detected in an external Linux system as `/dev/ttyACMx`
 - the network interface is named “usb0” and has IPv4 address 192.168.3.1 by default.
- Interfaces A (SFP) and B (HOST) are detected in the device’s Linux system as `gbe0` and `gbe1` correspondingly on Smart SFP 1G copper and Smart SFP 1G fiber. There is only interface and B (HOST) detected as `gbe1` available on Smart SFP 1G RS-232, Smart SFP 1G GNSS and Smart SFP 1G USB.
 - `gbe0` interface has IPv4 address 192.168.1.1 by default
 - `gbe1` interface has IPv4 address 192.168.2.1 by default
- All interfaces can get IPv4 addresses by DHCP protocol. The address can be obtained according to the configuration file in `/etc/network/interfaces.d/gbe` or the “dhclient” utility.
- The IP address 192.168.3.1 is assigned by default to the `usb0` hidden interface of Smart SFP, which supports Ethernet over USB mode when connecting Smart SFP to a PC via the OTG port of the service tool.

```
usb0      Link encap:Ethernet  HWaddr 00:21:ce:3c:00:14
          inet addr:192.168.3.1  Bcast:192.168.3.255  Mask:255.255.255.0
```

6.3. Default accounts

There are three different types of user accounts on the device: root, admin, user.

Table 6.1. Default accounts

Login	root	user	admin
Password default	PleaseChangeTheRootPassword	PleaseChangeTheUserPassword	PleaseChangeTheAdminPassword
Description	Standard Linux account with all rights and permissions	Standard Linux account	Standard Linux account
Shell default	/bin/bash	/bin/bash	/bin/bash
SSH	Key-based authentication only	Key-based and password authentication	Key-based and password authentication
Vendor-specific command-line interface	Full rights to change the state of the device when working with CLI	Full rights only to view the current settings when working with CLI	Full rights to change the state of the device when working with CLI

6.4. Connect to Smart SFP

6.4.1. SSH

To connect to Smart SFP via SSH, you must first configure the IP address of the personal computer (PC) so that the PC and Smart SFP are on the same subnet. You can connect via port A (see Figure 7.1) and port B (see Figure 7.3).

6.4.1.1. Windows

1. Connect Smart SFP according to the scheme in Figure 7.1 (for port A) or Figure 7.3 (for port B).
2. Open a terminal program with SSH support (for example, PuTTY or HyperTerminal) on a PC.
3. Set the IP address of the Smart SFP port through which you are connecting and open the connection:

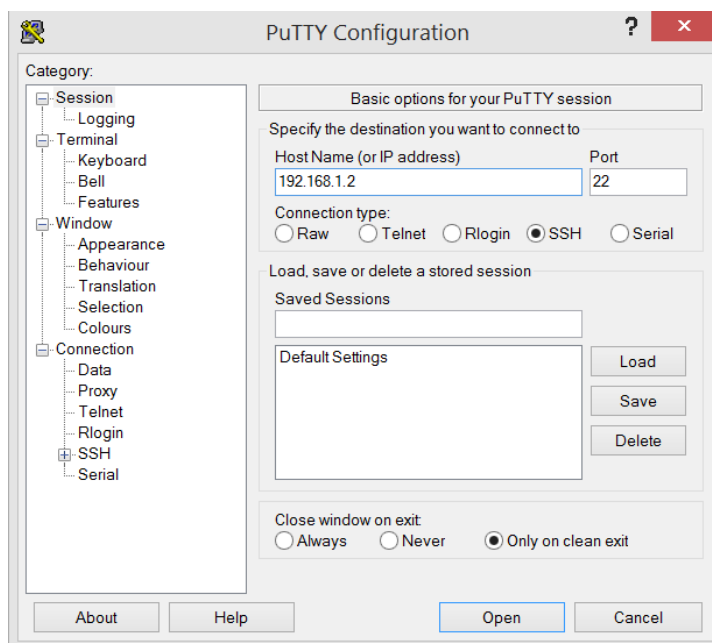


Figure 6.3. SSH connection via port A

4. Enter a username (“admin” or “user”) and password (see section 6.3).

```
Login as: admin
admin@192.168.1.2's password:
Linux smart-sfp 5.10.59-probe-1.9.0 #1 SMP PREEMPT Wed Jun 8 13:16:36 UTC 2022 armv7l
*****
                        Welcome to Smart SFP!
*****
Last login: Tue Apr 16 14:53:34 2024 from 192.168.1.10
admin@smart-sfp:~$
```

Figure 6.4. Successful connection

6.4.1.2. OS Linux

1. Connect Smart SFP according to the scheme in Figure 20 (for port A) or Figure 22 (for port B).
2. Connect from PC via SSH, specifying the account (“user” or “admin”) and the IP address of the port. For example:

ssh admin@192.168.1.2
3. If the connection is successful, the program will prompt you to enter a username and password (see section 6.3).

6.4.2. SSH connection using a key

6.4.2.1. Installing a key on Smart SFP

To connect via SSH using a key, you need a pair: a private and a public key. You can use a pair you already have or generate a new pair on the computer from which connections using the key will be made. To install a public key on a Smart SFP, you need to:

1. Connect to Smart SFP via SSH using the password.
2. Create a folder to store the authorization file:

```
admin@smart-sfp:~$ mkdir .ssh/
```

3. Create a file responsible for authorization and enter the key text into it. The text in quotes should be replaced with the text of your public key in a similar format:

```
admin@smart-sfp:~$ echo "ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIEtsAiu+XtVxIhMR4/WT0jsvvdX32/sT/fy+u03sj7m user@pc" >>
.ssh/authorized_keys
```

6.4.2.2. Connection using a key

To connect using a key, you need to pass the private key to your SSH client:

- when using PuTTY, you need to specify the path to the private key when setting up a connection in the menu Connection -> SSH -> Auth -> Credentials -> Private key;
- when using a standard SSH client in Linux, the path to the private key should be specified using the -i switch:

```
ssh -i /home/user/.ssh/my_private_key admin@192.168.2.1
Linux smart-sfp 5.10.59-probe-1.9.0 #1 SMP PREEMPT Wed Jun 8 13:16:36 UTC 2022 armv7l
*****
                        Welcome to Smart SFP!
*****
Last login: Tue Apr 16 11:25:18 2024 from 192.168.2.212
admin@smart-sfp:~$
```

7. Smart SFP connection schemes

Smart SFP 1G copper and Smart SFP 1G fiber can be connected in two ways:

1. “In-line” – Smart SFP is installed as a regular SFP module, user traffic is transmitted through the probe in transit mode:
 - Access only through the line interface (port A). An IP address for management and testing is assigned to the gbe0 interface (port A).
 - Access via both ports (A and B). Interfaces gbe 0 (port A) and gbe 1 (port B) are combined into a bridge. An IP address for management and testing is assigned to the bridge interface.
2. “Out-of-line” – the line interface, optical or electrical, remains unconnected. No user traffic is transmitted through the probe. Access is only through port B. An IP address for management and testing is assigned to interface gbe1 (port B).

Smart SFP 1G RS-232, Smart SFP 1G GNSS and Smart SFP 1G USB are used with the “out-of-line” configuration. Configuration is only required for the gbe1 interface (port B). The gbe0 interface settings should be skipped because this modification of Smart SFP does not have a gbe0 interface.

7.1. In-line connection

7.1.1. Access through the line interface (port A)

Access to the Smart SFP is only through port A in this connection and configuration option. All user traffic will be transmitted through the probe without any changes in transit mode.



Figure 7.1. In-line connection with management via line interface

To configure this operating mode, you should assign an IP address on the gbe0 interface (port A) and delete or comment the IP address of the gbe1 interface (port B):

1. Connect to the device via SSH using the IP address of the gbe0 interface (port A).
2. Log in as admin and then switch to the root account:

```
admin@smart-sfp:~$ su
```

3. Mount the file system for writing:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

4. Edit the /etc/network/interfaces.d/gbe file as follows:

```
root@smart-sfp:~# nano /etc/network/interfaces.d/gbe
#SFP Port
auto gbe0
iface gbe0 inet static
address <interface ip address>
netmask <subnet mask>

#Host port
auto gbe1
iface gbe1 inet manual
#       address 192.168.2.1
#       netmask 255.255.255.0
```

- To apply the settings, reboot the Smart SFP using the “reboot” command or the command below to apply the settings without rebooting. Attention, if you change the IP address on port A, management of the Smart SFP may be lost:

```
ifdown gbe0 && ifup gbe0 && ifdown gbe1 && ifup gbe1
```

7.1.2. Access via both ports (A and B)

Access to the Smart SFP is via both ports (A and B) in this connection and configuration option. All user traffic will be transmitted through the probe without any changes in transit mode.

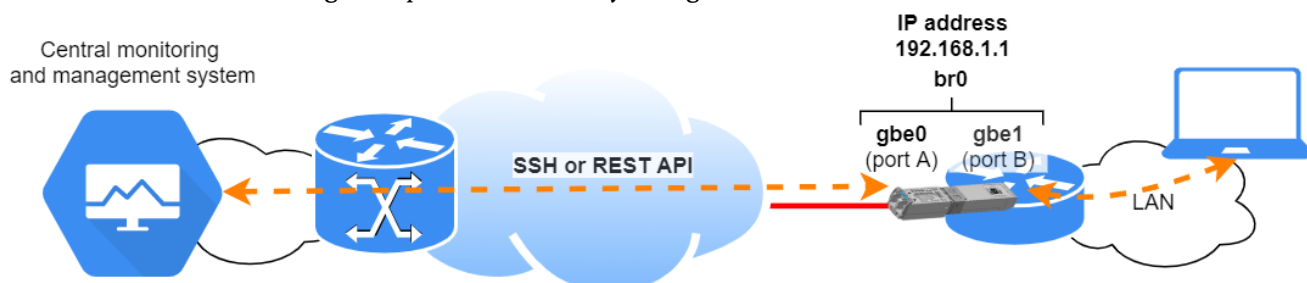


Figure 7.2. In-line connection with management via both ports

To configure this operating mode, it is necessary to combine the gbe0 (port A) and gbe1 (port B) interfaces into a “bridge”. Then, all the probe interfaces, including the new bridge interface, should be assigned the MAC address of one of the probe interfaces:

1. Connect to the device via SSH using the IP address of the gbe0 interface (port A).
2. Log in as admin and then switch to the root account:

```
admin@smart-sfp:~$ su
```

3. Mount the file system for writing:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

4. Look at what MAC addresses are used on the interfaces:

```
root@smart-sfp:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: gbe0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN group default qlen 1000
   link/ether 00:21:ce:44:00:d2 brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.1/24 brd 192.168.1.255 scope global gbe0
       valid_lft forever preferred_lft forever
3: gbe1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN group default qlen 1000
   link/ether 00:21:ce:44:00:d3 brd ff:ff:ff:ff:ff:ff
   inet 192.168.2.1/24 brd 192.168.2.255 scope global gbe1
       valid_lft forever preferred_lft forever
```

5. Use the MAC address 00:21:ce:44:00:d2 of the gbe0 interface (port A) and edit the /etc/network/interfaces.d/gbe file as follows:

```
root@smart-sfp:~# cat /etc/network/interfaces.d/gbe
#SFP Port
auto gbe0
iface gbe0 inet manual
pre-up ifconfig gbe0 hw ether 00:21:ce:44:00:d2

#Host port
auto gbe1
iface gbe1 inet manual
pre-up ifconfig gbe1 hw ether 00:21:ce:44:00:d2

auto br0
iface br0 inet static
address <interface ip address>
netmask <subnet mask>
hwaddress ether 00:21:ce:44:00:d2
bridge_ports gbe0 gbe1
bridge_stp off
post-up /usr/sbin/promiscctl -p a -c off
post-up /usr/sbin/promiscctl -p b -c off
pre-up ebtables -A FORWARD -j DROP
```

6. To apply the settings, reboot the device using the command:

```
reboot
```

7.2. Out-of-line connection

Access to the Smart SFP is only through port B in this connection and configuration option. No user traffic is transmitted through the probe.

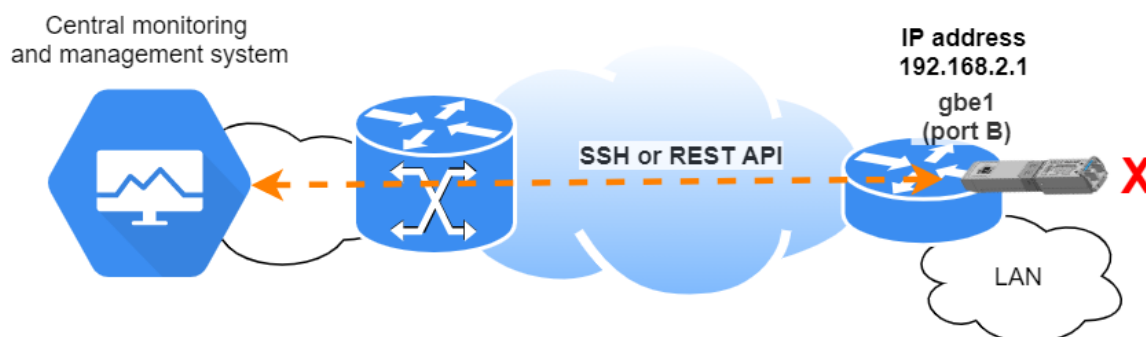


Figure 7.3. Out-of-line connection

To configure this operating mode, you should assign an IP address on the gbe1 interface (port B) and delete or comment the IP address of the gbe0 interface (port A):

1. Connect to the device via SSH using the IP address of the gbe1 interface (port B).
2. Log in as admin and then switch to the root account:

```
admin@smart-sfp:~$ su
```

3. Mount the file system for writing:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

4. Edit the /etc/network/interfaces.d/gbe file as follows:

```
root@smart-sfp:~# nano /etc/network/interfaces.d/gbe
#SFP Port
auto gbe0
iface gbe0 inet manual
#address 192.168.1.1
#netmask 255.255.255.0

#Host port
auto gbe1
iface gbe1 inet static
address <interface >
netmask <netmask>
```

5. To apply the settings, reboot the Smart SFP using the “reboot” command or the command below to apply the settings without rebooting. Attention, if you change the IP address on port B, management of the Smart SFP may be lost:

```
ifdown gbe0 && ifup gbe0 && ifdown gbe1 && ifup gbe1
```

Note. If a Smart SFP with an electrical interface connected out-of-line does not establish a connection with the connected device, you should disable auto-negotiation, set the connection speed to 1000 Mbps (see section 8.1.2) and the transmission mode to duplex. You should make these settings for port B (gbe1) of the Smart SFP and for the port of the device to which the Smart SFP is connected (for example, a switch or router).

8. Changing Smart SFP port settings

This chapter describes configuration of Smart SFP 1G copper and Smart SFP 1G fiber. Only interface B (HOST), detected as gbe1, will be available for configuration on Smart SFP 1G RS-232, Smart SFP 1G GNSS and Smart SFP 1G USB.

8.1. Changing the speed of port A and port B

8.1.1. Automatic speed change on Smart SFP ports

Auto-negotiation is enabled by default on Smart SFP. If you connect the Smart SFP to the port where a certain speed will be set, then you do not need to make additional settings on the Smart SFP. Auto-negotiation will automatically set the network speed.

1. Connect to the device via SSH using the IP address of the gbe0 interface (port A) or gbe1 interface (port B).
2. Log in as admin and then switch to the root account:

```
admin@smart-sfp:~$ su
```

There are two ways to enable auto-negotiation on Smart SFP ports:

1. By executing the command on the command line. The changes made in this way will be canceled after restarting the device.
 - Enter the command for gbe0 (port A):

```
root@smart-sfp:~# ethtool -s gbe0 autoneg on
```

- Enter the command for gbe1 (port B):

```
root@smart-sfp:~# ethtool -s gbe1 autoneg on
```

2. Changing the configuration file. Changes in the configuration file are permanent and are not canceled after the device is rebooted.
 - Mount the file system for writing:

```
root@smart-sfp:~# mount -o remount,rw /
```

- Edit the /etc/network/interfaces.d/gbe file as follows:

```
root@smart-sfp:~# nano /etc/network/interfaces.d/gbe
#SFP Port
auto gbe0
iface gbe0 inet static
address 192.168.1.1
netmask 255.255.255.0
#Autonegotiation for gbe0
post-up ethtool -s gbe0 autoneg on

#Host port
auto gbe1
iface gbe1 inet static
address 192.168.2.1
netmask 255.255.255.0
#Autonegotiation for gbe1
post-up ethtool -s gbe1 autoneg on
```

- To apply the settings, reboot the Smart SFP using the “reboot” command or the command below to apply the settings without rebooting.

```
root@smart-sfp:~# ifdown gbe0 && ifup gbe0 && ifdown gbe1 && ifup gbe1
```

8.1.2. Changing the speed on the Smart SFP ports manually

1. Connect to the device via SSH using the IP address of the gbe0 interface (port A) or gbe1 interface (port B).
2. Log in as admin and then switch to the root account:

```
admin@smart-sfp:~$ su
```

There are two ways to change the speed manually on Smart SFP:

1. By executing the command on the command line. The changes made in this way will be canceled after restarting the device.

- For a Smart SFP with an optical interface enter the command:

For gbe0 (port A):

```
root@smart-sfp:~# ethtool -s gbe0 autoneg off speed 1000
```

For gbe1 (port B):

```
root@smart-sfp:~# ethtool -s gbe1 autoneg off speed 1000
```

- For a Smart SFP with an electrical interface enter the command:

For gbe0 (port A):

```
root@smart-sfp:~# ethtool -s gbe0 autoneg off speed <10/100/1000>
```

For gbe1 (port B):

```
root@smart-sfp:~# ethtool -s gbe1 autoneg off speed 1000
```

2. Changing the configuration file. Changes in the configuration file are permanent and are not canceled after the device is rebooted.

- Mount the file system for writing:

```
root@smart-sfp:~# mount -o remount,rw /
```

- Edit the /etc/network/interfaces.d/gbe file as follows:

For a Smart SFP with an optical interface:

```
root@smart-sfp:~# nano /etc/network/interfaces.d/gbe
#SFP Port
auto gbe0
iface gbe0 inet static
address 192.168.1.1
netmask 255.255.255.0
#for gbe0
post-up ethtool -s gbe0 autoneg off speed 1000

#Host port
auto gbe1
iface gbe1 inet static
address 192.168.2.1
netmask 255.255.255.0
#for gbe1
post-up ethtool -s gbe1 autoneg off speed 1000
```

For a Smart SFP with an electrical interface:

```
root@smart-sfp:~# nano /etc/network/interfaces.d/gbe
#SFP Port
auto gbe0
iface gbe0 inet static
address 192.168.1.1
netmask 255.255.255.0
#for gbe0
post-up ethtool -s gbe0 autoneg off speed <10/100/1000>

#Host port
auto gbe1
iface gbe1 inet static
address 192.168.2.1
netmask 255.255.255.0
#for gbe1
post-up ethtool -s gbe1 autoneg off speed 1000
```

- To apply the settings, reboot the Smart SFP using the “reboot” command or the command below to apply the settings without rebooting.

```
root@smart-sfp:~# ifdown gbe0 && ifup gbe0 && ifdown gbe1 && ifup gbe1
```

8.2. Changing the MAC address of port A and port B

1. Connect to the device via SSH using the IP address of the gbe0 interface (port A) or gbe1 interface (port B).
2. Log in as admin and then switch to the root account:

```
admin@smart-sfp:~$ su
```

There are two ways to change the MAC addresses on the Smart SFP ports:

1. By executing the command on the command line. The changes made in this way will be canceled after restarting the device.
 - For port A (gbe0) run the commands:

```
root@smart-sfp:~# ifconfig gbe0 down
root@smart-sfp:~# ifconfig gbe0 hw ether <MAC address>
root@smart-sfp:~# ifconfig gbe0 up
```

- For port B (gbe1) run the commands:

```
root@smart-sfp:~# ifconfig gbe1 down
root@smart-sfp:~# ifconfig gbe1 hw ether <MAC address>
root@smart-sfp:~# ifconfig gbe1 up
```

2. Changing the configuration file. Changes in the configuration file are permanent and are not canceled after the device is rebooted.
 - Mount the file system for writing:

```
root@smart-sfp:~# mount -o remount,rw /
```

- Edit the /etc/network/interfaces.d/gbe file as follows:


```
root@smart-sfp:~# nano /etc/network/interfaces.d/gbe
#SFP Port
auto gbe0
iface gbe0 inet static
address 192.168.1.1
netmask 255.255.255.0
#for gbe0
hwaddress ether <MAC address>

#Host port
auto gbe1
iface gbe1 inet static
address 192.168.2.1
netmask 255.255.255.0
#for gbe1
hwaddress ether <MAC address>
```

- To apply the settings, reboot the Smart SFP using the “reboot” command or the command below to apply the settings without rebooting.

```
root@smart-sfp:~# ifdown gbe0 && ifup gbe0 && ifdown gbe1 && ifup gbe1
```

9. Packet transmission via Smart SFP 1G fiber and copper

9.1. Block diagram

This diagram shows the processing steps for packets received on port A. The same steps will be performed for packets received on port B.

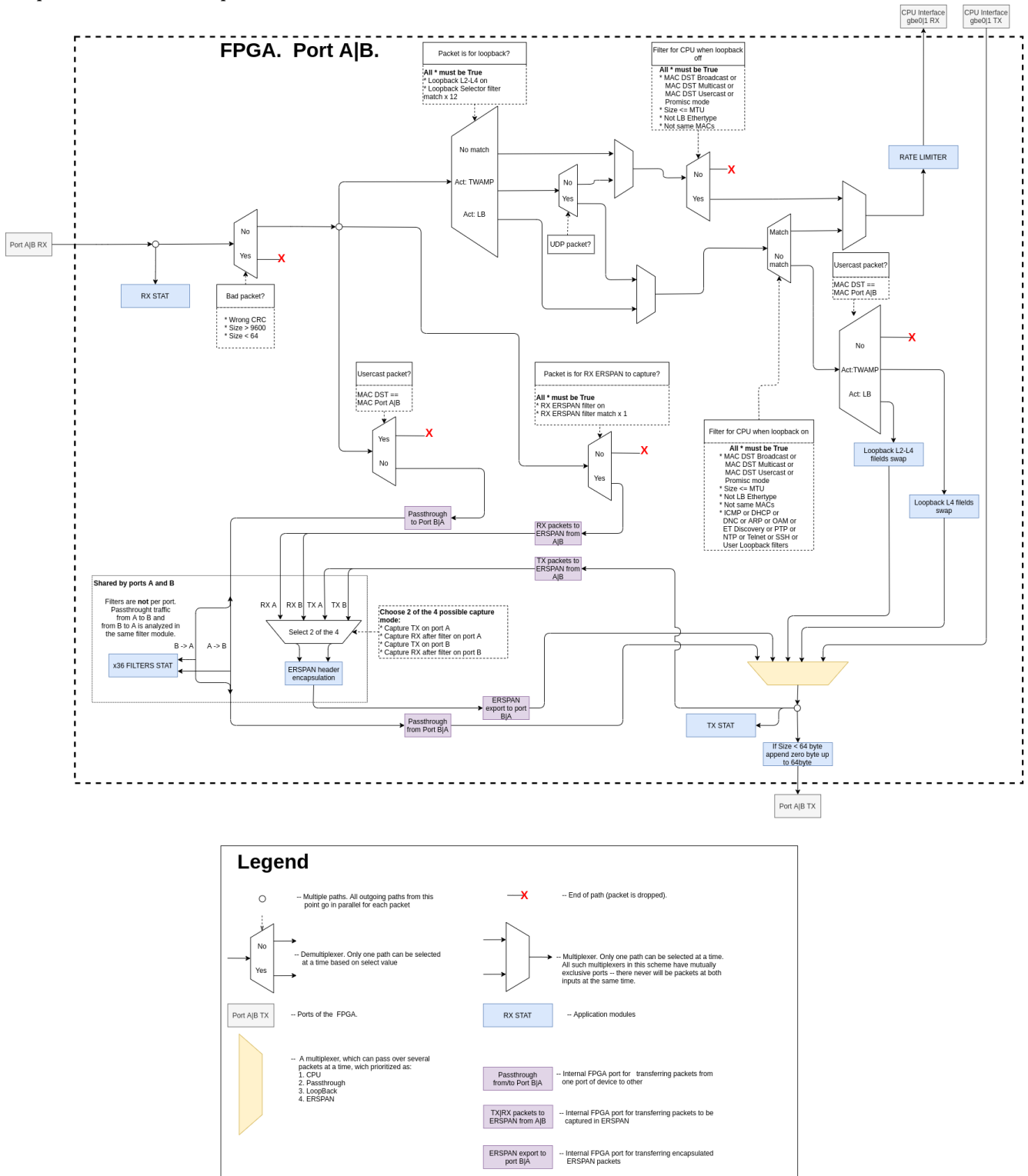


Figure 9.1. Block diagram

9.2. Traffic priorities inside Smart SFP 1G fiber and copper

There are several types of traffic – traffic from the Smart SFP CPU, transit traffic, and test traffic (loopback). Traffic priorities:

1. Transit traffic (high).
2. Loopback and TWAMP Light traffic.
3. Test traffic from the built-in generator, Y.1564.
4. Traffic from/to the CPU (for example, management traffic or application traffic running on Linux).
5. Mirrored packages (low).

9.3. Package processing steps

9.3.1. Passthrough

9.3.1.1. Which packets are sent from one port to another in transit mode

All packages except those that meet one of the following conditions:

- Frame size is less than 64 bytes
- Frame size is greater than 9600 bytes
- Frame contains bit errors (invalid FCS)
- Frame MAC DST is equal to the receiving port MAC address

9.3.1.2. Which packets reach the CPU

A packet reaches the CPU if MAC DST is equal to the port MAC address or frame contains Ethernet broadcast MAC DST or frame contains Ethernet multicast MAC DST or promiscuous mode is enabled in the CPU for the network interface, and all of the following conditions are true:

- Frame size is less than the MTU of port
- Frame size is equal to or greater than 64 bytes
- Frame size is equal to or less than 9600 bytes
- Frame does not contain bit errors (valid FCS)
- A packet does not belong to a special group of test packets (none of the above conditions are met):
 - MAC DST is equal to MAC SRC
 - Ethernet Configuration Testing Protocol packet (EtherType field value is 0x9000)

9.3.2. Loopback

9.3.2.1. Which packet fields are swapped before the packets are redirected backward

- Loopback layer 2:
MAC SRC ↔ MAC DST (for all packets)
- Loopback layer 3:
MAC SRC ↔ MAC DST (for all packets)

IPv4 SRC ↔ IPv4 DST (only for IPv4 packets)

- Loopback layer 4:

MAC SRC ↔ MAC DST (for all packets)

IPv4 SRC ↔ IPv4 DST (only for IPv4 packets)

UDP PORT SRC ↔ UDP PORT DST (only for UDP packets)

TCP PORT SRC ↔ TCP PORT DST (only for TCP packets)

9.3.2.2. Which packets reach the CPU

When the Loopback is turned on, packets that meet the following conditions are not being redirected backward and are sent to the CPU for further processing in case MAC DST is equal to the port MAC address, or frame contains Ethernet broadcast MAC DST or frame contains Ethernet multicast MAC DST, or promiscuous mode is enabled on the CPU for the network interface on which Loopback is turned on, and all of the following conditions are true:

- Frame size is less than the MTU of port on which Loopback is turned on
- The packet belongs to one of the protocols listed in table 9.1 or TCP/UDP DST port is equal to the specified CPU filter (see section 2.3.5 Filter)
- Frame size is equal to or greater than 64 bytes
- Frame size is equal to or less than 9600 bytes
- Frame does not contain bit errors (valid FCS)
- A packet does not belong to a special group of test packets (none of the above conditions are met):
 - MAC DST is equal to MAC SRC
 - Ethernet Configuration Testing Protocol packet (EtherType field value is 0x9000)

Table 9.1. Protocols whose messages are processed by the CPU

Protocol	The condition of sending to the CPU
ICMP	“Protocol” field value in IP-header is “1”
DHCP	UDP packet with DST port number 67 or 68
DNS	UDP or TCP packet with DST port number 53
ARP	“EtherType” field value is “0x0806”
OAM	MAC DST is equal to 01:80:c2:00:00:02
SSH	TCP packet with DST port number 22
PTP, layer 4	UDP packet with DST port number 319 or 320
NTP	UDP packet with DST port number 123
ET-discovery	UDP packet with DST port number 32792
REST API	TCP packet with DST port number 80
REST API with encryption	TCP packet with DST port number 443

9.3.2.3. Which packets are not being redirected backward

When the Loopback is turned on, packets that meet the following conditions are not being redirected backward:

- Frame size is less than 64 bytes
- Frame size is greater than 9600 bytes

- Frame contains bit errors (invalid FCS)
- Frame contains Ethernet multicast MAC DST
- MAC DST is not equal to the Loopback port MAC address
- MAC DST is equal to the MAC SRC
- Ethernet Configuration Testing Protocol frame (EtherType field value is 0x9000)
- MAC DST is equal to the port MAC address, or frame contains Ethernet broadcast MAC DST or promiscuous mode is enabled on the CPU for the network interface on which Loopback is turned on, and all of the following conditions are true:
 - Frame size is less than the MTU of the port on which Loopback is turned on
 - The packet belongs to one of the protocols listed in table 9.1 or TCP/UDP DST port is equal to the specified CPU filter (see section 2.3.5 Filter)

10. Configure device functions (CLI and API)

Available settings may vary depending on the Smart SFP modification and installed feature set (see chapter 2.3, 2.4, 2.5 and 2.6).

Data transfer configuration for Smart SFP 1G RS-232 is described in chapter 11.

10.1. Overview

10.1.1. CLI

Log in to the vendor-specific command-line interface with the “run-klish” command from the Linux shell.

```
admin@smart-sfp:~$ run-klish
smart-sfp(admin)#
!           Comments
configure   Enter configuration mode
exit        Exit from the CLI
show        Show information
system      Enter system command mode
up          Go one level up (^Z)
```

To switch to the common system control mode, use the “system” command.

```
smart-sfp(admin)# system
smart-sfp(admin)(system)#
!           Comments
exit        Exit from the CLI
passwd       Change password of selected user
reboot       Reboot the system
shutdown     Shutdown the system
su           Switch user
up           Go one level up (^Z)
```

To switch to the specific features configuration mode, use the “configure terminal” command.

```
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)#
```

To switch to the “wlt-rx” configuration mode for the selected profile, use the “wlt-rx config profile0/1” command.

```
smart-sfp(admin)(config)# wlt-rx config profile0
* Enter `show` to view the configuration
smart-sfp(admin)(config-wlt-rx [profile0])#
!           Comments
duration     Measure duration
exit         Exit from the CLI
flow         Flow
frame        Frame
header       WLT header
interval     Interval (seconds)
port         Choise port
rate         Measure rate
show         Show WLT information
up           Go one level up (^Z)
wait         Wait (seconds)
```

To switch to the “wlt-tx” configuration mode for the selected profile, use the “wlt-tx config profile0/1” command.

```
smart-sfp(admin)(config)# wlt-tx config profile0
* Enter `show` to view the configuration
smart-sfp(admin)(config-wlt-tx [profile0])#
!           Comments
duration    Measure duration
exit        Exit from the CLI
flow        Flow
frame       Frame
header      WLT header
interval    Interval
port        Choise port
rate        Measure rate
show        Show WLT information
up          Go one level up (^Z)
```

To switch to the loopback configuration mode for the selected profile, use the “loopback config profile0/1” command.

```
smart-sfp(admin)(config)# loopback config profile0
* Enter `show` to view the configuration
smart-sfp(admin)(config-loopback[profile0])#
!           Comments
duration    Loopback duration
exit        Exit from the CLI
learn       ARP notification settings
show        Show loopback information
topology    Loopback topology
type        Loopback type
up          Go one level up (^Z)
```

To switch to the Smart SFP optical transceiver configuration mode, use the “ssfp config” command.

```
smart-sfp(admin)(config)# ssfp config
* Enter `show` to view the configuration
smart-sfp(admin)(config-ssfp)#
!           Comments
exit        Exit from the CLI
rx_loss_mask RX loss signal management
show        Show SSFP information
tx_fault_mask TX fault signal management
up          Go one level up (^Z)
```

To switch to the Y.1564 configuration mode for the selected profile, use the “y1564 config profile0/1” command.

```
smart-sfp(root)(config)# y1564 config profile0
* Enter `show` to view the configuration
smart-sfp(root)(config-y1564[profile0])#
!           Comments
cfg         Configuration tests settings
count       Service count
exit        Exit from the CLI
learn       ARP notification settings
perf        Performance test settings
service     Enter service configure mode
show        Show Y.1564 information
step        Step count
topology    Y.1564 topology
up          Go one level up (^Z)
```

To switch to the service configuration mode from the Y.1564 configuration mode, use the “service 0/N” command.

```
smart-sfp(root)(config-y1564[profile0])# service 0
* Enter `show` to view the configuration
smart-sfp(root)(config-y1564[service0])#
!      Comments
exit   Exit from the CLI
frame  Y.1564 frame
header Frame header
rate   Service bandwidth
sac    Loss rate in percents
show   Show service settings
up     Go one level up (^Z)
```

To switch to the filters configuration mode for the selected profile, use the “filter config profile0/1” command.

```
smart-sfp(root)(config)# filter config profile0
* Enter `show` to view the configuration
smart-sfp(root)(config-filter[profile0])#
!      Comments
cpu    CPU filter
exit   Exit from the CLI
sel    Selector filter
show   Show filter information
topology Filter topology
up     Go one level up (^Z)
```

To switch to the filter statistics configuration mode for the selected profile, use the “fstatistic config profile0/1” command.

```
smart-sfp(root)(config)# fstatistic config profile0
* Enter `show` to view the configuration
smart-sfp(root)(config-fstatistic[profile0])#
!      Comments
exit   Exit from the CLI
show   Show filter information
up     Go one level up (^Z)
num    Choise a filter's number
```

To switch to the timesync function configuration mode for the selected profile, use the “timesync config profile0/1” command.

```
smart-sfp(root)(config)# tyesync config profile0
* Enter `show` to view the configuration
smart-sfp(root)(config-fstatistic[profile0])#
!      Comments
duration Set synchronization's type
exit     Exit from the CLI
mode     Set synchronization's mode
ntp      NTP mode
ptp      PTP mode
show     Show time synchronization information
topology Synchronization topology
type     Set synchronization's type
up       Go one level up (^Z)
```

To switch to the loaned ip configuration mode for the selected profile, use the “loaned-ip config profile0/1” command.


```

smart-sfp(admin)(config)# loaned-ip config profile0
* Enter `show` to view the configuration
smart-sfp(admin)(config-loaned-ip[profile0])#
!           Comments
exit        Exit from the CLI
ip          Loaned IP
range       Port range limit
show        Show loaned IP information
topology    Loaned IP topology
up          Go one level up (^Z)

```

To switch to the traffic capturing and mirroring configuration mode for the selected profile, use the “erspan config profile0/1” command.

```

smart-sfp(root)(config)# erspan config profile0
* Enter `show` to view the configuration
smart-sfp(root)(config-erspan[profile0])#
!           Comments
duration    ERSPAN duration
erspan      ERSPAN rule
exit        Exit from the CLI
filter      Filter settings
show        Show ERSPAN information
topology    ERSPAN topology
up          Go one level up (^Z)

```

To switch to the configuration mode for the service of managing additional network stack settings for the selected port, use the “network config porta/portb” command.

```

smart-sfp(admin)(config)# network config porta
* Enter `apply` to apply it
smart-sfp(admin)(config-network [porta])#
!           Comments
cpu-multicast Set CPU multicast filter
exit         Exit from the CLI
show         Show current configuration
up           Go one level up (^Z)

```

To switch to the timestamp insertion configuration mode for the selected profile, use the “tsins config profile0/1” command.

```

smart-sfp(root)(config)# tsins config profile0

Enter `show` to view the configuration
smart-sfp(root)(config-tsins[profile0])#
!           Comments
duration    Duration
exit        Exit from the CLI
level       Template level
pattern     User pattern
show        Show tsins information
status      Rule status
tspoint     Timestamp point
up          Go one level up (^Z)

```

10.1.2. Profiles

Two independent profiles are available in the device - “profile0” and “profile1”. The profile is a set of Smart SFP feature settings. Profiles are saved on the device, allowing you not to re-enter the settings and carry out testing with preset parameters for two different configurations. Profiles are automatically saved when the module is rebooted using the “reboot” command.

Note. When testing, you can use both profiles at the same time. For example, turn on loopback for each profile.

10.1.3. REST API

10.1.3.1. JSON REST API interaction

You may read about JSON RPC here: <https://www.jsonrpc.org/>. JSON REST API is available using HTTP (TCP-port 80) and HTTPS (TCP-port 443) protocols over both interfaces (A or B). Example: <http://192.168.2.1:80/api> or <https://192.168.2.1:443/api>.

10.1.3.2. Create work session with rest api

Request example

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": ["00000000000000000000000000000000", "session", "login", {
    "username": "admin",
    "password": "admin",
    "timeout": 300
  }]
}
```

Note. Username and password match system accounts in Linux.

Response example

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [0, {
    "ubus_rpc_session": "9013d223fd551350757ed8797259d14a",
    "timeout": 300,
    "expires": 300,
    "acls": {
      "access-group": {
        "root": ["read", "write"],
        "unauthenticated": ["read"],
        "user": ["read", "write"]
      },
      "toolbox": {
        "*": ["read"]
      },
      "ubus": {
        "*": ["*"],
        "session": ["access", "login"]
      },
      "uci": {
        "*": ["read", "write"]
      }
    },
    "data": {
      "username": "root"
    }
  }]
}
```

Note. Ubus_rpc_session value is needed for further work.

10.1.3.3. Executing a directly useful request to a service

Request format

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [ "<received ubus_rpc_session>", "<service>", "<command>", {
    <command parameters>
  } ]
}
```

Note:

- service – the name of the service implementing specified functionality (for example, loopback, statistics, etc.).
- command – a direct command for the service (for example: start, stop, show, etc.). Commands are dependent on service implementation and required functionality.
- command parameters – parameters in JSON format passed to the command.

Response format

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [0, {<response>}]
}
```

Note. Response – the response from the specified service on the execution of the specified command. The response is in JSON format.

A response may contain a “retcode” value:

“retcode” - return codes:

- 0 – Operation completed successfully
- 150 – General error unspecified
- 151 – General hardware error
- 152 – Incorrect contents of param
- 153 – Not supported feature
- 154 – Syntax error -> can't parse JSON for ex.
- 155 – Wrong type of known item/node
- 156 – Mandatory item/node absent
- 157 – Requested resource (interface for ex) busy by another process (test)
- 158 – No link on one of the requested interfaces (may be cable not connected)
- 159 – Timeout occurred during waiting for the response (typically it's a network timeout but can also be timeout of local hardware)
- 160 – The specified object (interface/profile/..) not found

10.2. Commands description

10.2.1. Loopback commands

10.2.1.1. In configuration mode

loopback clear

Command to clear accumulated loopback statistics for the selected profile.

Syntax

```
loopback clear {profile0 | profile1}
```

Parameters

- profile0 – clear statistics for the loopback profile0
- profile1 – clear statistics for the loopback profile1

Example

```
smart-sfp(root)(config)# loopback clear profile0  
ok
```

loopback config

Command to enter loopback configuration mode for the selected profile.

Syntax

```
loopback config {profile0 | profile1}
```

Parameters

- profile0 – enter loopback configuration mode for the profile0
- profile1 – enter loopback configuration mode for the profile1

Example

```
smart-sfp(admin)(config)# loopback config profile0  
smart-sfp(admin)(config-loopback[profile0])#
```

loopback start

Command to start loopback for the selected profile.

Syntax

```
loopback start {profile0 | profile1}
```

Parameters

- profile0 – start loopback for the profile0
- profile1 – start loopback for the profile1

Default

Loopback off

Note

If the filter is enabled (see “filter start” command), for the loopback to work, you must enable at least one selector filter for the route=loopback (see “sel” command). For the loopback to work for all traffic, all values in the settings of this filter rule number 0 need to be zero.

If filter is disabled (see “filter stop” command), the selector filter settings do not matter, and the loopback will work for all traffic, as described in 9.3.2.

```
smart-sfp(root)(config-filter[profile0])# sel num 0 route loopback
ok
smart-sfp(root)(config-filter[profile0])# show sel setting
Selector filters:
      Number      IP src      IP dst      Port src      Port dst      VLAN id      Route
      0            0.0.0.0      0.0.0.0      0              0              0            loopback
      1            0.0.0.0      0.0.0.0      0              0              0            none
      2            0.0.0.0      0.0.0.0      0              0              0            none
      3            0.0.0.0      0.0.0.0      0              0              0            none
      4            0.0.0.0      0.0.0.0      0              0              0            none
      5            0.0.0.0      0.0.0.0      0              0              0            none
      6            0.0.0.0      0.0.0.0      0              0              0            none
      7            0.0.0.0      0.0.0.0      0              0              0            none
      8            0.0.0.0      0.0.0.0      0              0              0            none
      9            0.0.0.0      0.0.0.0      0              0              0            none
      10           0.0.0.0      0.0.0.0      0              0              0            none
      11           0.0.0.0      0.0.0.0      0              0              0            none
```

Example

```
smart-sfp(admin)(config)# loopback start profile0
ok
smart-sfp(root)(config)# show activity
Port a:
      loopback
```

loopback stop

Command to stop loopback for the selected profile.

Syntax

```
loopback stop {profile0 | profile1}
```

Parameters

- profile0 – stop loopback for the profile0
- profile1 – stop loopback for the profile1

Default

Loopback off

Example

```
smart-sfp(root)(config)# loopback stop profile1
ok
smart-sfp(root)(config)# show activity
No activity on ports
```

show loopback results

Command to display loopback results.

Syntax

```
show loopback results {profile0 | profile1}
```

Parameters

- profile0 – loopback results for the profile0
- profile1 – loopback results for the profile1

Example

```
smart-sfp(admin)# show loopback results profile0
Status:                false
Message:               ok
Test duration:
  Start time:          --:--:--:-----
  Stop time:           --:--:--:-----
  Elapsed time:        00:00:00:000000
                        Rx                Tx
porta:
  Frames               0                0
  Bytes                0                0
  Rate(kbps)           0                0
portb:
  Frames               0                0
  Bytes                0                0
  Rate(kbps)           0                0
Meas duration:
  Start time:          --:--:--:-----
  Stop time:           --:--:--:-----
  Elapsed time:        00:00:00:000000
```

show loopback setting

Command to display loopback settings.

Syntax

```
show loopback setting {profile0 | profile1}
```

Parameters

- profile0 – loopback settings for the profile0
- profile1 – loopback settings for the profile1

Example

```
smart-sfp(admin)# show loopback setting profile0
Status:                false
Type:                  layer4
Filters:               false
Duration type:         useconds
Duration:              endless
Learn frame:
  Time (ms):           0
```

show loopback topology

Command to display loopback topology.

Syntax

```
show loopback topology {profile0 | profile1}
```

Parameters

- profile0 – loopback topology for the profile0
- profile1 – loopback topology for the profile1

Example

```
smart-sfp(admin)# show loopback topology profile0
Interface: 0
      Status: true
      Port:   porta
      Mode:   local
      Host:
Interface: 1
      Status: false
      Port:   portb
      Mode:   local
      Host:
```

10.2.1.2. In profile configuration mode

duration

Command to set the duration of the Loopback function operating.

Syntax

```
duration {endless | time <value>}
```

Parameters

- endless – the “Loopback” function will work until the command “loopback stop profile 0/1” is entered
- time <value> – user-specified value in the format hh:mm:ss:uuuuuu, where hh – hours, mm – minutes, ss – seconds, uuuuuu – microseconds

Valid values: 00:00:00:000000-99:59:59:999999. “00:00:00:000000” means that the “Loopback” function will work until the “loopback stop profile 0/1” command is entered.

Default

endless

Example

```
smart-sfp(admin)(config-loopback[profile0])# duration time 01:30:00:000000
ok
```

learn

Command to set learning frames transmission interval. This frame has the same source and destination MAC address. It informs the network devices about Smart SFP in the network.

Syntax

```
learn {time <value>}
```

Parameters

- time <value> – learning frames transmission interval (ms). Valid values: 0–65535.
0 ms – learning frames transmission (TX) is off

Default

Learning frames transmission (TX) is off (0 ms).

Example

```
smart-sfp(admin)(config-loopback[profile0])# learn time 1000
ok
```

show

Command to display information about loopback settings and topology.

Syntax

show {setting | topology}

Parameters

- setting – show loopback settings: type (loopback layer), duration type, duration of the function operating, learning frames sending interval
- topology – show current interface topology settings: status (true – logical interface is used, false – logical interface not used), port – physical port corresponding to the logical interface, mode – logical interface, host – not supported yet

Example

```
smart-sfp(admin)(config-loopback[profile0])# show setting
Type:          layer4
Duration type:  useconds
Duration:       endless
Learn frame:
Time (ms):      0

smart-sfp(admin)(config-loopback[profile0])# show topology
Interface:  0
            Status:  true
            Port:    porta
            Mode:     local
            Host:
Interface:  1
            Status:  false
            Port:    portb
            Mode:     local
            Host:
```

topology

Command to bind physical port to the logical interface.

Syntax

topology {iface <0 | 1> port <a | b> status <on | off>}

Parameters

- iface <0 | 1> – interface number
- port <a | b> – physical port corresponding to the selected logical interface
- status <on | off> – enable or disable an interface for use in the profile 0/1

Default

iface 0 – port A, status – on.

iface 1 – port B, status – off.

Example

```
smart-sfp(root)(config-loopback[profile0])# topology iface 0 port a status on
ok
```


type

Command to set loopback layer.

Syntax

type {layer <2 | 3 | 4>}

Parameters

- layer <2 | 3 | 4> – loopback layer (see 9.3.2)

Default

Layer 4

Example

```
smart-sfp(admin)(config-loopback[profile0])# type layer2
ok
```

10.2.2. Loopback API

10.2.2.1. Loopback settings change

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "0711798292de52e0fc7c07139c163086",
    "loopback",
    "setprm",
    {
      "ids": {
        "profile": "profile0"
      },
      "parameters": {
        "status": false,
        "trial": {
          "ifaces": {
            "0": {
              "name": "porta",
              "mode": "local",
              "host": "",
              "disabled": false
            },
            "1": {
              "name": "portb",
              "mode": "local",
              "host": "",
              "disabled": true
            }
          }
        },
        "wait_time_ms": 1000,
        "learn_time_ms": 1000
      }
    },
    "loopback": {
      "duration_us": "0",
      "type": "layer4",
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "loopback",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.2.2. Loopback settings view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "579ba6e5001a2c30dfedc4df87c6f522",
    "loopback",
    "getprm",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "loopback",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "object": "loopback",
          "method": "setprm",
          "ids": {
            "profile": "profile0"
          },
          "parameters": {
            "status": false,
            "trial": {
              "ifaces": {
                "0": {
                  "name": "porta",
                  "mode": "local",
                  "host": "",
                  "disabled": false
                },
                "1": {
                  "name": "portb",

```

```
    "mode": "local",  
    "host": "",  
    "disabled": true  
  },  
  "wait_time_ms": 1000,  
  "learn_time_ms": 1000  
},  
"loopback": {  
  "duration_us": "0",  
  "type": "layer4",  
}  
}  
]  
}
```

10.2.2.3. Loopback start

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "4f3f1b77d38cb303024001bd5bb5ab90", "loopback", "start", {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "loopback",
      "method": "start",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.2.4. Loopback stop

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "0cfebb5ee35494b45b858c23df7c4b5e",
    "loopback", "stop",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

```

    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "loopback",
      "method": "stop",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}

```

10.2.3. Statistics commands in configuration mode

pstatistics clear

Command to clear accumulated statistics.

Syntax

```
pstatistics clear [port <a | b>]
```

Parameters

- no arguments – clear all statistics
- port <a | b> – clear statistics for the port A/B

Example

```

smart-sfp(admin)(config)# pstatistics clear
ok

```

show pstatistics

The command to display the statistics.

Syntax

```
show pstatistics [port <a | b>]
```

Parameters

- no arguments – show statistics for the both ports
- port <a | b> – port for which statistics is displayed

Example

```

smart-sfp(admin)(config)# show pstatistics
                                Rx      Tx
Interface: porta
  Frames(num):      0      190
  Bytes(num):       0     13676
  Rate(L2 bps):     0      576
Interface: portb
  Frames(num):     372     163
  Bytes(num):     35768   23841
  Rate(L2 bps):    2480    1648

```

10.2.4. Statistics API

10.2.4.1. Clear statistics

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "73df5a27f81b59d86ae3d7b3bd5deb02",
    "pstatistics",
    "clear",
    {
      "ids": {
        "interface": "porta"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "pstatistics",
      "method": "clear",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.4.2. Forced synchronization

Executed before requesting statistics data to get the actual values.

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "d3e69587530923ebf2e5335b985293fc",
    "pstatistics",
    "sync",
    {
      "ids": {
        "interface": "porta"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
```

```

        "id": 1,
        "result": [
            0,
            {
                "object": "pstatistics",
                "method": "getsts",
                "retcode": 0,
                "retmsg": "OK",
            }
        ]
    }

```

10.2.4.3. Statistics request

Before requesting statistics, a forced synchronization should be performed. Without forced synchronization, values are updated every 275 seconds.

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "d3e69587530923ebf2e5335b985293fc",
        "pstatistics",
        "getsts",
        {
            "ids": {
                "interface": "porta"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "pstatistics",
            "method": "getsts",
            "retcode": 0,
            "retmsg": "OK",
            "answer": [
                {
                    "ids": {
                        "interface": "porta"
                    },
                    "statuses": {
                        "rx_frames": "213199245",
                        "rx_bytes": "13644751680",
                        "rx_rate_l2_bps": "0",
                        "tx_frames": "199362884",
                        "tx_bytes": "12759170324",
                        "tx_rate_l2_bps": "0",
                    }
                }
            ]
        }
    ]
}

```

10.2.5. Commands in the Smart SFP optical transceiver configuration mode

show ssfp results

The command to display Smart SFP signal states.

Syntax

```
show ssfp results
```

Parameters

no arguments

Example

```
smart-sfp(root)(config)# show ssfp results
      Status:      false
      Message:      ok
      Rate select:  false
      TX detect:    true
      TX disable:   false
      TX fault:     false
      RX loss:      true
```

- Rate select - requested receiver bandwidth. False - lowest possible bandwidth. True - maximum possible throughput. The value is ignored. Smart SFP always runs at 1 Gbps.
- TX detect – true: data transmitting detected, false: no data transmitted
- TX disable – true: transmitter off, false: transmitter on
- TX fault – true: transmitter error detected, false: transmitter works without errors
- RX loss – true: optical power is less than the worst sensitivity of the receiver, false: valid incoming optical signal

show ssfp setting

Command to display Smart SFP optical transceiver settings.

Syntax

```
show ssfp setting
```

Parameters

no arguments

Example

```
smart-sfp(admin)(config)# show ssfp setting
      Status:      true
      TX fault mask: false
      RX loss mask: true
```

10.2.6. Smart SFP optical transceiver API

10.2.6.1. Smart SFP optical transceiver settings view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
```

```

        "params": [
            "01ab888cd33959eddd2bfac1de652b82",
            "device.optics",
            "getsts",
            {}
        ]
    }

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "device.optics",
            "method": "getsts",
            "retcode": 0,
            "retmsg": "OK",
            "answer": [
                {
                    "statuses": {
                        "status": true,
                        "retcode": 0,
                        "optics": {
                            "tx_flt": false,
                            "rx_los": true,
                            "tx_dis": false,
                            "tx_sd": true,
                            "rate_sel": false
                        }
                    }
                }
            ]
        }
    ]
}

```

10.2.7. Y.1564 commands

10.2.7.1. In configuration mode

y1564 config

Command to enter the Y.1564 function configuration mode for the selected profile.

Syntax

```
y1564 config {profile0 | profile1}
```

Parameters

- profile0 – enter the configuration mode for the profile0
- profile1 – enter the configuration mode for the profile1

Example

```

smart-sfp(admin)(config)# y1564 config profile0
smart-sfp(admin)(config-y1564[profile0])#

```

y1564 start

Command to start Y.1564 for the selected profile.

Syntax

```
y1546 start {profile0 | profile1}
```

Parameters

- profile0 – start Y.1564 for the profile0
- profile1 – start Y.1564 for the profile1

Default

Y.1564 off

Example

```
smart-sfp(root)(config)# y1564 start profile0
ok
```

y1564 stop

Command to stop Y.1564 for the selected profile.

Syntax

```
y1546 stop {profile0 | profile1}
```

Parameters

- profile0 – stop Y.1564 for the profile0
- profile1 – stop Y.1564 for the profile1

Default

Y.1564 off

Example

```
smart-sfp(root)(config)# y1564 stop profile0
ok
```

show y1564 results

Command to display Y.1564 results.

Syntax

```
show y1564 results {profile0 | profile1}
```

Parameters

- profile0 – Y.1564 results for the profile0
- profile1 – Y.1564 results for the profile1

Example

```
smart-sfp(root)(config)# show y1564 results profile0
Status:          false
Message:         ok
Start time:      --:--:--:-----
Stop time:       --:--:--:-----
Elapsed time:    00:00:00:000000
Test            cir
Part            0
```

show y1564 setting

Command to display the settings of the Y.1564 function.

Syntax

```
show y1564 setting [service <value>] {profile0 | profile1}
```

Parameters

- service <value> – display settings for the specified service number. Valid values: 1-4.
- profile0 – settings for the profile0
- profile1 – settings for the profile1

Example

```
smart-sfp(admin)(config)# show y1564 setting profile0
  Status:                false
  Services count:        1
  Step count:            1
  Configuration tests:
    Duration type:       useconds
    Duration:            00:01:00:000000
    CIR test:            true
    EIR test:            true
    Traf. policing:      true
  Performance test:
    Duration type:       useconds
    Duration:            00:01:00:000000
    Test PERF:          true
  Learn frame:
    Time (ms):           1000
smart-sfp(admin)(config)# show y1564 setting service 0 profile0
  Frame size:            64
  Frame header:
    Src IP:              192.168.1.1
    Src MAC:              00:21:CE:28:00:2D
    Src port:             60000
    Dst IP:               192.168.2.2
    Dst MAC:              00:21:CE:28:00:2E
    Dst port:             50000
    QOS type:             diffserv
                        dscp:                0
                        tos:                 0
                        precedence:          0
  SAC:
    FTD(ms):              1.00000
    FDV(ms):              1.00000
    Mfactor(mbps):        0.10000
    FLR(%):               0.10000
    OOP(%):               0.00000
  Bandwidth:
    CIR rate(L2 mbps):    10.00000
    EIR rate(L2 mbps):    1.00000
    TP rate(L2 mbps):     11.00000
```

show y1564 topology

Command to display the topology of the Y.1564 function.

Syntax

```
show y1564 topology {profile0 | profile1}
```

Parameters

- profile0 – topology for the profile0
- profile1 – topology for the profile1

Example

```
smart-sfp(admin)(config)# show y1564 topology profile0
Interface: rx
Status: true
Port: porta
Mode: local
Host:
Interface: tx
Status: true
Port: porta
Mode: local
Host:
```

10.2.7.2. In profile configuration mode

cfg

Command to change configuration tests settings.

Syntax

```
cfg {duration <bytes | packets | useconds> <value> | < cir | eir | tp > < on | off >}
```

Parameters

- duration bytes <value> – configuration test duration in bytes
- duration packets <value> – configuration test duration in packets
- duration useconds <value> – user-specified value in the format hh:mm:ss:uuuuuu, where hh – hours, mm – minutes, ss – seconds, uuuuuu – microseconds
- < cir | eir | tp > – test type: CIR, EIR, Traffic Policing
- < on | off > – enable or disable test

Default

Duration type: useconds

Duration: 00:01:00:000000

CIR test: true

EIR test: true

Traf. policing: true

Example

```
smart-sfp(root)(config-y1564[profile0])# learn time 0
ok
```

count

Command to set service count.

Syntax

```
count <value>
```

Parameters

- <value> – service count (1–4)

Default

1

Example

```
smart-sfp(root)(config-y1564[profile0])# count 2
ok
```

learn

Command to set learning frames transmission interval.

Syntax

```
learn {time <value>}
```

Parameters

- time <value> – learning frames transmission interval (ms). 0 ms – transmission (TX) of learning frames is disabled. Valid values: 0-65535.

Default

Transmission (TX) of learning frames is disabled (0 ms).

Example

```
smart-sfp(admin)(config-loopback[profile0])# learn time 1000
ok
```

perf

Command to change performance test settings.

Syntax

```
perf {duration <bytes | packets | useconds> <value> | < on | off >}
```

Parameters

- duration bytes <value> – performance test duration in bytes
- duration packets <value> – performance test duration in packets
- duration useconds <value> – user-specified value in the format hh:mm:ss:uuuuuu, where hh – hours, mm – minutes, ss – seconds, uuuuuu – microseconds
- <on | off> – enable or disable test

Default

Duration type: useconds

Duration: 00:01:00:000000

Test PERF: true

Example

```
smart-sfp(root)(config-y1564[profile0])# perf on
ok
smart-sfp(root)(config-y1564[profile0])# perf off
ok
smart-sfp(root)(config-y1564[profile0])# perf duration useconds 00:10:00:000000
```

```
ok
smart-sfp(root)(config-y1564[profile0])# perf duration packets 555555
ok
smart-sfp(root)(config-y1564[profile0])# perf duration bytes 666666
ok
```

service

Command to enter service configuration mode.

Syntax

```
service {<value>}
```

Parameters

- <value> – service number. Valid values: 0-3.

Example

```
smart-sfp(admin)(config-y1564[profile0])# service 0
smart-sfp(admin)(config-y1564[service0])#
```

step

Command to set step count for CIR test.

Syntax

```
step <value>
```

Parameters

- <value> – step count for CIR test (1–10).

Default

1

Example

```
smart-sfp(root)(config-y1564[profile0])# step 3
ok
```

show setting

Command to display Y.1564 settings.

Syntax

```
show setting
```

Parameters

no arguments

Example

```
smart-sfp(root)(config-y1564[profile0])# show setting
Status:                false
Services count:         2
Step count:             3
Configuration tests:
  Duration type:        useconds
  Duration:             00:01:00:000000
  CIR test:             true
  EIR test:             true
```

```

    Traf. policing:  true
Performance test:
    Duration type:   useconds
    Duration:        00:01:00:000000
    Test PERF:       true
Learn frame:
                                Time (ms):      0

```

topology

Command to bind physical port to the logical interface.

Syntax

```
topology {iface <rx | tx> port a}
```

Parameters

- iface <rx | tx> – rx: analyzer logical interface, tx: generator logical interface
- port a – physical port corresponding to the selected logical interface; allows the interface to be used in profile 0/1

Default

iface rx – port A, status – true.

iface tx – port A, status – true.

Example

```

smart-sfp(admin)(config-y1564[profile0])# topology iface rx port a
ok
smart-sfp(admin)(config-y1564[profile0])# topology iface tx port a
ok
smart-sfp(admin)(config-y1564[profile0])# show topology
    Interface: rx
                        Status:  true
                        Port:    porta
                        Mode:    local
                        Host:
    Interface: tx
                        Status:  true
                        Port:    porta
                        Mode:    local
                        Host:

```

10.2.7.3. In service configuration mode

frame

Command to set frame size.

Syntax

```
frame {size <value>}
```

Parameters

- size <value> – frame size in bytes. Valid values: 64-9600.

Default

64

Example

```
smart-sfp(root)(config-y1564[service0])# frame size 128
ok
```

header

Command to set header parameters.

Syntax

```
header {<src | dst> ip <value>} | {<src | dst> mac <value>} | {<src | dst> udp
<value>} | {vlan count <off | 1 | 2 | 3>} | {vlan number <value> id <value> pri
<value>} | {mpls count <off | 1 | 2 | 3>} | {mpls number <value> value <value> tc
<value> ttl <value>} | {qos <precedence | dscp>} | {tos <value>} | {precedence
<value>} | {dscp <value>}
```

Parameters

- <src | dst> ip <value> – source or destination IP address
- <src | dst> mac <value> – source or destination MAC address
- <src | dst> udp <value> – source or destination UDP port number
- vlan count <off | 1 | 2 | 3> – VLAN label count: 1, 2, or 3. “off” - VLAN label insertion is disabled.
- vlan number <value> id <value> pri <value> – VLAN identifier and VLAN priority for the selected VLAN number. Valid values: vlan number: 0-2, vlan id: 0-4095, vlan priority: 0-7.
- mpls count <off | 1 | 2 | 3> – MPLS label count: 1, 2, or 3. “off” - MPLS label insertion is disabled.
- mpls number <value> value <value> tc <value> ttl <value> – MPLS value, traffic class and time to live (ttl) value for the selected MPLS number. Valid values: mpls number: 0-2, value: 0-65535, tc: 0-7, ttl: 0-255.
- qos <precedence | dscp> – Quality of Service type
- tos <value> – Type of Service value. Valid values: 0-15.
- precedence <value> – precedence value. Valid values: 0-7.
- dscp <value> – DSCP value. Valid values: 0-63.

Default

Frame size: 64

Frame header:

Src IP: 192.168.1.1

Src MAC: 00:21:CE:28:00:2D

Src port: 60000

Dst IP: 192.168.2.2

Dst MAC: 00:21:CE:28:00:2E

Dst port: 50000

QOS type: diffserv

dscp: 0

tos: 0

precedence: 0

Example

```
smart-sfp(root)(config-y1564[service0])# header dst ip 192.168.2.2
ok
smart-sfp(root)(config-y1564[service0])# header dst mac 00:21:CE:28:00:2E
ok
smart-sfp(root)(config-y1564[service0])# header src ip 192.168.1.1
ok
smart-sfp(root)(config-y1564[service0])# header src mac 00:21:CE:28:00:2D
ok
smart-sfp(root)(config-y1564[service0])# header vlan count 1
ok
smart-sfp(root)(config-y1564[service0])# header vlan number 1 id 7 pri 4
ok
smart-sfp(root)(config-y1564[service0])# header qos precedence
ok
smart-sfp(root)(config-y1564[service0])# header precedence 4
ok
```

rate

Command to set service bandwidth.

Syntax

```
rate {<cir | eir | tp> <value> <percents | kbps | mbps> <layer1 | layer2 | layer3 | layer4>}
```

Parameters

- <cir | eir | tp> – test type: CIR, EIR or Traffic Policing.
- <value> <percents | kbps | mbps> – rate value in percents, kbps or mbps. Valid values: 0–100%; for kbps and mbps, valid values will depend on the selected layer and frame size for the service (see section 16.12).
- <layer1 | layer2 | layer3 | layer4> – rate layer

Default

Bandwidth:

CIR rate(L2 mbps): 10.00000

EIR rate(L2 mbps): 1.00000

TP rate(L2 mbps): 11.00000

Example

```
smart-sfp(admin)(config-y1564[service0])# rate cir 90 mbps layer2
ok
```

sac

Command to set value of service acceptance criteria.

Syntax

```
sac {<fdv | flr | ftd | mfactor | oop> <value>}
```

Parameters

- <fdv | flr | ftd | mfactor | oop> – FDV (Frame Delay Variation), FLR (Frame Loss Ratio), FTD (Frame Transfer Delay), M factor (is added to allow for the effect of the traffic policer's CBS and EBS settings, and test time)

- <value> – FTD value in ms, FDV value in ms, Mfactor value in mbps, FLR value in %, OOP value in %

Default

FTD(ms): 1.00000

FDV(ms): 1.00000

Mfactor(mbps): 0.10000

FLR(%): 0.10000

OOP(%): 0.00000

Example

```
smart-sfp(admin)(config-y1564[service0])# sac mfactor 0.1
ok
smart-sfp(admin)(config-y1564[service0])# sac oop 99
ok
```

show service

Command to display service settings.

Syntax

show service

Parameters

no arguments

Example

```
smart-sfp(admin)(config-y1564[service0])# show service
Frame size:                64
Frame header:
  Src IP:                   192.168.1.1
  Src MAC:                   00:21:CE:44:00:6A
  Src port:                  60000
  Dst IP:                    192.168.1.2
  Dst MAC:                   00:21:CE:28:3E:9E
  Dst port:                  50000
  QOS type:                  diffserv
                             dscp:                0
                             tos:                  0
                             precedence:            0
SAC:
  FTD(ms):                   1.00000
  FDV(ms):                   1.00000
  Mfactor(mbps):             0.10000
  FLR(%):                    0.10000
  OOP(%):                    99.00000
Bandwidth:
  CIR rate(L2 mbps):         90.00000
  EIR rate(L2 mbps):         1.00000
  TP rate(L2 mbps):          100.00000
```

10.2.8. Y.1564 API

10.2.8.1. View Y.1564 common settings

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "71ee8eda60763580355b6c742e6a588a",
    "y1564",
    "getprm",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "y1564",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "object": "y1564",
        "method": "setprm",
        "ids": {
          "profile": "profile0"
        }
      }],
      "parameters": {
        "status": false,
        "y1564": {
          "service_count": 2,
          "step_count": 3,
          "base_flow_id": 4660,
          "conf_duration": {
            "type": "useconds",
            "useconds": "60000000",
            "packets": "0",
            "bytes": "0",
            "pld_useconds": "60000000",
            "pld_bytes": "0"
          },
          "perf_duration": {
            "type": "useconds",
            "useconds": "60000000",
            "packets": "0",
            "bytes": "0",
            "pld_useconds": "60000000",
            "pld_bytes": "0"
          },
          "cir_enabled": true,
          "eir_enabled": true,
          "tp_enabled": true,
          "perf_enabled": true
        }
      }
    }
  ]
}
```

```

    },
    "trial": {
      "ifaces": {
        "rx": {
          "name": "porta",
          "mode": "local",
          "host": "",
          "disabled": false
        },
        "tx": {
          "name": "porta",
          "mode": "local",
          "host": "",
          "disabled": false
        }
      },
      "wait_time_ms": 1000,
      "learn_time_ms": 0
    }
  }
}
]
}

```

10.2.8.2. View Y.1564 service settings

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "71ee8eda60763580355b6c742e6a588a",
    "y1564",
    "getprm",
    {
      "ids": {
        "profile": "profile0",
        "service": 0
      }
    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "y1564",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "object": "y1564",
        "method": "setprm",
        "ids": {
          "profile": "profile0",
          "service": 0
        }
      }],
      "parameters": {

```

```

"frame_size": 64,
"header": {
  "src": {
    "mac": "00:21:CE:28:00:2D",
    "ip": "192.168.1.1",
    "udp_port": 60000
  },
  "dst": {
    "mac": "00:21:CE:28:00:2E",
    "ip": "192.168.2.2",
    "udp_port": 50000
  },
  "vlan": {
    "count": 0,
    "tags": [{
      "pri": 0,
      "id": 4095
    },
    {
      "pri": 0,
      "id": 4094
    },
    {
      "pri": 0,
      "id": 4093
    }
  ]
},
  "mpls": {
    "count": 0,
    "labels": [{
      "value": 128,
      "tc": 0,
      "ttl": 0
    },
    {
      "value": 128,
      "tc": 0,
      "ttl": 0
    },
    {
      "value": 128,
      "tc": 0,
      "ttl": 0
    }
  ]
},
  "qos_type": "diffserv",
  "dscp": 0,
  "ecn": 0,
  "tos": 0,
  "precedence": 0
},
"sac": {
  "loss_percents": "0.10000",
  "frame_delay_ms": "1.00000",
  "delay_variation_ms": "1.00000",
  "mfactor_mbps": "0.10000",
  "unordered_percents": "0.44000"
},
"bandwidth": {
  "cir_rate": {
    "value": "10.00000",
    "units": "mbps",
    "layer": 2
  },
  "eir_rate": {

```



```

        },
        "tx": {
            "name": "porta",
            "mode": "local",
            "host": "",
            "disabled": false
        }
    },
    "wait_time_ms": 1000,
    "learn_time_ms": 0
}
]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "y1564",
            "method": "setprm",
            "retcode": 0,
            "retmsg": "OK"
        }
    ]
}

```

10.2.8.4. Change Y.1564 service settings

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "cf2ed9b520ebdf600bbd051f304a38c0",
        "y1564",
        "setprm",
        {
            "ids": {
                "profile": "profile0",
                "service": 0
            },
            "parameters": {
                "frame_size": 64,
                "header": {
                    "src": {
                        "mac": "00:21:CE:28:00:2D",
                        "ip": "192.168.1.1",
                        "udp_port": 60000
                    },
                    "dst": {
                        "mac": "00:21:CE:28:00:2E",
                        "ip": "192.168.2.2",
                        "udp_port": 50000
                    }
                },
                "vlan": {
                    "count": 0,
                    "tags": [{

```


Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "y1564",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.8.5. Y.1564 start

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "1c4776dc3739a1c31e7283e380a361f0",
    "y1564",
    "start",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "y1564",
      "method": "start",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.8.6. Y.1564 stop

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "a8ecbd440bc6421186487b716cdeb994",
    "y1564",
    "stop",
    {
      "ids": {
```



```

        "profile": "profile0"
    }
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "y1564",
            "method": "stop",
            "retcode": 0,
            "retmsg": "OK"
        }
    ]
}

```

10.2.8.7. View Y.1564 common results

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "11a896b1dc6dbf2f8e6cb5c7740fc388",
        "y1564",
        "getsts",
        {
            "ids": {
                "profile": "profile0"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "y1564",
            "method": "getsts",
            "retcode": 0,
            "retmsg": "OK",
            "answer": [{
                "ids": {
                    "profile": "profile0"
                },
                "statuses": {
                    "status": false,
                    "retcode": 0,
                    "retmsg": "OK",
                    "sys_time": {
                        "start_us": "0",
                        "stop_us": "0",
                        "elapsed_us": "0"
                    }
                }
            }]
        }
    ]
}

```

```

    },
    "speed_mbps": 0,
    "cur_test": "cir",
    "cur_part": 0
  }
}
]
}

```

10.2.8.8. View Y.1564 test results

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "11a896b1dc6dbf2f8e6cb5c7740fc388",
    "y1564",
    "getsts",
    {
      "ids": {
        "profile": "profile0",
        "test": "cir",
      }
    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "y1564",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "ids": {
          "profile": "profile0",
          "test": "cir"
        },
        "statuses": {
          "status": false,
          "part_count": 6,
          "sys_time": {
            "start_us": "0",
            "stop_us": "0",
            "elapsed_us": "0"
          }
        }
      }]
    }
  ]
}

```

10.2.8.9. View Y.1564 measurement results

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "11a896b1dc6dbf2f8e6cb5c7740fc388",
    "y1564",
    "getsts",
    {
      "ids": {
        "profile": "profile0",
        "test": "cir",
        "part": 0
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "y1564",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "ids": {
          "profile": "profile0",
          "test": "cir",
          "part": 0
        },
        "statuses": {
          "status": false,
          "rate_mbps": "0.00000",
          "sys_time": {
            "start_us": "0",
            "stop_us": "0",
            "elapsed_us": "0"
          },
          "meas_time": {
            "start_us": "0",
            "stop_us": "0",
            "elapsed_us": "0"
          },
          "flows": [{
            "service": 0,
            "rate_mbps": "0.00000",
            "check_code": 1,
            "info_rate": {
              "min_mbps": "inf",
              "max_mbps": "0.00000",
              "average_mbps": "0.00000"
            },
            "frame_delay": {
              "min_ms": "inf",
              "max_ms": "0.00000",
              "average_ms": "0.00000"
            }
          }
        ]
      }
    ]
  ]
}
```

```

        "delay_variation": {
            "max_s": "0.00000",
            "average_s": "0.00000"
        },
        "stat": {
            "rx_pkts": "0",
            "rx_unordered_pkts": "0",
            "tx_pkts": "0"
        },
        "loss_percents": "0.00000",
        "unordered_percents": "0.00000"
    }
}]]]]

```

10.2.9. Filters commands

10.2.9.1. In configuration mode

filter config

Command to enter the Filter function configuration mode for the selected profile.

Syntax

```
filter config {profile0 | profile1}
```

Parameters

- profile0 – enter the configuration mode for the profile0
- profile1 – enter the configuration mode for the profile1

Example

```

smart-sfp(admin)(config)# filter config profile0
smart-sfp(admin)(config-filter[profile0])#

```

filter start

Command to enable traffic filtering for the selected profile.

Syntax

```
filter start {profile0 | profile1}
```

Parameters

- profile0 – enable traffic filtering for the profile0
- profile1 – enable traffic filtering for the profile1

Example

```

smart-sfp(root)(config)# filter start profile0
ok
smart-sfp(root)(config)# show activity
Port a:
    filters
Port b:
    filters

```

filter stop

Command to disable traffic filtering for the selected profile.

Syntax

```
filter stop {profile0 | profile1}
```

Parameters

- profile0 – disable traffic filtering for the profile0
- profile1 – disable traffic filtering for the profile1

Example

```
smart-sfp(root)(config)# filter stop profile1
ok
smart-sfp(root)(config)# show activity
No activity on ports
```

show filter

Command to display traffic filtering settings and results.

Syntax

```
show filter {cpu setting | results | sel setting | topology} {profile0 | profile1}
```

Parameters

- cpu setting – display CPU filter settings
- results – display traffic filtering results
- sel setting – display selector filter settings
- topology – display topology settings
- profile0 – traffic filtering settings/results for the profile0
- profile1 – traffic filtering settings/results for the profile1

Example

```
smart-sfp(admin)(config)# show filter cpu setting profile0
CPU filters:
  Number  Port  dst  Status
  0        0      False
  1        0      False

smart-sfp(root)(config)# show filter results profile0
Status:                false
Message:                ok
Duration:
  Start time:           --:--:--:-----
  Stop time:            --:--:--:-----
  Elapsed time:          00:00:00:000000
Selector filters of interface 0
  Number      Bytes      Packets
  0            0          0
  1            0          0
  2            0          0
  3            0          0
  4            0          0
  5            0          0
  6            0          0
  7            0          0
  8            0          0
  9            0          0
  10           0          0
  11           0          0
Selector filters of interface 1
```

```

Number          Bytes          Packets
0               0              0
1               0              0
2               0              0
3               0              0
4               0              0
5               0              0
6               0              0
7               0              0
8               0              0
9               0              0
10              0              0
11              0              0
smart-sfp(admin)(config)# show filter sel setting profile0
Selector filters:
Number  IP src  IP dst  Port src  Port dst  VLAN id  Route
0       any   any    any      any      any      any      none
1       any   any    any      any      any      any      none
2       any   any    any      any      any      any      none
3       any   any    any      any      any      any      none
4       any   any    any      any      any      any      none
5       any   any    any      any      any      any      none
6       any   any    any      any      any      any      none
7       any   any    any      any      any      any      none
8       any   any    any      any      any      any      none
9       any   any    any      any      any      any      none
10      any   any    any      any      any      any      none
11      any   any    any      any      any      any      none
smart-sfp(admin)(config)# show filter topology profile0
Interface: 1
      Status:  true
      Port:    portb
Interface: 0
      Status:  true
      Port:    porta

```

10.2.9.2. In profile configuration mode

topology

Command to bind physical port to the logical interface.

Syntax

```
topology {iface <0 | 1> port <a | b> status <on | off>}
```

Parameters

- iface <0 | 1> – interface number
- port <a | b> – physical port corresponding to the selected logical interface
- status <on | off> – enable or disable an interface for use in the profile 0/1

Default

iface 0 – port A, status – on.

iface 1 – port B, status – on.

Example

```
smart-sfp(root)(config-filter[profile0])# topology iface 0 port a status off
ok
```

show

Command to display topology and filter settings.

Syntax

```
show {topology | <cpu | sel> setting}
```

Parameters

- topology – show current interface topology settings: status (true – logical interface is used, false – logical interface not used), port – physical port corresponding to the logical interface, mode – not supported yet, host – not supported yet
- <cpu | sel> setting – cpu: show filter settings for cpu filters, sel: show filter settings for selector filters

Example

```
smart-sfp(admin)(config-filter[profile0])# show topology
Interface:  0
            Status:  true
            Port:    porta
            Mode:    local
            Host:
Interface:  1
            Status:  false
            Port:    portb
            Mode:    local
            Host:

smart-sfp(root)(config-filter[profile0])# show cpu setting
CPU filters:
  Number  Port  dst  Status
    0      0      false
    1      0      false

smart-sfp(root)(config-filter[profile0])# show sel setting
Selector filters:
  Number  IP src  IP dst  Port src  Port dst  VLAN id  Route
    0      0.0.0.0  0.0.0.0  0          0          0        none
    1      0.0.0.0  0.0.0.0  0          0          0        none
    2      0.0.0.0  0.0.0.0  0          0          0        none
    3      0.0.0.0  0.0.0.0  0          0          0        none
    4      0.0.0.0  0.0.0.0  0          0          0        none
    5      0.0.0.0  0.0.0.0  0          0          0        none
    6      0.0.0.0  0.0.0.0  0          0          0        none
    7      0.0.0.0  0.0.0.0  0          0          0        none
    8      0.0.0.0  0.0.0.0  0          0          0        none
    9      0.0.0.0  0.0.0.0  0          0          0        none
   10      0.0.0.0  0.0.0.0  0          0          0        none
   11      0.0.0.0  0.0.0.0  0          0          0        none
```

sel

Command to set filter options for selector filter.

Syntax

```
sel {num <value>} {ip {src <any | value> | dst <any | value>} | port {src <any | value> | dst <any | value>}} | vlan <any | value> | route <none | loopback | twamp>}
```

Parameters

- num <value> – filter number. Valid values: 0-11.
- ip src any – filtering of packets with any source IP address
- ip src <value> – filtering of packets with specified source IP address
- ip dst any – filtering of packets with any destination IP address
- ip dst <value> – filtering of packets with specified destination IP address
- port src any – filtering of packets with any source TCP/UDP port number
- port src <value> – filtering of packets with specified source TCP/UDP port number. Valid values: 0-65535.
- port dst any – filtering of packets with any destination TCP/UDP port number
- port dst <value> – filtering of packets with specified destination TCP/UDP port number. Valid values: 0-65535.
- vlan any – filtering of packets with any VLAN ID value
- vlan <value> – filtering of packets with specified VLAN ID value. Valid values: 0-4095.
- route <none | loopback | twamp> – package action mode: none – the packet will be processed in accordance with the general rules for packet transmission, loopback – the packet will be transmitted to the loopback, twamp – the packet will be transmitted to the twamp reflector

Note

For the TWAMP Light reflector to work, the “loopback” service must be enabled (see “loopback start” command). The value of the “layer” parameter can be any (layer2-layer4).

When the route of packets is "route = twamp" and the parameters "port src" and "port dst" are specified, it is necessary to specify the "ip dst" parameter, then the device will respond to ARP requests at the specified IP address. If the "ip dst" parameter is not specified, it may disrupt communication on the network.

Default

The selector filter is configured so that it does not loop and reflect anything.

Example

```
smart-sfp(root)(config-filter[profile0])# sel num 0 ip src 192.168.1.1
ok
smart-sfp(root)(config-filter[profile0])# sel num 0 ip dst 192.168.1.3
ok
smart-sfp(root)(config-filter[profile0])# sel num 0 port src 5000
ok
smart-sfp(root)(config-filter[profile0])# sel num 0 port dst 6000
ok
smart-sfp(root)(config-filter[profile0])# sel num 0 vlan id 7
ok
smart-sfp(root)(config-filter[profile0])# sel num 0 route loopback
ok
```


cpu

Command to set filter options for CPU filter.

Syntax

```
cpu {num <0 | 1>} {port <0-65535> status <on | off>}
```

Parameters

- num <0 | 1> – CPU filter number
- port <0-65535> – destination TCP/UDP port number
- status <on | off> – enable or disable filter

Default

CPU filter is off.

num 0, port dst 0, status – false.

num 1, port dst 0, status – false.

Example

```
smart-sfp(root)(config-filter[profile0])# cpu num 0 port 65535
ok
smart-sfp(root)(config-filter[profile0])# cpu num 0 status on
ok
```

10.2.10. Filters API

10.2.10.1. Filters settings change

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "0711798292de52e0fc7c07139c163086",
    "filters",
    "setprm",
    {
      "ids": {
        "profile": "profile0"
      },
      "parameters": {
        "status": false,
        "trial": {
          "ifaces": {
            "0": {
              "name": "porta",
              "mode": "local",
              "host": "",
              "disabled": false
            },
            "1": {
              "name": "portb",
              "mode": "local",
              "host": "",
              "disabled": true
            }
          }
        }
      }
    },
    "wait_time_ms": 1000,
  ]
}
```

```

        "learn_time_ms": 1000
      },
      "filter": {
        "duration_us": "0",
      }
    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "filters",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}

```

10.2.10.2. Filters settings view

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "579ba6e5001a2c30dfedc4df87c6f522",
    "filters",
    "getprm",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "filters",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "object": "filters",
          "method": "setprm",
          "ids": {
            "profile": "profile0"
          },
          "parameters": {
            "status": false,

```

```

        "trial": {
            "ifaces": {
                "0": {
                    "name": "porta",
                    "mode": "local",
                    "host": "",
                    "disabled":
                },
                "1": {
                    "name": "portb",
                    "mode": "local",
                    "host": "",
                    "disabled": true
                }
            },
            "wait_time_ms": 1000,
            "learn_time_ms": 1000
        },
        "filters": {
            "duration_us": "0",
        }
    }
}
]
}

```

10.2.10.3. Filters start

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "4f3f1b77d38cb303024001bd5bb5ab90",
        "filters",
        "start",
        {
            "ids": {
                "profile": "profile0"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "filters",
            "method": "start",
            "retcode": 0,
            "retmsg": "OK"
        }
    ]
}

```

10.2.10.1. Filters stop

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "0cfebb5ee35494b45b858c23df7c4b5e",
    "filters",
    "stop",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "filters",
      "method": "stop",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.10.2. Set selector filter options

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "4a546b5699d026addd84c45f8ec8a47f",
    "filters",
    "setprm",
    {
      "ids": {
        "profile": "profile0",
        "filter_type": "selector",
        "filter": 0
      },
      "parameters": {
        "route": "loopback",
        "ip_src": "0.0.0.0",
        "ip_dst": "0.0.0.0",
        "port_src": 0,
        "port_dst": 0,
        "vlan_id": 0
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "filters",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.10.3. View selector filter options

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "9ac07c1d33ea721b49403adf9f6d2fb6",
    "filters",
    "getprm",
    {
      "ids": {
        "profile": "profile0",
        "filter_type": "selector",
        "filter": 0
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "filters",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "object": "filters",
          "method": "setprm",
          "ids": {
            "profile": "profile0",
            "filter_type": "selector",
            "filter": 0
          },
          "parameters": {
            "enabled": false,
            "action": "loopback",
            "ip_src": "0.0.0.0",
            "ip_dst": "0.0.0.0",
            "port_src": 0,
            "port_dst": 0,
            "vlan_id": 0
          }
        }
      ]
    }
  ]
}
```

10.2.10.4. Set CPU filter options

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "4a546b5699d026add84c45f8ec8a47f",
    "filters",
    "setprm",
    {
      "ids": {
        "profile": "profile0",
        "filter_type": "cpu",
        "filter": 0
      },
      "parameters": {
        "port_dst": 0,
        "enabled": true
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "filters",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.10.5. View CPU filter options

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "9ac07c1d33ea721b49403adf9f6d2fb6",
    "filters",
    "getprm",
    {
      "ids": {
        "profile": "profile0",
        "filter_type": "cpu",
        "filter": 0
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "filters",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "object": "filter",
          "method": "setprm",
          "ids": {
            "profile": "profile0",
            "filter_type": "cpu",
            "filter": 0
          },
          "parameters": {
            "port_dst": 0,
            "enabled": true
          }
        }
      ]
    }
  ]
}
```

10.2.11. Filter statistics commands

10.2.11.1. In configuration mode

fstatistic clear

Command to clear filter statistic for the selected filter and profile.

Syntax

```
fstatistic clear {profile0 | profile1} {num <value> | range <A B> | all}
```

Parameters

- profile0 – clear filter statistic for the profile0
- profile1 – clear filter statistic for the profile1
- num <value> – filter number (0–35)
- range <A B> – filter number range from A to B; the minimum filter number (A) is 0, maximum (B) – 35.
- all – all filter numbers

Example

```
smart-sfp(admin)(config)# fstatistic clear profile0 range 4 6
ok
smart-sfp(admin)(config)# fstatistic clear profile0 num 35
ok
smart-sfp(admin)(config)# fstatistic clear profile0 all
ok
```

fstatistic config

Command to enter the configuration mode of the filter statistics function for the selected profile.

Syntax

```
fstatistic config {profile0 | profile1}
```

Parameters

- profile0 – enter the configuration mode for the profile0
- profile1 – enter the configuration mode for the profile1

Example

```
smart-sfp(admin)(config)# fstatistic config profile0
smart-sfp(admin)(config-fstatistic[profile0])#
```

fstatistic start

Command to enable filter statistics monitoring for the selected profile.

Syntax

```
fstatistic start {profile0 | profile1}
```

Parameters

- profile0 – enable filter statistics monitoring for the profile0
- profile1 – enable filter statistics monitoring for the profile1

Example

```
smart-sfp(root)(config)# fstatistic start profile0
ok
```

fstatistic stop

Command to disable filter statistics monitoring for the selected profile.

Syntax

```
fstatistic stop {profile0 | profile1}
```

Parameters

- profile0 – disable filter statistics monitoring for the profile0
- profile1 – disable filter statistics monitoring for the profile1

Example

```
smart-sfp(root)(config)# fstatistic stop profile1
ok
```

show fstatistic results

Command to display filter statistics results for the selected profile.

Syntax

```
show fstatistic results {profile0 | profile1}
```

Parameters

- profile0 – display filter statistics results for the profile0

- profile1 – display filter statistics results for the profile1

Example

```
smart-sfp(admin)(config)# show fstatistic results profile0
Status:      false
Message:     ok
Duration:
    Start time:  --:--:--.-----
    Stop time:   --:--:--.-----
    Elapsed time: --:--:--.-----
Number  Bytes  Packets
0        0      0
1        0      0
2        0      0
3        0      0
```

10.2.11.2. In profile configuration mode

filter

Command to set filter statistics aggregation parameters.

Syntax

```
filter {num <value> | range <A B> | all} {<on | off> |
    iface <a | b | all> |
    protocol <udp | tcp | mode <on | off | any>> |
    port <src | dst> <value | mode <on | off | any>> |
    ip <src | dst> <value | mode <on | off | any>> |
    mask <src | dst> <value> |
    vlan <pri | id> <value | mode <on|off|any>> |
    qos <precedence | diffserv | mode <on|off|any>> |
    precedence <value> |
    dscp <value>
```

Parameters

- num <value> – filter number (0–35)
- range <A B> – filter number range from A to B; the minimum filter number (A) is 0, maximum (B) – 35.
- all – all filter numbers
- <on | off> – enable or disable statistics aggregation for the selected filter
- iface <a | b | all> – port for which statistics will be aggregated: A, B or both.
- protocol <udp | tcp | mode <on | off | any>> – protocol type (UDP/TCP) and statistics aggregation mode:
 - “on” – statistics will be aggregated for packets with the specified protocol type
 - “off” – statistics will be aggregated for all packets (with any protocol type or without it)
 - “any” – statistics will be aggregated for packets with any protocol type
- port <src | dst> <value | mode <on | off | any>> – source or destination TCP/UDP port number and statistics aggregation mode:
 - “on” – statistics will be aggregated for packets with the specified TCP/UDP port number
 - “off” – statistics will be aggregated for all packets (with any TCP/UDP port number or without it)
 - “any” – statistics will be aggregated for packets with any TCP/UDP port number

Valid values for source and destination TCP/UDP port numbers: 0-65535.

- ip <src | dst> <value | mode <on | off | any>> – source or destination IP address and statistics aggregation mode:
 - “on” – statistics will be aggregated for packets with the specified IP address
 - “off” – statistics will be aggregated for all packets (with any IP address or without it)
 - “any” – statistics will be aggregated for packets with any IP address
- mask <src | dst> <value> – subnet mask value
- vlan <pri | id> <value | mode <on | off | any>> – VLAN identifier or VLAN priority and statistics aggregation mode:
 - “on” – statistics will be aggregated for packets with the specified VLAN identifier/VLAN priority
 - “off” – statistics will be aggregated for all packets (with any VLAN identifier/VLAN priority or without it)
 - “any” – statistics will be aggregated for packets with any VLAN identifier/VLAN priority

Vlan id valid values: 0-4095, vlan priority: 0-7.

- qos <precedence | diffserv | mode <on|off|any>> – Quality of Service type
- precedence <value> – precedence value. Valid values: 0-7.
- dscp <value> – DSCP value. Valid values: 0-63.

Default

Protocol – udp, ip src/dst – 0.0.0.0, port src/dst – any, mask – 0.0.0.0, vlan id – 0, vlan pri – 0, qos – 0, port – any, Enabled - false

Example

```
smart-sfp(admin)(config-fstatistic[profile0])# filter all ip src 192.168.10.1 mode on
ok
```

show setting

Command to display filter statistics information.

Syntax

show setting

Parameters

no arguments

Example

smart-sfp(root)(config-filter[profile0])# show setting												
Number	Protocol	IP src	Port src	IP dst	Port dst	VLAN id	VLAN pri	QoS	Port	Enabled		
0	tcp	192.168.2.1/255.255.255.127	off	off	off	off	off	off	any	false		
1	off	off	off	off	off	off	off	1	any	true		
2	off	off	off	off	off	off	off	off	any	false		
3	off	off	off	off	off	off	off	off	porta	false		
4	off	off	off	off	off	off	off	off	porta	false		
5	off	off	off	off	off	off	off	off	porta	false		

10.2.12. Filter statistics API

10.2.12.1. Change filter statistics settings

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "0711798292de52e0fc7c07139c163086",
    "fstd",
    "setprm",
    {
      "ids": {
        "profile": "profile0"
      },
      "parameters": {
        "status": false,
        "fstd": {
          "duration_us": "0",
        }
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "fstd",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.12.2. View filter statistics settings

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "579ba6e5001a2c30dfedc4df87c6f522",
    "fstd",
    "getprm",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
```

```

    "result": [
      0,
      {
        "object": "fstd",
        "method": "getprm",
        "retcode": 0,
        "retmsg": "OK",
        "answer": [
          {
            "object": "fstd",
            "method": "setprm",
            "ids": {
              "profile": "profile0"
            },
            "parameters": {
              "status": false,
              "fstd": {
                "duration_us": "0",
              }
            }
          }
        ]
      }
    ]
  }
}

```

10.2.12.3. View filter statistics activity

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "8e4d810676f4c9fb310189256d65eaa0",
    "fstd",
    "getsts",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "fstd",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "ids": {
            "profile": "profile0"
          },
          "statuses": {
            "status": false,
            "retcode": 0,
            "retmsg": "OK",
          }
        }
      ]
    }
  ]
}

```

```

        "sys_time": {
            "start_us": "7740420001",
            "stop_us": "7768040001",
            "elapsed_us": "27620000"
        },
        "meas_time": {
            "start_us": "7740520001",
            "stop_us": "7768020001",
            "elapsed_us": "27500000"
        },
        "pkts": "0",
        "bytes": "0"
    }
}
]
}

```

10.2.12.4. Change filter statistics aggregation parameters

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "4a546b5699d026add84c45f8ec8a47f",
        "fstd",
        "setprm",
        {
            "ids": {
                "profile": "profile0"
                "filter": 0
            },
            "parameters": {
                "status": false,
                "iface": "<porta | portb>",
                "iface_mode": "<off | on>",
                "proto": "udp",
                "proto_mode": "< on | off | any>",
                "header": {
                    "src": {
                        "net_ip": "192.168.1.1",
                        "net_mask": "192.168.1.1",
                        "net_mode": "< on | off | any>",
                        "udp_port": 6000,
                        "udp_port_mode": "< on | off | any>"
                    },
                    "dst": {
                        "net_ip": "192.168.2.2",
                        "net_mask": "192.168.1.1",
                        "net_mode": "< on | off | any>",
                        "udp_port": 5000,
                        "udp_port_mode": "< on | off | any>"
                    },
                    "vlan": {
                        "vlan_pri": 0,
                        "vlan_pri_mode": "< on | off | any>",
                        "vlan_id": 4095,
                        "vlan_id_mode": "< on | off | any>"
                    },
                    "qos": {
                        "qos_type": "diffserv",
                        "qos_mode": "< on | off | any>"
                    }
                }
            }
        }
    ]
}

```

```

        "dscp": 4,
        "ecn": 0,
        "tos": 4,
        "precedence": 3
    }
}
]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "fstd",
            "method": "setprm",
            "retcode": 0,
            "retmsg": "OK"
        }
    ]
}

```

10.2.12.5. View filter statistics aggregation parameters

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "9ac07c1d33ea721b49403adf9f6d2fb6",
        "fstd",
        "getprm",
        {
            "ids": {
                "profile": "profile0"
                "filter": 0
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "fstd",
            "method": "getprm",
            "retcode": 0,
            "retmsg": "OK",
            "answer": [
                {
                    "object": "fstd",
                    "method": "setprm",
                    "ids": {
                        "profile": "profile0"
                        "filter": 0
                    }
                },
            ]
        }
    ]
}

```

```

        "parameters": {
            "iface": "<porta | portb>",
            "iface_mode": "<off | on>",
            "proto": "udp",
            "proto_mode": "< on | off | any>",
            "header": {
                "src": {
                    "net_ip": "192.168.1.1",
                    "net_mask": "192.168.1.1",
                    "net_mode": "< on | off | any>",
                    "udp_port": 6000,
                    "udp_port_mode": "< on | off | any>"
                },
                "dst": {
                    "net_ip": "192.168.2.2",
                    "net_mask": "192.168.1.1",
                    "net_mode": "< on | off | any>",
                    "udp_port": 50000,
                    "udp_port_mode": "< on | off | any>"
                },
                "vlan": {
                    "vlan_pri": 0,
                    "vlan_pri_mode": "< on | off | any>",
                    "vlan_id": 4095,
                    "vlan_id_mode": "< on | off | any>"
                },
                "qos": {
                    "qos_type": "diffserv",
                    "qos_mode": "< on | off | any>",
                    "dscp": 4,
                    "ecn": 0,
                    "tos": 4,
                    "precedence": 3
                }
            }
        }
    }
}

```

10.2.12.6. View filter statistics aggregation results

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "8e4d810676f4c9fb310189256d65eaa0",
        "fstd",
        "getsts",
        {
            "ids": {
                "profile": "profile0"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,

```

```

"result": [
  0,
  {
    "object": "fstd",
    "method": "getsts",
    "retcode": 0,
    "retmsg": "OK",
    "answer": [
      {
        "ids": {
          "profile": "profile0"
          "filter": 0
        },
        "statuses": {
          "pkts": "0",
          "bytes": "0"
        }
      }
    ]
  }
]
]]]]

```

10.2.13. Time synchronization commands

10.2.13.1. In configuration mode

timesync config

Command to enter the configuration mode of time synchronization function for the selected profile.

Syntax

```
timesync config {profile0 | profile1}
```

Parameters

- profile0 – enter the configuration mode for the profile0
- profile1 – enter the configuration mode for the profile1

Example

```

smart-sfp(admin)(config)# timesync config profile0
smart-sfp(admin)(config-timesync[profile0])#

```

timesync start

Command to start time synchronization for the selected profile.

Syntax

```
timesync start {profile0 | profile1}
```

Parameters

- profile0 – start time synchronization for the profile0
- profile1 – start time synchronization for the profile1

Example

```

smart-sfp(root)(config)# timesync start profile0
ok

```

timesync stop

Command to stop time synchronization for the selected profile.

Syntax

```
timesync stop {profile0 | profile1}
```

Parameters

- profile0 – stop time synchronization for the profile0
- profile1 – stop time synchronization for the profile1

Example

```
smart-sfp(root)(config)# timesync stop profile0
ok
```

show timesync results

Command to display the results of the time synchronization function.

Syntax

```
show timesync results {profile0 | profile1}
```

Parameters

- profile0 – results for the profile0
- profile1 – results for the profile1

Example

```
smart-sfp(admin)# show timesync results profile0
Status:          false
Message:         ok
Start time:      --:--:--:-----
Stop time:       --:--:--:-----
Elapsed time:    00:00:00:000000
Offset:          0.00000
Delay:           0.00000
```

show timesync setting

Command to display the settings of the time synchronization function.

Syntax

```
show timesync setting {common | ntp | ptp} {profile0 | profile1}
```

Parameters

- common – common settings
- ntp – ntp settings
- ptp – ptp settings
- profile0 – settings for the profile0
- profile1 – settings for the profile1

Example

```
smart-sfp(admin)# show timesync setting common profile0
Status:          false
Duration:        endless
Mode:            slave
Type:            ptp
smart-sfp(admin)(config)# show timesync setting ntp profile0
Count:          1
```

```
Host 0: 192.168.5.2
smart-sfp(admin)(config)# show timesync setting ptp profile0
Domain: 0
Delay: p2p
```

show timesync topology

Command to display the topology of the synchronization function.

Syntax

```
show timesync topology {profile0 | profile1}
```

Parameters

- profile0 – topology for the profile0
- profile1 – topology for the profile1

Example

```
smart-sfp(admin)# show timesync topology profile0
Interface: 0
Status: true
Port: portb
Mode: local
Host:
```

10.2.13.2. In profile configuration mode

duration

Command to set the duration of the synchronization function operating.

Syntax

```
duration {endless | useconds <value>}
```

Parameters

- endless – synchronization function will work until the command “timesync stop profile 0/1” is entered
- useconds <value> – user-specified value in the format hh:mm:ss:uuuuuu, where hh – hours, mm – minutes, ss – seconds, uuuuuu – microseconds

Valid values: 00:00:00:000000-99:59:59:999999. “00:00:00:000000” means that the synchronization function will work until the “timesync stop profile 0/1” command is entered.

Default

endless

Example

```
smart-sfp(admin)(config-synctime[profile0])# duration endless
ok
smart-sfp(admin)(config-synctime[profile0])# duration useconds 24:00:00:000000
ok
```

mode

Command to select the synchronization mode.

Syntax

```
mode {master | slave}
```

Parameters

- master – master (server) operating mode
- slave – slave (client) operating mode

Default

slave

Example

```
smart-sfp(admin)(config-synctime[profile0])# mode slave
ok
```

ntp count

Command to set the NTP server number.

Syntax

ntp count <value>

Parameters

- <value> – number of NTP servers. Valid values: 1-4.

Default

1

Example

```
smart-sfp(admin)(config-synctime[profile0])# ntp count 1
ok
```

ntp host

Command to set NTP server IP address.

Syntax

ntp host <value> <ip address>

Parameters

- <value> – NTP server number. Valid values: 0-3.
- <ip address> – IP address of NTP server

Default

value = 0

ip address = 192.168.222.111

Example

```
smart-sfp(admin)(config-synctime[profile0])# ntp host 0 192.168.222.1
ok
```

ptp delay

Command to select PTP delay mode.

Syntax

ptp {delay <p2p | e2e>}

Parameters

- delay <p2p | e2e> – set PTP delay mode: P2P or E2E

Default

Delay: p2p

Example

```
smart-sfp(admin)(config-synctime[profile0])# ptp delay e2e
ok
```

ptp domain

Command to select the domain number.

Syntax

```
ptp domain {<value>}
```

Parameters

- <value> – synchronization domain number. Valid values: 0-255.

Default

0

Example

```
smart-sfp(admin)(config-synctime[profile0])# ptp domain 0
ok
```

show setting

Command to display the time synchronization function settings.

Syntax

```
show setting {common | ntp | ptp}
```

Parameters

- common – show common settings
- ntp – show NTP settings
- ptp – show PTP settings

Example

```
smart-sfp(admin)(config-synctime[profile0])# show setting common
      Status:    false
      Duration:  endless
      Mode:      slave
      Type:      ntp
smart-sfp(admin)(config-synctime[profile0])# show setting ntp
      Count:    1
      Host 0:   0.0.0.0
smart-sfp(admin)(config-synctime[profile0])# show setting ptp
      Domain:   0
      Delay:    p2p
```

show topology

Command to display the synchronization function topology settings.

Syntax

show topology

Example

```
smart-sfp(admin)(config-synctime[profile0])# show topology
Interface: 0
Status:   true
Port:     porta
Mode:     local
Host:
```

topology

Command to select the synchronization port.

Syntax

topology iface port {a | b}

Parameters

- <a | b > – synchronization port

Default

a

Example

```
smart-sfp(admin)(config)# topology iface port a
ok
```

type

Command to set the synchronization type.

Syntax

type {ntp | ptp}

Parameters

- <ntp | ptp> – selecting synchronization type: NTP or PTP
- ptp – use the PTP protocol for synchronization

Default

ntp

Example

```
smart-sfp(admin)(config)# type ntp
ok
```

10.2.14. Time synchronization API

10.2.14.1. Change time synchronization settings

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
```

```

"method": "call",
"params": [
  "0711798292de52e0fc7c07139c163086",
  "synctime",
  "setprm",
  {
    "ids": {
      "profile": "profile0"
    },
    "parameters": {
      "status": false,
      "trial": {
        "ifaces": {
          "0": {
            "name": "porta",
            "mode": "local",
            "host": "",
            "disabled": false
          }
        }
      },
      "synctime": {
        "mode": "slave",
        "type": "ntp",
        "duration": {
          "useconds": "60000000"
        }
      }
    }
  }
]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "synctime",
      "method": "setprm",
      "retcode": 0
    }
  ]
}

```

10.2.14.2. Change NTP settings

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "0711798292de52e0fc7c07139c163086",
    "synctime",
    "setprm",
    {
      "ids": {
        "profile": "profile0",
        "type": "ntp"
      },
      "parameters": {
        "hosts": {
          "count": 4
          "host_0": "192.177.77.0"
          "host_1": "192.177.77.1"
        }
      }
    }
  ]
}

```

```

        "host_2": "192.177.77.2"
        "host_3": "192.177.77.3"
    }
}]]}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "synctime",
            "method": "setprm",
            "retcode": 0
        }
    ]
}

```

10.2.14.3. Change PTP settings

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "0711798292de52e0fc7c07139c163086",
        "synctime",
        "setprm",
        {
            "ids": {
                "profile": "profile0",
                "type": "ptp"
            },
            "parameters": {
                "domain": 1,
                "delay": "e2e"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "synctime",
            "method": "setprm",
            "retcode": 0
        }
    ]
}

```

10.2.14.4. Time synchronization settings view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "9ac07c1d33ea721b49403adf9f6d2fb6",
    "synctime",
    "getprm",
    {
      "ids": {
        "profile": "profile0",
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "synctime",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "ids": {
          "profile": "profile0"
        },
        "parameters": {
          "status": false,
          "trial": {
            "ifaces": {
              "0": {
                "name": "porta",
                "mode": "local",
                "host":
                "disabled":
              }
            }
          },
          "synctime": {
            "mode": "slave",
            "type": "ntp",
            "duration": {
              "useconds": "60000000"
            }
          }
        }
      ]
    }
  ]
}
```

10.2.14.5. View NTP settings

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
```



```

        "9ac07c1d33ea721b49403adf9f6d2fb6",
        "synctime",
        "getprm",
        {
            "ids": {
                "profile": "profile0",
                "type": "ntp"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "synctime",
            "method": "getprm",
            "retcode": 0,
            "retmsg": "OK",
            "answer": [{
                "ids": {
                    "profile": "profile0",
                    "type": "ntp"
                },
                "parameters": {
                    "hosts": {
                        "count": 4
                        "host_0": "192.177.77.0"
                        "host_1": "192.177.77.1"
                        "host_2": "192.177.77.2"
                        "host_3": "192.177.77.3"
                    }
                }
            }]
        }
    ]
}

```

10.2.14.6. View PTP settings

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "9ac07c1d33ea721b49403adf9f6d2fb6",
        "synctime",
        "getprm",
        {
            "ids": {
                "profile": "profile0",
                "type": "ptp"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,

```

```

{
  "object": "synctime",
  "method": "getprm",
  "retcode": 0,
  "retmsg": "OK",
  "answer": [{
    "ids": {
      "profile": "profile0",
      "type": "ptp"
    },
    "parameters": {
      "domain": 1,
      "delay": "e2e"
    }
  }
]
}

```

10.2.14.7. Start time synchronization

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "9ac07c1d33ea721b49403adf9f6d2fb6",
    "synctime",
    "start",
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "synctime",
      "method": "start",
      "retcode": 0
    }
  ]
}

```

10.2.14.8. Stop time synchronization

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "9ac07c1d33ea721b49403adf9f6d2fb6",
    "synctime",
    "stop",
  ]
}

```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "synctime",
      "method": "stop",
      "retcode": 0
    }
  ]
}
```

10.2.15. Loaned IP function commands

10.2.15.1. In configuration mode

loaned-ip config

Command to enter loaned IP function configuration mode for the selected profile

Syntax

```
loaned-ip config {profile0 | profile1}
```

Parameters

- profile0 – enter configuration mode for the profile0
- profile1 – enter configuration mode for the profile1

Example

```
smart-sfp(root)(config)# loaned-ip config profile0
smart-sfp(root)(config-loaned-ip[profile0])#
```

loaned-ip start

Command to enable the loaned IP function for the selected profile.

Syntax

```
loaned-ip start {profile0 | profile1}
```

Parameters

- profile0 – enable the loaned IP function for the profile0
- profile1 – enable the loaned IP function for the profile1

Example

```
smart-sfp(root)(config)# loaned-ip start profile0
ok
```

loaned-ip stop

Command to disable the loaned IP function for the selected profile.

Syntax

```
loaned-ip stop {profile0 | profile1}
```

Parameters

- profile0 – enable the loaned IP function for the profile0
- profile1 – enable the loaned IP function for the profile1

Example

```
smart-sfp(root)(config)# loaned-ip stop profile1
ok
```

show loaned-ip results

Command to display the results of the loaned IP function.

Syntax

```
show loaned-ip results {profile0 | profile1}
```

Parameters

- profile0 – results for the profile0
- profile1 – results for the profile1

Example

```
smart-sfp(admin)# show loaned-ip results profile0
Status:                false
Message:                ok
Duration:
    Start time:         --:--:--:-----
    Stop time:          --:--:--:-----
    Elapsed time:       00:00:00:000000
```

show loaned-ip setting

Command to display the settings of the loaned IP function.

Syntax

```
show loaned-ip setting {profile0 | profile1}
```

Parameters

- profile0 – display settings for the profile0
- profile1 – display settings for the profile1

Example

```
smart-sfp(admin)# smart-sfp(admin)(config)# show loaned-ip setting profile0
Status:                false
IP address:             0.1.2.3
Port range:             22 - 22
```

show loaned-ip topology

Command to display the loaned IP topology.

Syntax

```
show loaned-ip topology {profile0 | profile1}
```

Parameters

- profile0 – topology for the profile0
- profile1 – topology for the profile1

Example

```
smart-sfp(admin)# show loaned-ip topology profile0
  Interface: ext
                Status: true
                Port:   porta
                Mode:   local
                Host:
  Interface: int
                Status: true
                Port:   portb
                Mode:   local
                Host:
```

10.2.15.2. In profile configuration mode

ip

Command to set loaned IP address.

Syntax

```
ip <xxx.xxx.xxx.xxx>
```

Parameters

- xxx.xxx.xxx.xxx – IP address

Default

0.1.2.3

Example

```
smart-sfp(admin)(config-loaned-ip[profile0])# ip 192.168.88.10
ok
```

range

Command to set TCP/UDP port range.

Syntax

```
range <min_range> <max_range>
```

Parameters

- <min_range> – minimum value of the port range. Valid values: 0-65535.
- <max_range> – maximum value of the port range. Valid values: 0-65535.

Default

22-22

Example

```
smart-sfp(admin)(config-loaned-ip[profile0])# range 22 28
ok
```

show

Command to display information about loaned IP address.

Syntax

```
show <setting> <topology>
```

Parameters

- <setting> – view current profile settings
- <topology> – view the current settings of the interface topology

Example

```
smart-sfp(admin)(config-loaned-ip[profile0])# show setting
    Status:      false
    IP address:  0.1.2.3
    Port range:  22 - 28
smart-sfp(admin)(config-loaned-ip[profile0])# show topology
    Interface: ext
                        Status: true
                        Port:   porta
                        Mode:   local
                        Host:
    Interface: int
                        Status: true
                        Port:   portb
                        Mode:   local
                        Host:
```

topology

Command to set loaned IP address topology.

Syntax

topology iface <ext | int> port <a | b> status <on | off>

Parameters

- <ext | int> – selection of the interface to be used for borrowing or disabling
- <a | b> – selection of the port
- <on | off> – enable/disable port in test topology

Default

topology iface ext port a status on

Example

```
smart-sfp(admin)(config-loaned-ip[profile0])# topology iface ext port a status on
ok
```

10.2.16. Loaned IP function API

10.2.16.1. Current loaned IP settings view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params":
[
  "4767a307c3f8959f3697e74966cea287",
    " loanedip ",
    "getprm",
    {
      "ids": {
```

```

        "profile": "profile0"
      }
    ]
  }

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "lip",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK"
      "answer":
    [
      {
        "object": "lip",
        "method": "setprm",
        "ids": {
          "profile": "profile0"
        },
        "parameters": {
          "status": false,
          "trial": {
            "ifaces": {
              "ext": {
                "name": "porta"
                "mode": "local",
                "host": "",
                "disabled": false
              },
              "int": {
                "name": "portb",
                "mode": "local",
                "host": "",
                "disabled": false
              }
            }
          },
          "wait_time_ms": 1000,
          "learn_time_ms": 1000
        },
        "loanedip": {
          "ip": "XXX.XXX.XXX.XXX",
          "min_port": 1000,
          "max_port": 2000,
          "duration_us": "0"
        }
      }
    ]
  ]
}

```

10.2.16.2. Loaned IP settings change

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params":
  [
    "0d9ff2d7a5362138d7b601cd8aada21b",

```

```

        "loanedip",
        "start",
        {
          "ids": {
            "profile": "profile0"
          }
        }
      ]
    }

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "lip",
      "method": "start",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}

```

10.2.16.3. Loaned IP service start

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "0d9ff2d7a5362138d7b601cd8aada21b",
    "loanedip",
    "start",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "lip",
      "method": "start",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}

```


10.2.16.4. Loaned IP service stop

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "ef43ded600b5929427a4566b1de308f8",
    "loanedip",
    "stop",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "lip",
      "method": "stop",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.16.5. Loaned IP service activity view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "8e4d810676f4c9fb310189256d65eaa0",
    "loanedip",
    "getsts",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "lip",
      "method": "getsts",
      "retcode": 0,

```


Syntax

```
erspan stop {profile0 | profile1}
```

Parameters

- profile0 – stop traffic capturing and mirroring for the profile0
- profile1 – stop traffic capturing and mirroring for the profile1

Example

```
smart-sfp(root)(config)# erspan stop profile0
ok
```

show erspan

Command to display traffic capturing and mirroring statistics and settings for the selected profile.

Syntax

```
show erspan {filters | results [capture | erspan | filter] | setting [erspan 0]
| topology [erspan | capture]} {profile0 | profile1}
```

Parameters

- filters – show filters settings
- results [capture | erspan | filter]:
 - no arguments – show common statistics
 - capture – display packet capturing statistics
 - erspan – display statistics of the captured packets mirroring rules
 - filter – show filters statistics
- setting [erspan 0] – display traffic mirroring settings, in the current software version, only one mirroring rule with number "0" is available;
- topology [erspan | capture]:
 - no arguments – show test topology settings (device port communication with the logical test interface)
 - erspan – show test topology settings, displaying the state and the port through which the captured traffic will be sent
 - capture – show test topology settings, displaying the status and ports on which the traffic will be captured.
- profile0 – results for the profile0
- profile1 – results for the profile1

Example

```
smart-sfp(root)(config)# show erspan filters profile0
Number Dir BSO Protocol IP src Port src IP dst Port dst VLAN id Espan Cap Enabled
0 rx off off off off off off off off off erspan0 capture0 false
1 rx off off off off off off off off off erspan0 capture0 false

smart-sfp(root)(config)# show erspan results profile0
Status: false
Message: ok
Duration:
Start time: --:--:--:-----
Stop time: --:--:--:-----
```

Elapsed time: --:--:--.-----

```
smart-sfp(root)(config)# show erspan results capture profile0
  Capture      Packets
  0             123
  1             0

smart-sfp(root)(config)# show erspan results erspan profile0
  Erspan      Packets      Drop packets
  0           123          0

smart-sfp(root)(config)# show erspan results filter profile0
  Filter  Packets
  0        0
  1        0

smart-sfp(root)(config)# show erspan setting profile0
  Status:      false
  Duration type: useconds
  Duration:    endless

smart-sfp(root)(config)# show erspan setting erspan 0 profile0
  Session ID:  1
  HW ID:       4
  MTU:         128
  Frame header:
    Src IP:     192.168.1.1
    Src MAC:    00:21:CE:28:00:2D
    Src port:   60000
    Dst IP:     192.168.2.2
    Dst MAC:    00:21:CE:28:00:2E
    Dst port:   50000
    VLAN 0:
      pri:      3
      id:       7
  QOS type:    diffserv
  dscp:        0
  tos:         0
  precedence:  0

smart-sfp(root)(config)# show erspan topology profile0
  Interface: capture0
    Status:      false
    Port:        porta
  Interface: capture1
    Status:      false
    Port:        portb
  Interface: erspan0
    Status:      false
    Port:        porta

smart-sfp(root)(config)# show erspan topology erspan profile0
  Interface: erspan0
    Status:      false
    Port:        porta

smart-sfp(root)(config)# show erspan topology capture profile0
  Interface: capture0
    Status:      true
    Port:        portb
  Interface: capture1
    Status:      true
    Port:        porta
```

erspan clear

Command to clear accumulated statistics for the selected filters.

Syntax

```
erspan clear {profile0 | profile1}
```

Parameters

- profile0 – clear statistics for the profile0
- profile1 – clear statistics for the profile1

Example

```
smart-sfp(root)(config)# erspan clear profile0
ok
```

10.2.17.2. In profile configuration mode

duration

Command to set traffic capturing duration.

Syntax

```
duration {endless | useconds <value>}
```

Parameters

- endless – packet capture will be performed until the command “erspan stop profile 0/1” is entered
- useconds <value> – capturing duration value in us in the format hh:mm:ss:uuuuuu, where hh – hours, mm – minutes, ss – seconds, uuuuuu – microseconds.

Valid values: 00:00:00:000000-99:59:59:999999. “00:00:00:000000” means that packet capturing will continue until the “erspan stop profile 0/1” command is entered.

Default

00:00:05

Example

```
smart-sfp(admin)(config-erspan[profile0])# duration endless
ok
smart-sfp(admin)(config-erspan[profile0])# duration useconds 00:00:00:000200
ok
```

erspan

Command to configure the traffic capturing and mirroring function.

Syntax

```
erspan 0 {session id <value> | hw_id <value> | mtu <value> | header {{<src | dst>
ip <value>} | {{<src | dst> mac <value | auto>} | {{<src | dst> udp <value>} | {vlan
count <off | 1 | 2 | 3>} | {vlan number <value> id <value> pri <value>} | {qos
<precedence | diffserv>} | {tos <value>} | {preced <value>} | {dscp <value>}}}}
```

Parameters

- session id <value> – ERSPAN Type III header Session ID value
- hw_id <value> – ERSPAN Type III header Hw ID value
- mtu <value> – MTU value (frame size). Valid values: 67–2048.
- header:

- <src | dst> ip <value> – source or destination IP address
- <src | dst> mac <value | auto> – source or destination MAC address; “auto” – MAC address will be assigned automatically based on the previously entered IP address
- vlan count <off | 1 > – VLAN label count (“off” – VLAN tag insertion is disabled)
- vlan number <value> id <value> pri <value> – VLAN identifier and VLAN priority for the selected VLAN number. Valid values of vlan number: 0, vlan id: 0–4095, vlan priority: 0–7.
- qos <precedence | diffserv> – Quality of Service type
- tos <value> – Type of Service value. Valid values: 0–15.
- preced <value> – precedence value. Valid values: 0–7.
- dscp <value> – DSCP value. Valid values: 0–63.

Default

MTU – 128 bytes

Header:

```

Src IP: 192.168.1.1
Src MAC: 00:21:CE:28:00:2D
Src port: 60000
Dst IP: 192.168.2.2
Dst MAC: 00:21:CE:28:00:2E
Dst port: 50000
VLAN: off
QOS type: diffserv
        dscp: 0
        tos: 0
        precedence: 0

```

Example

```

smart-sfp(root)(config-erspan[profile0])# erspan 0 session_id 3
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 hw_id 4
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 mtu 512
ok

Vsmart-sfp(root)(config-erspan[profile0])# erspan 0 header src ip 10.0.0.1
ok

Vsmart-sfp(root)(config-erspan[profile0])# erspan 0 header dst ip 10.0.0.3
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 header src mac 00:21:CE:28:00:2D
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 header dst auto
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 src udp 1234

```

```

ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 dst udp 4321
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 header vlan count 1
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 header vlan number 0 id 7 pri 3
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 header qos precedence
ok

smart-sfp(root)(config-erspan[profile0])# erspan 0 header dscp 4

```

filter

Command to set traffic capturing and mirroring parameters.

Syntax

```

filter {num <value> | range <X Y> | all} {<on | off> | erspan 0 | capture <N> |
direction <rx | tx> | bso <on | off> | protocol <udp | tcp | mode <on | off | any>>
| ip <src | dst> <value | mode <on | off | any>> | port <src | dst> <value | mode
<on | off | any>> | vlan id <value | mode <on|off|any>>}}

```

Parameters

- num <value> – filter number
- range <X Y> – filter numbers range from X to Y; the minimum filter number is 0, the maximum filter number is 1;
- all – all filter numbers
- <on | off> – enable/disable the selected filter
- erspan 0 – the number of the mirroring rule for captured packets
- capture <N> – rule number to specify the source of traffic capturing. Valid values: 0, 1.
- direction <rx | tx> – traffic capturing direction, rx – packets entering the port, tx – packets leaving the port
- bso <on | off> – enable / disable capture of packets with errors
- protocol <ICMP | IGMP | TCP | UDP | GRE | mode <on | off | any>> – protocol type (ICMP/IGMP/TCP/UDP/GRE) and traffic capture mode:
 - “on” – capture will be performed for packets with the specified protocol type
 - “off” – capture will be performed for all packets (with any protocol type or without it)
 - “any” – capture will be performed for packets with any protocol type
- ip <src | dst> <value | mode <on | off | any>> – source or destination IP address and traffic capture mode:
 - “on” – capture will be performed for packets with the specified IP address
 - “off” – capture will be performed for all packets (with any IP address or without it)
 - “any” – capture will be performed for packets with any IP address
- port <src | dst> <value | mode <on | off | any>> – source or destination TCP/UDP port number and traffic capture mode:

- “on” – capture will be performed for packets with the specified TCP/UDP port number
- “off” – capture will be performed for all packets (with any TCP/UDP port number or without it)
- “any” – capture will be performed for packets with any TCP/UDP port number

Valid values for source and destination TCP/UDP port numbers: 0-65535.

- vlan id <value | mode <on | off | any>> – VLAN identifier (valid values: 1–4095) and traffic capture mode:
 - “on” – capture will be performed for packets with the specified VLAN identifier
 - “off” – capture will be performed for all packets (with any VLAN identifier or without it)
 - “any” – capture will be performed for packets with any VLAN identifier

Default

Filters are turned off.

Filters parameter values:

Parameter	Value	Mode
Ersparn	Ersparn0	-
Capture	Caprur0	-
Direction	Rx	-
Protocol	UDP	Off
IP src	192.168.1.1	Off
IP dst	192.168.2.2	Off
Port src	6000	Off
Port dst	50000	Off
VLAN id	4095	Off
BSO	-	Off
Ersparn	Ersparn0	-
Capture	Caprur0	-

Example

```

smart-sfp(root)(config-ersparn[profile0])# filter num 0 on
ok

smart-sfp(root)(config-ersparn[profile0])# filter range 0 1 ersparn 0
ok

smart-sfp(root)(config-ersparn[profile0])# filter num 1 capture 0
ok

smart-sfp(root)(config-ersparn[profile0])# filter all direction rx
ok

smart-sfp(root)(config-ersparn[profile0])# filter num 1 bso off
ok

smart-sfp(root)(config-ersparn[profile0])# filter all protocol udp
ok

smart-sfp(root)(config-ersparn[profile0])# filter range 0 1 protocol mode any
ok

smart-sfp(root)(config-ersparn[profile0])# filter num 1 ip src 192.168.1.1
ok

smart-sfp(root)(config-ersparn[profile0])# filter num 0 ip src mode off

```



```

ok

smart-sfp(root)(config-erspan[profile0])# filter range 0 1 ip dst 192.168.1.3
ok

smart-sfp(root)(config-erspan[profile0])# filter num 1 ip dst mode on
ok

smart-sfp(root)(config-erspan[profile0])# filter all port src 5000
ok

smart-sfp(root)(config-erspan[profile0])# filter num 1 port src mode any
ok

smart-sfp(root)(config-erspan[profile0])# filter range 0 1 port dst 6000
ok

smart-sfp(root)(config-erspan[profile0])# filter num 1 port dst mode on
ok

smart-sfp(root)(config-erspan[profile0])# filter all vlan id 7
ok

smart-sfp(root)(config-erspan[profile0])# filter range 0 1 vlan id mode off
ok

```

show

Command to show information about settings and topology of the traffic capturing and mirroring function.

Syntax

```
show {erspan 0 | filter | setting | topology [erspan | capture]}
```

Parameters

- erspan 0 – display traffic mirroring settings, in the current software version, only one mirroring rule with number “0” is available
- filter – show filters settings
- setting – show traffic capturing and mirroring basic settings
- topology – show current topology settings:
 - no arguments – show test topology settings (device port communication with the logical test interface)
 - erspan – show test topology settings, displaying the state and the port through which the captured traffic will be sent
 - capture – show test topology settings, displaying the status and ports on which the traffic will be captured

Example

```

smart-sfp(root)(config-erspan[profile0])# show erspan 0
  Session ID:      0
  HW ID:           0
  MTU:             128
  Frame header:
    Src IP:         192.168.1.1
    Src MAC:        00:21:CE:28:00:2D
    Src port:       60000
    Dst IP:         192.168.2.2
    Dst MAC:        00:21:CE:28:00:2E
    Dst port:       50000

```

```

QoS type:      diffserv
dscp:          0
tos:           0
precedence:    0

smart-sfp(root)(config-erspan[profile0])# show filter
Number  Dir  BSO  Protocol  IP src  Port src  IP dst  Port dst  VLAN id  Espan  Cap  Enabled
0       rx  off  off       off     off      off     off      off      erspan0  capture0  false
1       rx  off  off       off     off      off     off      off      erspan0  capture0  false

smart-sfp(root)(config-erspan[profile0])# show setting
Status:      false
Duration type:  useconds
Duration:     endless

smart-sfp(root)(config-erspan[profile0])# show topology
Interface: capture0
Status:      true
Port:        portb
Interface: capture1
Status:      true
Port:        porta
Interface: erspan0
Status:      true
Port:        porta

smart-sfp(root)(config-erspan[profile0])# show topology erspan
Interface: erspan0
Status:      true
Port:        porta

smart-sfp(root)(config-erspan[profile0])# show topology capture
Interface: capture0
Status:      true
Port:        portb
Interface: capture1
Status:      true
Port:        porta

```

topology

Command to define the physical ports on which traffic capturing and mirroring will be performed.

Syntax

```
topology {iface <erspan0 | capture0 | capture1> port <a | b> status <on | off>}
```

Parameters

- iface <erspan0 | capture0 | capture1> – logical interface that defines the operation with traffic – mirroring (erspan0) or capturing (capture0 and capture1)
- port <a | b> – physical port on which the selected operation will be performed
- status <true | false> – enable or disable using interface in profile 0/1.

Default

capture0 – port b, status – true

capture1 – port a, status – true

erspan0 – port a, status – true

Example

```
smart-sfp(root)(config-erspan[profile0])# topology iface capture1 status off
ok
```

```
smart-sfp(root)(config-erspan[profile0])# topology iface capture0 port b status on
ok

smart-sfp(root)(config-erspan[profile0])# topology iface erspan0 port a status on
ok
```

10.2.18. Traffic capturing and mirroring API

10.2.18.1. Common settings change

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "0711798292de52e0fc7c07139c163086",
    "erspan",
    "setprm",
    {
      "ids": {
        "profile": "profile0"
      },
      "parameters": {
        "status": false,
        "trial": {
          "ifaces": {
            "erspan0": {
              "name": "porta",
              "mode": "local",
              "host": "",
              "disabled": false
            },
            "capture0": {
              "name": "porta",
              "mode": "local",
              "host": "",
              "disabled": false
            },
            "capture1": {
              "name": "portb",
              "mode": "local",
              "host": "",
              "disabled": false
            }
          }
        },
        "span": {
          "duration": {
            "type": "useconds",
            "useconds": "5000000",
            "packets": "0",
            "bytes": "1000"
          }
        }
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
```

```

        "id": 1,
        "result": [
            0,
            {
                "object": "erspan",
                "method": "setprm",
                "retcode": 0,
                "retmsg": "OK"
            }
        ]
    }

```

10.2.18.2. Common settings view

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "4767a307c3f8959f3697e74966cea287",
        "erspan",
        "getprm",
        {
            "ids": {
                "profile": "profile0"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "erspan",
            "method": "getprm",
            "retcode": 0,
            "retmsg": "OK",
            "answer": [{
                "object": "erspan",
                "method": "setprm",
                "ids": {
                    "profile": "profile0"
                }
            },
            "parameters": {
                "status": false,
                "trial": {
                    "ifaces": {
                        "erspan0": {
                            "name": "porta",
                            "mode": "local",
                            "host": "",
                            "disabled": false
                        },
                        "capture0": {
                            "name": "porta",
                            "mode": "local",
                            "host": "",
                            "disabled": false
                        },
                        "capture1": {

```

```

        "name": "portb",
        "mode": "local",
        "host": "",
        "disabled": false
    }
},
"span": {
    "duration": {
        "type": "useconds",
        "useconds": "5000000",
        "packets": "0",
        "bytes": "1000"
    }
}
}]
}
]
}

```

10.2.18.3. Captured packet mirroring settings change

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "0711798292de52e0fc7c07139c163086",
        "erspan",
        "setprm",
        {
            "ids": {
                "profile": "profile0",
                "iface": "erspan0"
            },
            "parameters": {
                "session_id": 0,
                "hw_id": 0,
                "mtu": 128
            },
            "header": {
                "src": {
                    "mac": "00:21:CE:28:00:2D",
                    "ip": "192.168.1.1",
                    "udp_port": 60000
                },
                "dst": {
                    "mac": "00:21:CE:28:00:2E",
                    "ip": "192.168.2.2",
                    "udp_port": 50000
                },
                "vlan": {
                    "count": 0,
                    "tags": [{
                        "pri": 0,
                        "id": 4095
                    },
                    {
                        "pri": 0,
                        "id": 4094
                    },
                    {
                        "pri": 0,
                        "id": 4093
                    }
                ]
            }
        }
    ]
}

```


Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "erspan",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "object": "erspan",
        "method": "setprm",
        "ids": {
          "profile": "profile0",
          "iface": "erspan"
        },
        "parameters": {
          "session_id": 0,
          "hw_id": 0,
          "mtu": 128
        },
        "header": {
          "src": {
            "mac": "00:21:CE:28:00:2D",
            "ip": "192.168.1.1",
            "udp_port": 60000
          },
          "dst": {
            "mac": "00:21:CE:28:00:2E",
            "ip": "192.168.2.2",
            "udp_port": 50000
          },
          "vlan": {
            "count": 0,
            "tags": [{
              "pri": 0,
              "id": 4095
            },
            {
              "pri": 0,
              "id": 4094
            },
            {
              "pri": 0,
              "id": 4093
            }
          ]
        },
        "mpls": {
          "count": 1,
          "labels": [{
            "value": 128,
            "tc": 0,
            "ttl": 0
          },
          {
            "value": 128,
            "tc": 0,
            "ttl": 0
          },
          {
            "value": 128,
            "tc": 0,
            "ttl": 0
          }
        ]
      }
    ]
  ],
}
```

```

        "qos_type": "diffserv",
        "dscp": 0,
        "ecn": 0,
        "tos": 0,
        "precedence": 0
    }
}
]]]]

```

10.2.18.5. Capturing packets settings view

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "4767a307c3f8959f3697e74966cea287",
        "erspan",
        "getprm",
        {
            "ids": {
                "profile": "profile0",
                "iface": "capture0"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "erspan",
            "method": "getprm",
            "retcode": 0,
            "retmsg": "OK",
            "answer": [{
                "object": "erspan",
                "method": "setprm",
                "ids": {
                    "profile": "profile0",
                    "iface": "capture0"
                },
                "parameters": {
                }
            }]
        }
    ]
}

```

10.2.18.6. Filters settings change

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "0711798292de52e0fc7c07139c163086",

```



```

        "erspan",
        "setprm",
        {
            "ids": {
                "profile": "profile0"
                "filter": 0
            },
            "parameters": {
                "status": false,
                "erspan": "erspan0"
                "capture": "capture0"

                "direction": "rx",

                "bso": "< on | off >",

                "proto": "udp",
                "proto_mode": "< on | off | any>",

                "header": {
                    "src": {
                        "ip": "192.168.1.1",
                        "ip_mode": "< on | off | any>",
                        "port": 6000,
                        "port_mode": "< on | off | any>"
                    },
                    "dst": {
                        "ip": "192.168.2.2",
                        "ip_mode": "< on | off | any>",
                        "port": 50000,
                        "port_mode": "< on | off | any>"
                    },
                    "vlan": {
                        "id": 4095,
                        "id_mode": "< on | off | any>"
                    }
                }
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "erspan",
            "method": "setprm",
            "retcode": 0,
            "retmsg": "OK"
        }
    ]
}

```

10.2.18.7. Filters settings view

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [

```

```

"4767a307c3f8959f3697e74966cea287",
"erspan",
"getprm",
{
    "ids": {
        "profile": "profile0",
        "filter": 0
    }
}
]]}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "erspan",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "object": "erspan",
        "method": "setprm",
        "ids": {
          "profile": "profile0",
          "filter": 0
        },
        "parameters": {
          "status": false,

          "erspan": "erspan0"
          "capture": "capture0"

          "direction": "rx",

          "bso": "< on | off >",

          "proto": "udp",
          "proto_mode": "< on | off | any>",

          "header": {
            "src": {
              "ip": "192.168.1.1",
              "ip_mode": "< on | off | any>",
              "port": 6000,
              "port_mode": "< on | off | any>"
            },
            "dst": {
              "ip": "192.168.2.2",
              "ip_mode": "< on | off | any>",
              "port": 50000,
              "port_mode": "< on | off | any>"
            },
            "vlan": {
              "id": 4095,
              "id_mode": "< on | off | any>"
            }
          }
        }
      ]
    }
  ]
}

```

10.2.18.8. Common statistics view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "8e4d810676f4c9fb310189256d65eaa0",
    "erspan",
    "getsts",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "erspan",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "ids": {
            "profile": "profile0"
          },
          "statuses": {
            "status": false,
            "retcode": 0,
            "retmsg": "OK",
            "sys_time": {
              "start_us": "7740420001",
              "stop_us": "7768040001",
              "elapsed_us": "27620000"
            }
          }
        }
      ]
    }
  ]
}
```

10.2.18.9. Captured packet mirroring statistics view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "8e4d810676f4c9fb310189256d65eaa0",
    "erspan",
    "getsts",
    {
      "ids": {
```

```

        "profile": "profile0",
        "iface": "erspan0"
    }
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "erspan",
            "method": "getsts",
            "retcode": 0,
            "retmsg": "OK",
            "answer": [
                {
                    "ids": {
                        "profile": "profile0",
                        "iface": "erspan0"
                    },
                    "statuses": {
                        "pkts": "0",
                        "pkts_drop": "0"
                    }
                }
            ]
        }
    ]
}

```

10.2.18.10. Capturing packets statistics view

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "8e4d810676f4c9fb310189256d65eaa0",
        "erspan",
        "getsts",
        {
            "ids": {
                "profile": "profile0",
                "iface": "capture0"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "erspan",

```

```

"method": "getsts",
"retcode": 0,
"retmsg": "OK",
"answer": [
  {
    "ids": {
      "profile": "profile0",
      "iface": "erspan0"
    },
    "statuses": {
      "pkts": "0"
    }
  }
]
]]]]

```

10.2.18.11. Filters statistics view

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "8e4d810676f4c9fb310189256d65eaa0",
    "erspan",
    "getsts",
    {
      "ids": {
        "profile": "profile0",
        "filter": 0
      }
    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "erspan",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "ids": {
            "profile": "profile0",
            "filter": 0
          },
          "statuses": {
            "pkts": "0"
          }
        }
      ]
    }
  ]
}

```

10.2.19. Commands for the service activation test with interval statistics generator function

10.2.19.1. In configuration mode

wlt-tx config

Command to enter the configuration mode of the traffic generation function for the selected profile.

Syntax

```
wlt-tx config {profile0 | profile1}
```

Parameters

- profile0 – enter configuration mode for the profile0
- profile1 – enter configuration mode for the profile1

Example

```
smart-sfp(admin)(config)# wlt-tx config profile0  
smart-sfp(admin)(config-wlt-tx [profile0])#
```

wlt-tx start

Command to start the traffic generation for the selected profile.

Syntax

```
wlt-tx start {profile0 | profile1}
```

Parameters

- profile0 – start the traffic generation for the profile0
- profile1 – start the traffic generation for the profile1

Example

```
smart-sfp(admin)(config)# wlt-tx start profile0  
ok
```

wlt-tx stop

Command to stop the traffic generation for the selected profile.

Syntax

```
wlt-tx stop {profile0 | profile1}
```

Parameters

- profile0 – start the traffic generation for the profile0
- profile1 – start the traffic generation for the profile1

Example

```
smart-sfp(admin)(config)# wlt-tx stop profile0  
ok
```

show wlt-tx header

Command to display the header settings of the traffic generator.

Syntax

```
show wlt-tx header {profile0 | profile1}
```

Parameters

- profile0 – header settings for the profile0
- profile1 – header settings for the profile1

Example

```
smart-sfp(admin)(config)# show wlt-tx header profile0
  Src IP:          192.168.1.1
  Src MAC:         00:21:CE:28:00:2D
  Src port:        60000
  Dst IP:          192.168.2.2
  Dst MAC:         00:21:CE:28:00:2E
  Dst port:        50000
  QoS type:        diffserv
                   dscp:      0
                   tos:       0
                   precedence: 0
```

show wlt-tx results local

Command to display information about interval statistics of the generator.

Syntax

```
show wlt-tx results local {profile0 | profile1}
```

Parameters

- profile0 – display statistics for the profile0
- profile1 – display statistics for the profile1

Example

```
smart-sfp(admin)(config)# show wlt-tx results local profile0
  Status:          false
  Test duration:
    Start time:    12:08:02
    Stop time:     12:08:48
    Elapsed time:  00:00:45
  Bucket (num):    4
  Packets (num):   14880952
  Bytes (num):     952380928
  Rate (L1):       999.99997 mbps
  Rate (L2):       761.90474 mbps
  Rate (L3):       547.61903 mbps
  Rate (L4):       309.52380 mbps
  Meas duration:
    Start time:    01:56:32
    Stop time:     01:56:42
    Elapsed time:  00:00:10
```

show wlt-tx results total

Command to display information about integral statistics of the generator.

Syntax

```
show wlt-tx results total {profile0 | profile1}
```

Parameters

- profile0 – display statistics for the profile0
- profile1 – display statistics for the profile1

Example

```
smart-sfp(admin)(config)# show wlt-tx results total profile0
  Status:                false
  Message:                ok
  Test duration:
    Start time:          12:08:02
    Stop time:           12:08:48
    Elapsed time:        00:00:45
  Bucket (num):          4
  Packets (num):          59523810
  Bytes (num):            3809523840
  Rate (L1):              1000.00001 mbps
  Rate (L2):              761.90477 mbps
  Rate (L3):              547.61905 mbps
  Rate (L4):              309.52381 mbps
  Meas duration:
    Start time:          01:56:02
    Stop time:           --:--:--
    Elapsed time:        00:00:40
```

show wlt-tx setting

Command to display the current generator status and settings.

Syntax

```
show wlt-tx setting {profile0 | profile1}
```

Parameters

- profile0 – display settings for the profile0
- profile1 – display settings for the profile1

Example

```
smart-sfp(admin)(config)# show wlt-tx setting profile0
  Status:                false
  Port (name):            porta
  Frame size (byte):      64
  Flow ID (num):          4
  Interval(sec):          10
  Duration type:          seconds
  Duration (sec):         01:01:00
  Rate:                   100.00000 (l2 percents)
```

10.2.19.2. In profile configuration mode

duration

Command to set the duration of traffic generation.

Syntax

```
duration time <value>
```


Parameters

- <value> – generation duration in the “hh:mm:ss” format. Valid values: 00:00:00–99:59:59. “00:00:00” means that the traffic generation function will work until the “wlt-tx stop” command is entered.

Default

Default test duration by is 1 minute – 00:01:00.

Note

The generation duration value may not be a multiple of the interval duration. In this case, the integral statistics will display the correct number of received bytes, packets, losses, and other metrics. The last interval in the interval statistics will have a speed slower than the other intervals, but this speed will not affect the speed displayed in the integral statistics.

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# duration time 01:00:00
ok
```

flow

Command to set flow identifier value for the test stream. The value is set by the generator and is necessary for correct test traffic identification on the analyzer.

Syntax

```
flow id <value>
```

Parameters

- <value> – test stream unique identifier

Default

4

Note

“flow id” value for analyzer and generator must be the same.

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# flow id 1
ok
```

frame

Command to set the frame size.

Syntax

```
frame size <value>
```

Parameters

- <value> – frame size in bytes

Default

64

Note

Minimum frame size (L2) – 64 bytes. Maximum frame size (L2) – 9600 bytes.

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# frame size 512
ok
```

When using the VLAN tag in test traffic, you must specify the right packet size for the generator and analyzer to calculate the test results correctly.

#	Test case	VLAN tag	Packet size X bytes	Test stream header setting
1	Generator	No	X	header vlan count off
	Analyzer	No	X	header vlan count off
2	Generator	Yes	X	header vlan count <number of tags>
	Analyzer	Yes	X	header vlan count <number of tags>
3	Generator	Yes	X	header vlan count <number of tags>
	Analyzer	No	$Y = X - (4 * \text{<number of tags>})$, if $Y < 64$, then specify 64	header vlan count off
4	Generator	No	X	header vlan count off
	Analyzer	Yes	$Y = X + (4 * \text{<number of tags>})$	header vlan count <number of tags>

On the generator and analyzer, you can only set the rate at L3 or L4; the rate value for the generator and analyzer should be the same. In the analyzer and generator statistics, the L1 and L2 rates for the same interval will not match — this is expected behavior since the vlan tag is not taken into account in this case.

header

Command to set the header parameters.

Syntax

```
header {dscp <value>} | {dst ip <value>} | {dst mac <value>} | {dst udp <value>}  
| {precedence <value>} | {qos type <precedence | diffserv>} | {src ip <value>} |  
{src mac <value>} | {src udp <value>} | {tos <value>} | {vlan count <off | 1 | 2 |  
3>} | {vlan number <0 | 1 | 2> id <value> pri <value>}
```

Parameters

- dscp <value> – DSCP value. Valid values: 0–63.
- dst ip <value> – destination IP address
- dst mac <value> – destination MAC address
- dst udp <value> – destination UDP port number. Valid values: 0–65535.
- precedence <value> – precedence value. Valid values: 0–7.
- qos type <precedence | diffserv> – Quality of Service value
- src ip <value> – source IP address
- src mac <value> – source MAC address

- src udp <value> – source UDP port number. Valid values: 0-65535.
- tos <value> – Type of Service value. Valid values: 0-15.
- vlan count <off | 1 | 2 | 3> – number of VLAN tags: 1, 2, or 3 (“off” – VLAN tag insertion is disabled)
- vlan number <0 | 1 | 2> id <value> pri <value> – VLAN ID or VLAN priority value for the specified VLAN number. Vlan id valid values: 0-4095, vlan priority: 0-7.

Default

Src IP: 192.168.1.1

Src MAC: 00:21:CE:28:00:2D

Src port: 60000

Dst IP: 192.168.2.2

Dst MAC: 00:21:CE:28:00:2E

Dst port: 50000

QOS type: diffserv

dscp: 0

tos: 0

precedence: 0

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# header dscp 4
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header dst ip 192.168.2.11
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header dst mac 00:21:ce:2c:09:e3
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header dst udp 9090
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header precedence 4
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header qos type diffserv
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header src ip 192.168.2.1
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header src mac 00:21:ce:2c:0c:02
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header src udp 9090
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header tos 2
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header vlan count 1
ok
smart-sfp(admin)(config-wlt-tx [profile0])# header vlan number 1 id 10 pri 1
ok
```

interval

Command to set the time interval of a certain duration, within which the analyzer counts statistics (losses, latency, jitter) are calculated on the analyzer.

Syntax

interval <value>

Parameters

- <value> – the duration of the interval is set in the range from 1 s to 60 s, step 1 s

Default

10

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# interval 15
ok
```

port

Command to select the port for traffic generation.

Syntax

port {a}

Parameters

- a – port A

Default

a

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# port a
ok
```

rate

Command to select traffic generation rate.

Syntax

rate <value> {percents | kbps | mbps} {layer1 | layer2 | layer3 | layer4 | layer5}

Parameters

- <value> <percents | kbps | mbps> – rate value in percents, kbps or Mbps. Valid values: 0–100%; for kbps and mbps, valid values will depend on the selected layer and frame size for the service (see section 16.12).
- <layer1 | layer2 | layer3 | layer4 | layer5> – rate layer

Default

100.00000 (12 percents)

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# rate 100 mbps layer3
ok
```

show header

Command to display the header settings.

Syntax

show header

Parameters

no arguments

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# show header
  Src IP:          192.168.1.1
  Src MAC:         00:21:CE:28:00:2D
  Src port:        60000
  Dst IP:          192.168.2.2
  Dst MAC:         00:21:CE:28:00:2E
  Dst port:        50000
  QoS type:        diffserv
                   dscp:      0
                   tos:       0
                   precedence: 0
```

show setting

Command to display the generator settings.

Syntax

show setting

Parameters

no arguments

Example

```
smart-sfp(admin)(config-wlt-tx [profile0])# show setting
  Status:      false
  Port (name): porta
  Frame size (byte): 512
  Flow ID (num): 1
  Interval(sec): 15
  Duration type: seconds
  Duration (sec): 01:00:00
  Rate:        100.00000 (13 mbps)
```

10.2.20. Service activation test with interval statistics generator function API

10.2.20.1. Current generator settings view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "4767a307c3f8959f3697e74966cea287",
    "wlt-tx",
    "getprm",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "wlt-tx",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "object": "wlt-x",
        "method": "setprm",
        "ids": {
          "profile": "profile0"
        },
        "parameters": {
          "status": false,
          "wlt": {
            "flow_id": 3,
            "interval": 10,
            "framesize": 64
          },
          "duration": {
            "type": "seconds",
            "seconds": 1800,
            "packets": "0",
            "bytes": "0"
          },
          "rate": {
            "value": "100.00000",
            "units": "percents",
            "layer": 2
          },
          "trial": {
            "ifaces": {
              "tx": {
                "name": "portb",
                "mode": "local",
                "host": "",
                "disabled": false
              }
            },
            "wait_time_ms": 1000,
            "learn_time_ms": 0
          },
          "header": {
            "src": {
              "mac": "00:01:02:03:04:05",
              "ip": "192.168.1.1",
              "udp_port": 5000
            },
            "dst": {
              "mac": "00:21:CE:28:00:2E",
              "ip": "192.168.2.2",
              "udp_port": 50000
            },
            "vlan": {
              "count": 0,
              "tags": [{
                "pri": 0,
                "id": 4095
              },
              {
                "pri": 0,
```



```

        "src": {
            "mac": "00:01:02:03:04:05",
            "ip": "192.168.1.1",
            "udp_port": 5000
        },
        "dst": {
            "mac": "00:21:CE:28:00:2E",
            "ip": "192.168.2.2",
            "udp_port": 50000
        },
        "vlan": {
            "count": 0,
            "tags": [{
                "pri": 0,
                "id": 4095
            },
            {
                "pri": 0,
                "id": 4094
            },
            {
                "pri": 0,
                "id": 4093
            }
        ]
    },
    "qos_type": "diffserv",
    "dscp": 0,
    "ecn": 0,
    "tos": 0,
    "precedence": 0
}
    }
}
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "wlt-tx",
            "method": "setprm",
            "retcode": 0,
            "retmsg": "OK"
        }
    ]
}

```

10.2.20.3. Traffic generation start

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "efdaca9bb2458242c4efeea0eba6ba02",
        "wlt-tx",
        "start",
        {
            "ids": {

```



```

        "profile": "profile0"
    }
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "wlt-tx",
            "method": "start",
            "retcode": 0,
            "retmsg": "OK"
        }
    ]
}

```

10.2.20.4. Traffic generation stop

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",
    "params": [
        "889a2d77b63e4c2ceb20655a36acf033",
        "wlt-tx",
        "stop",
        {
            "ids": {
                "profile": "profile0"
            }
        }
    ]
}

```

Response example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "result": [
        0,
        {
            "object": "wlt-tx",
            "method": "stop",
            "retcode": 0,
            "retmsg": "OK"
        }
    ]
}

```

10.2.20.5. Traffic generation results view

Request example:

```

{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "call",

```

```

    "params": [
      "04ccbd78190e9bf0f7492301cf9d1a08",
      "wlt-tx",
      "getsts",
      {
        "ids": {
          "profile": "profile0"
        }
      }
    ]
  }
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "wlt-tx",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [{
        "ids": {
          "profile": "profile0"
        }
      }],
      "statuses": {
        "status": false,
        "retcode": 0,
        "int": {
          "bucket": 0,
          "packets": "0",
          "bytes": "0",
          "meas_time": {
            "start_s": "0",
            "stop_s": "0",
            "elapsed_s": 0
          },
          "rate_l1": {
            "value": "0.00000",
            "units": "mbps",
            "layer": 1
          },
          "rate_l2": {
            "value": "0.00000",
            "units": "mbps",
            "layer": 2
          },
          "rate_l3": {
            "value": "0.00000",
            "units": "mbps",
            "layer": 3
          },
          "rate_l4": {
            "value": "0.00000",
            "units": "mbps",
            "layer": 4
          }
        },
        "total": {
          "packets": "0",
          "bytes": "0",
          "meas_time": {
            "start_s": "0",
            "stop_s": "0",
            "elapsed_s": 0
          }
        }
      }
    ]
  }
}

```

```

    },
    "rate_l1": {
      "value": "0.00000",
      "units": "mbps",
      "layer": 1
    },
    "rate_l2": {
      "value": "0.00000",
      "units": "mbps",
      "layer": 2
    },
    "rate_l3": {
      "value": "0.00000",
      "units": "mbps",
      "layer": 3
    },
    "rate_l4": {
      "value": "0.00000",
      "units": "mbps",
      "layer": 4
    }
  },
  "sys_time": {
    "start_s": "0",
    "stop_s": "0",
    "elapsed_s": 0
  }
}
}]
}
]
}

```

10.2.21. Commands for the service activation test with interval statistics analyzer function

10.2.21.1. In configuration mode

wlt-rx config

Command to enter the configuration mode of the traffic analyzer function for the selected profile.

Syntax

```
wlt-rx config {profile0 | profile1}
```

Parameters

- profile0 – enter configuration mode for the profile0
- profile1 – enter configuration mode for the profile1

Example

```

smart-sfp(admin)(config)# wlt-rx config profile0
smart-sfp(admin)(config-wlt-rx [profile0])#

```

wlt-rx start

Command to start the traffic analyzer for the selected profile.

Syntax

```
wlt-rx start {profile0 | profile1}
```

Parameters

- profile0 – start the traffic analyzer for the profile0
- profile1 – start the traffic analyzer for the profile1

Example

```
smart-sfp(admin)(config)# wlt-rx start profile0
ok
```

wlt-rx stop

Command to stop the traffic analyzer for the selected profile.

Syntax

```
wlt-rx stop {profile0 | profile1}
```

Parameters

- profile0 – stop the traffic analyzer for the profile0
- profile1 – stop the traffic analyzer for the profile1

Example

```
smart-sfp(admin)(config)# wlt-rx stop profile0
ok
```

show wlt-rx header

Command to display the header of the traffic analyzer.

Syntax

```
show wlt-rx header {profile0 | profile1}
```

Parameters

- profile0 – header settings for the profile0
- profile1 – header settings for the profile1

Example

```
smart-sfp(admin)(config)# show wlt-rx header profile0
Src IP:          192.168.1.1
Src MAC:         00:21:CE:28:00:2D
Src port:        60000
Dst IP:          192.168.2.2
Dst MAC:         00:21:CE:28:00:2E
Dst port:        50000
QOS type:        diffserv
                dscp:      0
                tos:       0
                precedence: 0
```

show wlt-rx results local

Command to display information about interval statistics of the analyzer.

Syntax

```
show wlt-rx results local {profile0 | profile1}
```

Parameters

- profile0 – display statistics for the profile0
- profile1 – display statistics for the profile1

Example

```
smart-sfp(admin)(config)# show wlt-rx results local profile0
Status:                                false
Test duration:
    Start time:                        14:03:38
    Stop time:                         14:04:47
    Elapsed time:                      00:01:08
Bucket (num):                          6
Packets (num):                         7440476
Bytes (num):                           476190464
Jitter:
    Min (us):                         0.00000
    Avg (us):                         0.00000
    Max (us):                         0.00000
FTD:
    Min (us):                         12.34500
    Avg (us):                         16.56781
    Max (us):                         13.75640
TOS (num):                             0
OOP (num):                             0
Err (num):                             0
Loss (num):                            0
Rate (L1):                             499.99999 mbps
Rate (L2):                             380.95237 mbps
Rate (L3):                             273.80952 mbps
Rate (L4):                             154.76190 mbps
Meas duration:
    Start time:                       14:04:34
    Stop time:                        14:04:44
    Elapsed time:                     00:00:10
```

show wlt-rx results total

Command to display information about integral statistics of the analyzer.

Syntax

```
show wlt-rx results total {profile0 | profile1}
```

Parameters

- profile0 – display statistics for the profile0
- profile1 – display statistics for the profile1

Example

```
smart-sfp(admin)(config)# show wlt-rx results total profile0
Status:                                false
Message:                               ok
Test duration:
    Start time:                        14:03:38
    Stop time:                         14:04:47
    Elapsed time:                      00:01:08
Bucket (num):                          6
Packets (num):                         44642857
Bytes (num):                           2857142848
Jitter:
    Min (us):                         0.00000
    Avg (us):                         0.00000
    Max (us):                         0.00000
```

```

FTD:
      Min (us):      12.34500
      Avg (us):      16.56781
      Max (us):      13.75640
TOS (num):          0
OOP (num):          0
Err (num):          0
Loss (num):         0
Rate (L1):          500.00000 mbps
Rate (L2):          380.95238 mbps
Rate (L3):          273.80952 mbps
Rate (L4):          154.76190 mbps
Meas duration:
      Start time:    14:03:44
      Stop time:     14:04:44
      Elapsed time:  00:01:00

```

show wlt-rx setting

Command to display traffic analyzer settings.

Syntax

```
show wlt-rx setting {profile0 | profile1}
```

Parameters

- profile0 – display settings for the profile0
- profile1 – display settings for the profile1

Example

```

smart-sfp(admin)(config)# show wlt-rx setting profile0
      Status:          false
      Port (name):     porta
      Frame size (byte): 64
      Flow ID (num):   4
      Interval(sec):   10
      Duration type:   seconds
      Duration (sec):  01:01:00
      Rate:            100.00000 (12 percents)

```

10.2.21.2. In profile configuration mode

duration

Command to set the duration of traffic analysis.

Syntax

```
duration time <value>
```

Parameters

- <value> – analysis duration value in the “hh:mm:ss” format. Valid values: 00:00:00–99:59:59. “00:00:00” means that the traffic analyzer function will work until the “wlt-rx stop” command is entered.

Default

Default test duration is 1 minute – 00:01:00.

Note

The test duration value set on the analyzer must be equal to the test duration value set on the generator. The test duration value set on the analyzer may not be a multiple of the interval duration. In this case, the integral statistics will display the correct number of received bytes, packets, losses, and other metrics. The last interval in the interval statistics will have a speed slower than the other intervals, but this speed will not affect the speed displayed in the integral statistics.

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# duration time 01:00:00
ok
```

flow

Command to set flow identifier value for the test stream. The value is set by the generator and is necessary for correct test traffic identification on the analyzer.

Syntax

```
flow id <value>
```

Parameters

- <value> – test stream unique identifier

Default

4

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# flow id 1
ok
```

frame

Command to set the frame size.

Syntax

```
frame size <value>
```

Parameters

- <value> – frame size in bytes

Default

64

Note

Minimum frame size (L2) – 64 bytes. Maximum frame size (L2) – 9600 bytes.

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# frame size 512
ok
```

When using the VLAN tag in test traffic, you must specify the right packet size for the generator and analyzer to calculate the test results correctly.

#	Test case	VLAN tag	Packet size X bytes	Test stream header setting
1	Generator	No	X	header vlan count off
	Analyzer	No	X	header vlan count off
2	Generator	Yes	X	header vlan count <number of tags>
	Analyzer	Yes	X	header vlan count <number of tags>
3	Generator	Yes	X	header vlan count <number of tags>
	Analyzer	No	$Y = X - (4 * \text{<number of tags>})$, if $Y < 64$, then specify 64	header vlan count off
4	Generator	No	X	header vlan count off
	Analyzer	Yes	$Y = X + (4 * \text{<number of tags>})$	header vlan count <number of tags>

On the generator and analyzer, you can only set the rate at L3 or L4; the rate value for the generator and analyzer should be the same. In the analyzer and generator statistics, the L1 and L2 rates for the same interval will not match — this is expected behavior since the vlan tag is not taken into account in this case.

header

Command to set the header parameters.

Syntax

```
header {dscp <value>} | {dst ip <value>} | {dst udp <value>} | {precedence <value>}
| {qos type <precedence | diffserv>} | {tos <value>} | {vlan count <off | 1 | 2 |
3>}
```

Parameters

- dscp <value> – DSCP value. Valid values: 0–63.
- dst ip <value> – destination IP address
- dst udp <value> – destination UDP port number. Valid values: 0–65535.
- precedence <value> – precedence value. Valid values: 0–7.
- qos type <precedence | diffserv> – Quality of Service type
- tos <value> – Type of Service value. Valid values: 0–15.
- vlan count <off | 1 | 2 | 3> – number of VLAN tags: 1, 2, or 3 (“off” – VLAN tag insertion is disabled)

The VLAN ID is not specified on the analyzer. The number of VLAN tags is used only to recalculate the speed at different layers (L1–L4). If VLAN tag insertion is disabled on the analyzer “vlan count off” and the generator is configured to generate traffic with a single VLAN tag “vlan count 1”, the analyzer will successfully accept this traffic and calculate all parameters, but the interval and integral speeds for layers L3, L4, and L5 will be calculated incorrectly.

Default

Dst IP: 192.168.2.2

Dst port: 50000

QOS type: diffserv
dscp: 0
tos: 0
precedence: 0

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# header dscp 4
ok
smart-sfp(admin)(config-wlt-rx [profile0])# header dst ip 192.168.2.1
ok
smart-sfp(admin)(config-wlt-rx [profile0])# header dst udp 9090
ok
smart-sfp(admin)(config-wlt-rx [profile0])# header precedence 4
ok
smart-sfp(admin)(config-wlt-rx [profile0])# header qos type diffserv
ok
smart-sfp(admin)(config-wlt-rx [profile0])# header tos 7
ok
smart-sfp(admin)(config-wlt-rx [profile0])# header vlan count 3
ok
```

interval

Command to set the time interval of a certain duration in which statistics (losses, latency, jitter) are calculated on the analyzer.

Syntax

interval <value>

Parameters

- <value> – the duration of the interval is set in the range from 1 s to 60 s, step 1 s

Default

10

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# interval 15
ok
```

port

Command to select port for traffic analysis.

Syntax

port {a}

Parameters

- a – port A

Default

a

Example

```
M716(admin)(config-wlt-rx [profile0])# port a
ok
```

rate

Command to select traffic analysis rate.

Syntax

```
rate <value> { percents | kbps | mbps } { layer1 | layer2 | layer3 | layer4 | layer5 }
```

Parameters

- <value> <percents | kbps | mbps> – rate value in percents, kbps or Mbps. Valid values: 0–100%; for kbps and mbps, valid values will depend on the selected layer and frame size for the service (see section 16.12).
- <layer1 | layer2 | layer3 | layer4 | layer5> – rate layer

Default

100.00000 (12 percents)

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# rate 100 mbps layer3
ok
```

show header

Command to display the header settings.

Syntax

```
show header
```

Parameters

no arguments

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# show header
Src IP:          192.168.1.1
Src MAC:         00:21:CE:28:00:2D
Src port:        60000
Dst IP:          192.168.2.1
Dst MAC:         00:21:CE:28:00:2E
Dst port:        9090
QOS type:        diffserv
                  dscp:      4
                  tos:       7
                  precedence: 4
```

show setting

Command to display the analyzer settings.

Syntax

```
show setting
```

Parameters

no arguments

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# show setting
Status:          false
```

Port (name):	porta
Frame size (byte):	64
Flow ID (num):	4
Interval(sec):	15
Wait(sec):	1.00000
Duration type:	seconds
Duration (sec):	01:00:00
Rate:	100.00000 (13 mbps)

wait

The time at which frames from the previous interval are expected to arrive.

Syntax

wait <value>

Parameters

- <value> – wait time value.

Default

2.0 s

Note

The value is set in the range from 0 to the interval value specified by the “interval” command in decimal format. Valid values: 0–60.

Example

```
smart-sfp(admin)(config-wlt-rx [profile0])# wait 1.0
ok
```

10.2.22. Service activation test with interval statistics analyzer function API

10.2.22.1. Current analyzer settings view

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "4e3c43bd62f920517f77fc868cb67b65",
    "wlt-rx",
    "getprm",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "wlt-rx",
      "method": "getprm",

```

```

"retcode": 0,
"retmsg": "OK",
"answer": [{
  "ids": {
    "profile": "profile0"
  },
  "parameters": {
    "status": false,
    "wlt": {
      "flow_id": 4,
      "interval": 10,
      "wait": "2.00000",
      "framesize": 64
    },
    "duration": {
      "type": "seconds",
      "seconds": 60,
      "packets": "0",
      "bytes": "0"
    },
    "rate": {
      "value": "100.00000",
      "units": "percents",
      "layer": 2
    },
    "trial": {
      "ifaces": {
        "rx": {
          "name": "porta",
          "mode": "local",
          "host": "",
          "disabled": false
        }
      },
      "wait_time_ms": 1000,
      "learn_time_ms": 0
    },
    "header": {
      "src": {
        "mac": "00:21:CE:28:00:2D",
        "ip": "192.168.1.1",
        "udp_port": 60000
      },
      "dst": {
        "mac": "00:21:CE:28:00:2E",
        "ip": "192.168.2.2",
        "udp_port": 50000
      },
      "vlan": {
        "count": 0,
        "tags": [{
          "pri": 0,
          "id": 4095
        },
        {
          "pri": 0,
          "id": 4094
        },
        {
          "pri": 0,
          "id": 4093
        }
      ]
    },
    "qos_type": "diffserv",
    "dscp": 0,
    "ecn": 0,

```

```

        "tos": 0,
        "precedence": 0
    }
    }
    }
    }
    ]
}

```

10.2.22.2. Analyzer settings change

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "b2d38cb6a4c3ff7a98bd431681371683",
    "wlt-rx",
    "setprm",
    {
      "ids": {
        "profile": "profile0"
      },
      "parameters": {
        "status": false,
        "wlt": {
          "flow_id": 4,
          "interval": 10,
          "wait": "2.00000",
          "framesize": 64
        },
        "duration": {
          "type": "seconds",
          "seconds": 60,
          "packets": "0",
          "bytes": "0"
        },
        "rate": {
          "value": "100.00000",
          "units": "percents",
          "layer": 2
        },
        "trial": {
          "ifaces": {
            "rx": {
              "name": "porta",
              "mode": "local",
              "host": "",
              "disabled": false
            }
          },
          "wait_time_ms": 1000,
          "learn_time_ms": 0
        },
        "header": {
          "src": {
            "mac": "00:21:CE:28:00:2D",
            "ip": "192.168.1.1",
            "udp_port": 60000
          },
          "dst": {
            "mac": "00:21:CE:28:00:2E",
            "ip": "192.168.2.2",
            "udp_port": 50000
          }
        }
      }
    ]
  ]
}

```

```

        "vlan": {
            "count": 0,
            "tags": [{
                "pri": 0,
                "id": 4095
            },
            {
                "pri": 0,
                "id": 4094
            },
            {
                "pri": 0,
                "id": 4093
            }
        ]
    },
    "qos_type": "diffserv",
    "dscp": 0,
    "ecn": 0,
    "tos": 0,
    "precedence": 0
}
    }
}
    ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "wlt-rx",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}

```

10.2.22.3. Traffic analyzer start

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "efdaca9bb2458242c4efeea0eba6ba02",
    "wlt-rx",
    "start",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "wlt-rx",
      "method": "start",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.22.4. Traffic analyzer stop

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "889a2d77b63e4c2ceb20655a36acf033",
    "wlt-rx",
    "stop",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "wlt-rx",
      "method": "stop",
      "retcode": 0,

```

```

    "retmsg": "OK"
  }
]
}

```

10.2.22.5. Traffic analyzer results view

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "04ccbd78190e9bf0f7492301cf9d1a08",
    "wlt-rx",
    "getsts",
    {
      "ids": {
        "profile": "profile0"
      }
    }
  ]
}

```

Response example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "wlt-rx",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "ids": {
            "profile": "profile0"
          },
          "statuses": {
            "status": false,
            "retcode": 0,
            "retmsg": "OK",
            "int": {
              "bucket": 0,
              "packets": "0",
              "bytes": "0",
              "ftd": {
                "min": "inf",
                "avg": "0.00000",
                "max": "0.00000"
              },
              "jitter": {
                "min": "inf",
                "avg": "0.00000",
                "max": "0.00000"
              },
              "tos": "0",
              "oop": "0",
              "err": "0",
              "loss": "0",
              "meas_time": {
                "start_s": "0",
                "stop_s": "0",

```



```

        "elapsed_s": 0
    },
    "rate_l1": {
        "value": "0.00000",
        "units": "mbps",
        "layer": 1
    },
    "rate_l2": {
        "value": "0.00000",
        "units": "mbps",
        "layer": 2
    },
    "rate_l3": {
        "value": "0.00000",
        "units": "mbps",
        "layer": 3
    },
    "rate_l4": {
        "value": "0.00000",
        "units": "mbps",
        "layer": 4
    }
},
"total": {
    "bucket": 0,
    "packets": "0",
    "bytes": "0",
    "ftd": {
        "min": "inf",
        "avg": "0.00000",
        "max": "0.00000"
    },
    "jitter": {
        "min": "inf",
        "avg": "0.00000",
        "max": "0.00000"
    },
    "tos": "0",
    "oop": "0",
    "err": "0",
    "loss": "0",
    "meas_time": {
        "start_s": "0",
        "stop_s": "0",
        "elapsed_s": 0
    },
    "rate_l1": {
        "value": "0.00000",
        "units": "mbps",
        "layer": 1
    },
    "rate_l2": {
        "value": "0.00000",
        "units": "mbps",
        "layer": 2
    },
    "rate_l3": {
        "value": "0.00000",
        "units": "mbps",
        "layer": 3
    },
    "rate_l4": {
        "value": "0.00000",
        "units": "mbps",
        "layer": 4
    }
}
},

```

```

    "sys_time": {
        "start_s": "0",
        "stop_s": "0",
        "elapsed_s": 0
    },
}
}
]
}
]
}

```

10.2.23. Commands for the service of managing additional network stack settings

10.2.23.1. In configuration mode

network config

Command to enter the configuration mode of the service of managing additional network stack settings for the selected port.

Syntax

```
network config {porta | portb}
```

Parameters

- porta – enter configuration mode for the port A
- portb – enter configuration mode for the port B.

Example

```
smart-sfp(admin)(config)# network config porta
smart-sfp(admin)(config-network [porta])#
```

10.2.23.2. In port configuration mode

cpu-multicast

Command to enable/disable the “cpu-multicast” function. When this feature is enabled, multicast packets will be sent to the CPU.

Syntax

```
cpu multicast status {on | off}
```

Parameters

- status on – enable “cpu-multicast” function
- status off – disable “cpu-multicast” function.

By default

off

Example

```
smart-sfp(admin)(config-network [porta])# cpu-multicast status on
ok
```

show

Command to show information about the settings of the service of managing additional network stack settings for the selected port.

Syntax

show

Example

```
smart-sfp(admin)(config-network [porta])# show
Status:      true
Message:     OK
CPU-multicast: false
```

10.2.24. API for the service of managing additional network stack settings

10.2.24.1. Service status

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "01ab888cd33959eddd2bfac1de652b82",
    "network.advanced",
    "getsts",
    "ids": {
      "interface": "porta"
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "network.advanced",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "ids": {
            "interface": "porta"
          },
          "statuses": {
            "status": true,
            "retcode": 0,
            "advanced": {
              "cpu_multicast": false
            }
          }
        }
      ]
    }
  ]
}
```

10.2.24.2. Service management

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "1f9514cc2458f185a6fa56808ce9bb5e",
    "network.advanced",
    "setprm",
    {
      "ids": {
        "interface": "porta"
      },
      "parameters": {
        "status": true,
        "advanced": {
          "cpu_multicast": true
        }
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "network.advanced",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.25. Commands for the timestamp insertion “tsins” function

10.2.25.1. In configuration mode

tsins config

Command to enter the configuration mode of the timestamp insertion function for the selected profile.

Syntax

```
tsins config {profile0 | profile1}
```

Parameters

- profile0 – enter configuration mode for the profile0
- profile1 – enter configuration mode for the profile1

Example

```
smart-sfp(admin)(config)# tsins config profile0
smart-sfp(admin)(config-tsins[profile0])#
```

tsins start

Command to enable the timestamp insertion function for the selected profile.

Syntax

```
tsins start {profile0 | profile1}
```

Parameters

- profile0 – enable the function of timestamp insertion for the profile0
- profile1 – enable the function of timestamp insertion for the profile1

Example

```
smart-sfp(root)(config)# tsins start profile0
ok
```

tsins stop

Command to disable the timestamp insertion function for the selected profile.

Syntax

```
tsins stop {profile0 | profile1}
```

Parameters

- profile0 – disable the function of timestamp insertion for the profile0
- profile1 – disable the function of timestamp insertion for the profile1

Example

```
smart-sfp(root)(config)# tsins stop profile1
ok
```

show tsins results

Command to display the results of the timestamp insertion function operating.

Syntax

```
show tsins results {profile0 | profile1}
```

Parameters

- profile0 – results for the profile0
- profile1 – results for the profile1

Example

```
smart-sfp(admin)(config)# show tsins results profile0
Status:          true
Message:         ok
Duration:
Start time:      12:21:02
Stop time:       {}:{}:{}
Elapsed time:    00:49:10
```

show tsins setting

Command to display settings for the timestamp insertion function.

Syntax

```
show tsins setting {profile0 | profile1}
```

Parameters

- profile0 – results for the profile0
- profile1 – results for the profile1

Example

```
smart-sfp(admin)(config)# show tsins setting profile0
Status:          true
Duration (sec):  endless

Number  Port    Pattern          Level  Timestamp point  Status
0       porta  00112233AABBCCDD  udp    ingress          true
1       portb   00112233AABBCCDD  udp    egress           true
```

10.2.25.2. In profile configuration mode

duration

Command to set the duration of the timestamps insertion function operating.

Syntax

```
duration {endless | useconds <value>}
```

Parameters

- endless – timestamps insertion will be performed until the command “tsins stop profile 0/1” is entered
- useconds <value> – user-defined value in the format hh:mm:ss, where hh – hours, mm – minutes, ss – seconds.

Valid values: 00:00:00-99:59:59. “00:00:00” means that the timestamps insertion will be performed until the “tsins stop profile 0/1” command is entered.

By default

endless

Example

```
smart-sfp(admin)(config-tsins[profile0])# duration endless
ok
smart-sfp(admin)(config-tsins[profile0])# duration useconds 24:00:00
ok
```

level

Command for selecting the operating level of the timestamp insertion function.

Syntax

```
level {iface 0|1} {udp|ethernet}
```

Parameters

- iface 0 – direction of transit packets from the line interface (port A) to the host interface (port B)
- iface 1 – direction of transit packets from the host interface (port B) to the line interface (port A)
- udp – if this option is selected, the timestamp insertion function will search for the specified sequence only in the “data” field of IPv4/UDP packets

- ethernet – if this option is selected, the timestamp insertion function will search for the specified sequence only in the “payload” field of Ethernet frames

By default

udp

Example

```
smart-sfp(admin)(config-tsins[profile0])# level iface 0 ethernet
ok
```

pattern

Command to configure the sequence that the timestamp insertion function will search for.

Syntax

```
pattern {iface 0|1} {<value>}
```

Parameters

- iface 0 – direction of transit packets from the line interface (port A) to the host interface (port B)
- iface 1 – direction of transit packets from the host interface (port B) to the line interface (port A)
- <value> – a sequence of 16 hexadecimal digits in the format 00112233AABBCCDD, which the timestamp insertion function will search for in transit packets. The timestamp is inserted according to the NTP Timestamp Format (64 bits).

By default

00112233AABBCCDD

Example

```
smart-sfp(admin)(config)# pattern iface 0 1A2B3C4D5F6A7B8C
ok
```

show

Command to display information about the settings of the timestamp insertion function.

Syntax

```
show settings
```

Example

```
smart-sfp(admin)(config-tsins[profile0])# show settings
Status:          true
Duration (sec):  endless

Number  Port   Pattern                Level  Timestamp point  Status
0       porta AAAAAAAAAAAAAAAAAA    udp    ingress          true
1       portb 00112233AABBCCDD    udp    egress           true
```

status

Command to enable/disable direction for the timestamp insertion function.

Syntax

```
status {iface 0|1} {on|off}
```

Parameters

- iface 0 – direction of transit packets from the line interface (port A) to the host interface (port B)
- iface 1 – direction of transit packets from the host interface (port B) to the line interface (port A)
- on – enable direction for use in the profile 0/1
- off – disable direction for use in the profile 0/1.

By default

on

Example

```
smart-sfp(admin)(config-tsins[profile0])# status iface 0 on
ok
```

tspoint

Command to select the interface on which the specified sequence will be replaced with a timestamp.

Syntax

```
tspoint {iface 0|1} {ingress|egress}
```

Parameters

- iface 0 – direction of transit packets from the line interface (port A) to the host interface (port B)
- iface 1 – direction of transit packets from the host interface (port B) to the line interface (port A)
- ingress – direction for the timestamp insertion: the label is inserted at the Smart SFP input
- egress – direction for the timestamp insertion: the label is inserted at the Smart SFP output

By default

ingress

Example

```
smart-sfp(admin)(config-tsins[profile0])# tspoint iface 0 ingress
ok
```

10.2.26. API for the timestamp insertion function “tsins”

10.2.26.1. Changing timestamp insertion settings

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1, "method": "call",
  "params":
  [
    "4767a307c3f8959f3697e74966cea287",
    "tsins",
    "setprm",
    {
      "ids": {
        "profile": "profile1"
      },
      "parameters": {
        "duration": {
          "type": "seconds",
          "seconds": 0,

```



```

        "packets": "0",
        "bytes": "0"
    },
    "trial": {
        "ifaces": {
            "0": {
                'disabled': True,
                'host': '',
                'mode': 'local',
                'name': 'porta'},
            "1": {
                'disabled': False,
                'host': '',
                'mode': 'local',
                'name': 'portb'}}},
        'learn_time_ms': 1000,
        'wait_time_ms': 1000
    },
    "tsins": {
        "0": {
            "pattern": "FF00FF00AA00AA00",
            "level": "udp",
            "type": "egress"
        },
        "1": {
            "pattern": "BBBB0000CCCC0000",
            "level": "ethernet",
            "type": "ingress"
        }
    },
    "status": "false"
}
]
}

```

Response example:

```

{
  'id': 1,
  'jsonrpc': '2.0',
  'result': [0,
    {
      'method': 'setprm',
      'object': 'tsins',
      'retcode': 0,
      'retmsg': 'OK'
    }
  ]
}

```

10.2.26.2. View timestamp insertion settings

Request example:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "4767a307c3f8959f3697e74966cea287",
    "tsins",
    "getprm",
    {
      "ids": {
        "profile": "profile1"
      }
    }
  ]
}

```

Response example:

```
{'id': 1,
'jsonrpc': '2.0',
'result': [0,
  {'answer':
    [{'ids':
      {'profile': 'profile1'},
      'method': 'setprm',
      'object': 'tsins',
      'parameters':
        {'duration':
          {'bytes': '0',
           'packets': '0',
           'seconds': 0,
           'type': 'seconds'
          },
        'status': False,
        'trial':
          {'ifaces':
            {'0':
              {'disabled': False,
               'host': '',
               'mode': 'local',
               'name': 'porta'},
            '1':
              {'disabled': False,
               'host': '',
               'mode': 'local',
               'name': 'portb'
              }
            },
          'learn_time_ms': 1000,
          'wait_time_ms': 1000},
      'tsins':
        {'0':
          {'level': 'udp',
           'pattern': 'FF00FF00AA00AA00',
           'type': 'ingress'},
        '1':
          {'level': 'udp',
           'pattern': '00112233AABBCCDD',
           'type': 'ingress'}
        }
      }
    ]},
  {'method': 'getprm',
   'object': 'tsins',
   'retcode': 0,
   'retmsg': 'OK'
  }
]}]
```

10.2.26.3. View timestamp insertion statistics

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "4767a307c3f8959f3697e74966cea287",
    "tsins",
    "getsts",
    {
      "ids": {
```

```

        "profile": "profile0"
      }
    ]
  }

```

Response example:

```

{
  'id': 1,
  'jsonrpc': '2.0',
  'result': [0,
    {
      'answer': [
        {
          'ids': {
            'profile': 'profile0'
          },
          'statuses': {
            'retcode': 0,
            'retmsg': 'OK',
            'status': False,
            'sys_time': {
              'elapsed_s': 1720362,
              'start_s': '1683894062',
              'stop_s': '1685614425'
            }
          },
          'tsins': {
            '0': {
              'label_cnt': 0
            },
            '1': {
              'label_cnt': 0
            }
          }
        }
      ],
      'method': 'getsts',
      'object': 'tsins',
      'retcode': 0,
      'retmsg': 'OK'
    }
  ]
}

```

10.2.27. Smart SFP signals monitoring and managing commands

10.2.27.1. In configuration mode

ssfp config

Command to enter Smart SFP signal management mode.

Syntax

```
ssfp config
```

Example

```

smart-sfp(admin)(config)# ssfp config
* Enter `show` to view the configuration

```

show ssfp results

Command to display the results of the optical module part state.

Syntax

```
show ssfp results
```

Example

```

smart-sfp(admin)(config)# show ssfp results
Status:      true

```

```
Message:      ok
Rate select:  true
TX detect:    true
TX disable:   false
TX fault:     false
RX loss:      true
```

show ssfp setting

Command to display the current settings of the Smart SFP signals state.

Syntax

```
show ssfp setting
```

Example

```
smart-sfp(admin)(config)# show ssfp setting
Status:      true
TX fault:    false
RX loss:     false
```

10.2.27.2. In signal control mode

rx_los_mask

Command to control the RX_LOS signal.

Syntax

```
rx_los_mask {on | off}
```

Parameters

- on – enabling alarm masking for the RX_LOS signal
- off – disabling alarm masking for the RX_LOS signal

Default

on

Example

```
smart-sfp(admin)(config-ssfp)# rx_los_mask off
ok
```

show

Command to display the module signal control settings.

Syntax

```
show setting
```

Parameters

no arguments

Example

```
smart-sfp(admin)(config-ssfp)# show setting
Status:      true
TX fault:    false
RX loss:     true
```

tx_flt_mask

Command to control the TX_FAULT signal.

Syntax

```
tx_flt_mask {on | off}
```

Parameters

- on – enabling alarm masking for the TX_FAULT signal
- off – disabling alarm masking for the TX_FAULT signal

Default

on

Example

```
smart-sfp(admin)(config-ssfp)# tx_flt_mask off
ok
```

10.2.28. Smart SFP signals monitoring and managing API

10.2.28.1. State monitoring

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "01ab888cd33959eddd2bfac1de652b82",
    "device.optics",
    "getsts",
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "device.optics",
      "method": "getsts",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "statuses": {
            "status": true,
            "retcode": 0,
            "optics": {
              "tx_flt": false,
              "rx_los": true,
              "tx_dis": false,
              "tx_sd": true,
              "rate_sel": false
            }
          }
        }
      ]
    }
  ]
}
```

10.2.28.2. Signals control

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "1f9514cc2458f185a6fa56808ce9bb5e",
    "device.optics",
    "setprm",
    {
      "parameters": {
        "status": true,
        "optics": {
          "rx_los": true,
          "txflt": true
        }
      }
    }
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "device.optics",
      "method": "setprm",
      "retcode": 0,
      "retmsg": "OK"
    }
  ]
}
```

10.2.29. Show commands

show activity

Command to show information about activity on ports.

Syntax

show activity

Parameters

no arguments

Example

```
smart-sfp(admin)# show activity
Port a:
    nic_multicast
Port b:
    nic_multicast
```

show hwinfo

Command to show hardware monitor information.

Syntax

show hwinfo

Parameters

no arguments

Example

```
smart-sfp(admin)# show hwinfo
Vendor:    smart-sfp
Name:      smart-sfp
Type:      ssfp-2.1
HW:        2.2
P/N:       smart-sfp
S/N:       0013
```

show hwmon

Command to show information about CPU and FPGA temperature and optical driver hardware parameters.

Syntax

show hwmon

Parameters

no arguments

Example

```
smart-sfp(root)# show hwmon
CPU
    Temperature (C):    75
FPGA
    Temperature (C):    76
Optical driver
    Temperature (C):    86.850
    Voltage (V):        3.078
    Power TX (mkA):     217.090
    Voltage TX (V):     3.078
    Bias TX (mA):       13.750
    Modulation TX (mA): 26.200
    Power RX (mkA):     43.200
    Voltage RX (V):     3.078
smart-sfp(root)#
```

show version

Command to display Smart SFP software version.

Syntax

show version

Parameters

no arguments

Example

```
smart-sfp(admin)# show version
SW:      SA-SW-01
App:     1.0.104
Sys:     5.4.0
FPGA:    0.2.0-0
Kernel:  5.10.59-probe-1.9.0
```

10.2.1. Show commands API

10.2.1.1. View device information

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "725cbf6b421acff9b77aa99cff2c32eb",
    "devinfo",
    "show",
    {}
  ]
}
```

Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "devinfo",
      "method": "show",
      "retcode": 0,
      "retmsg": "OK",
      "result": {
        "device_name": "smart-sfp",
        "device_vendor": "smart-sfp",
        "part_num": "smart-sfp",
        "device_type": "smart-sfp",
        "kernel_version": "5.10.59-probe-1.9.0",
        "fpga_version": "0.2.0-0",
        "cpu_temp": 56,
        "hw_version": "2.2",
        "serial_num": "00180",
        "sys_version": "5.4.0",
        "app_version": "1.0.104",
        "sw_version": "SA-SW-02",
        "fpga_temp": 65,
        "opt_temp": "65",
        "opt_pwr_tx": "228515",
        "opt_bias_tx": "18",
        "opt_mod_tx": "30",
        "opt_pwr_rx": "31",
        "opt_volt_tx": "3085",
        "opt_volt_rx": "3085",
        "opt_volt": "3085"
      }
    }
  ]
}
```

10.2.1.2. View port usage information

Request example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "call",
  "params": [
    "18a6141589b4cf4bc6acc4494fd49a06",
    "lock",
    "getprm",
    {}
  ]
}
```


Response example:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": [
    0,
    {
      "object": "lock",
      "method": "getprm",
      "retcode": 0,
      "retmsg": "OK",
      "answer": [
        {
          "object": "lock",
          "method": "setprm",
          "ids": {
            "lock": 0
          },
          "parameters": {
            "status": true,
            "interface": "porta",
            "resource": "nic_multicast",
            "owner": "networkd",
            "text": ""
          }
        },
        {
          "object": "lock",
          "method": "setprm",
          "ids": {
            "lock": 1
          },
          "parameters": {
            "status": true,
            "interface": "portb",
            "resource": "nic_multicast",
            "owner": "networkd",
            "text": ""
          }
        }
      ]
    }
  ]
}
```

11. Data transfer configuration for Smart SFP 1G RS-232

Available settings can vary depending on the installed feature set (see chapter 2.4).

11.1. RS-232 data transfer over Ethernet/IP networks

When configuring RS-232 transmission over Ethernet/IP networks, the RS-232 port is managed using standard Linux utilities for serial ports. Documentation for these utilities is publicly available. For example, you can use the Minicom terminal utility or configure serial port data transmission with utilities like ser2net or socat.

On the Smart SFP 1G RS-232 with a single RS-232 port, it is available using this address:

- /dev/ttyPORT0

On the Smart SFP 1G RS-232 with three RS-232 ports, they are available using these addresses:

- /dev/ttyPORT0
- /dev/ttyPORT1
- /dev/ttyPORT2

11.1.1. Example of RS-232 data transmission configuration

This example shows how to set up continuous RS-232 data transmission between a pair of Smart SFP 1G RS-232 modules.

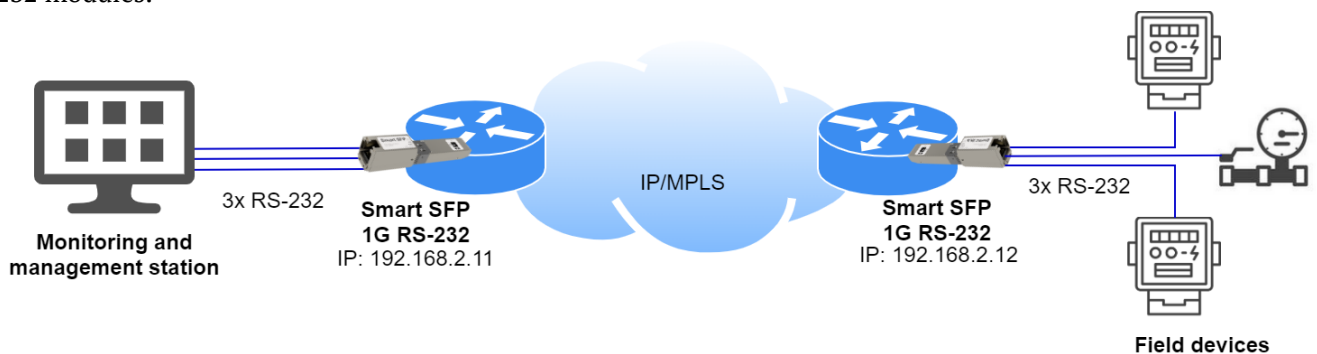


Figure 11.1. Example of an RS-232 data transmission diagram between a pair of Smart SFP 1G RS-232 modules

Before the RS-232 data transmission configuration, it is necessary to configure the IP addresses in accordance with Section 7.2. After that, follow these steps:

1. Connect to the device via SSH.
2. Log in as admin and get root privileges:

```
admin@smart-sfp:~$ su
```

3. Remount the file system for writing:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

4. On one module socat should be configured in the incoming connection standby mode. To do this, it is necessary to create socat utility settings files for each of the RS-232 interfaces used. For each RS-232 interface after "TCP-LISTEN:" it is required to specify its own TCP port number. The data transmission rate of the RS-232 interface is specified in the format b1200, b9600, etc. Other socat parameters are described in the publicly available documentation for this utility.

- a. For /dev/ttyPORT0, create the file /etc/config/socat-ttyPORT0.conf:

```
root@smart-sfp:/home/admin# nano /etc/config/socat-ttyPORT0.conf
```

With the following contents:

```
SOCAT_PARAMS=-d -d -s
SOCAT_CONNECTION=TCP-LISTEN:60000
SOCAT_SERIAL=/dev/ttyPORT0,raw,echo=0,user=root,group=dialout,b115200
```

Save changes by pressing Ctrl+X, then Y, then Enter.

b. For /dev/ttyPORT1, create the file /etc/config/socat-ttyPORT1.conf:

```
root@smart-sfp:/home/admin# nano /etc/config/socat-ttyPORT1.conf
```

With the following contents:

```
SOCAT_PARAMS=-d -d -s
SOCAT_CONNECTION=TCP-LISTEN:60001
SOCAT_SERIAL=/dev/ttyPORT1,raw,echo=0,user=root,group=dialout,b9600
```

Save changes by pressing Ctrl+X, then Y, then Enter.

c. For /dev/ttyPORT2, create the file /etc/config/socat-ttyPORT2.conf:

```
root@smart-sfp:/home/admin# nano /etc/config/socat-ttyPORT2.conf
```

With the following contents:

```
SOCAT_PARAMS=-d -d -s
SOCAT_CONNECTION=TCP-LISTEN:60002
SOCAT_SERIAL=/dev/ttyPORT2,raw,echo=0,user=root,group=dialout,b1200
```

Save changes by pressing Ctrl+X, then Y, then Enter.

5. On another module, socat should be configured in the outgoing connection mode. To do this, it is necessary to create socat utility settings files for each of the RS-232 interfaces used. For each RS-232 interface after "TCP:", it is necessary to specify the IP address and TCP port to connect in the format 192.168.2.12:65000. The data transmission rate of the RS-232 interface is specified in the format b1200, b9600, etc.

a. For /dev/ttyPORT0, create the file /etc/config/socat-ttyPORT0.conf

```
root@smart-sfp:/home/admin# nano /etc/config/socat-ttyPORT0.conf
```

With the following contents:

```
SOCAT_PARAMS=-d -d -s
SOCAT_CONNECTION=TCP:192.168.2.12:60000
SOCAT_SERIAL=/dev/ttyPORT0,raw,echo=0,user=root,group=dialout,b115200
```

Save changes by pressing Ctrl+X, then Y, then Enter.

b. For /dev/ttyPORT1, create the file /etc/config/socat-ttyPORT1.conf:

```
root@smart-sfp:/home/admin# nano /etc/config/socat-ttyPORT1.conf
```

With the following contents:

```
SOCAT_PARAMS=-d -d -s
SOCAT_CONNECTION=TCP:192.168.2.12:60001
SOCAT_SERIAL=/dev/ttyPORT1,raw,echo=0,user=root,group=dialout,b9600
```

Save changes by pressing Ctrl+X, then Y, then Enter.

c. For /dev/ttyPORT2, create the file /etc/config/socat-ttyPORT2.conf:

```
root@smart-sfp:/home/admin# nano /etc/config/socat-ttyPORT2.conf
```

With the following contents:

```
SOCAT_PARAMS=-d -s
SOCAT_CONNECTION=TCP:192.168.2.12:60002
SOCAT_SERIAL=/dev/ttyPORT2,raw,echo=0,user=root,group=dialout,b1200
```

Save changes by pressing Ctrl+X, then Y, then Enter.

6. On both modules, create a service file for data transmission using socat /usr/lib/systemd/system/socat@.service:

```
root@smart-sfp:/home/admin# nano /usr/lib/systemd/system/socat@.service
```

Add lines:

```
[Unit]
Description=RS232 data transfer using %i
After=network.target
StartLimitIntervalSec=10
StartLimitBurst=15

[Service]
Restart=always
RestartSec=5
EnvironmentFile=/etc/config/socat-%i.conf
Type=simple
PIDFile=/var/run/socat-%i.pid
ExecStart=/usr/bin/socat -L/var/run/socat-%i.pid $SOCAT_PARAMS $SOCAT_CONNECTION
$SOCAT_SERIAL
ExecStop=/bin/kill -SIGKILL $MAINPID

[Install]
WantedBy=multi-user.target
```

Save changes by pressing Ctrl+X, then Y, then Enter.

7. For both modules enable autorun of the data transmission service using socat:

```
root@smart-sfp:/home/admin# systemctl enable socat@ttyPORT0.service
root@smart-sfp:/home/admin# systemctl enable socat@ttyPORT1.service
root@smart-sfp:/home/admin# systemctl enable socat@ttyPORT2.service
```

8. To apply the settings, reboot both modules using the command:

```
reboot
```

11.1.2. Example of a terminal server using

This example shows how to use the Smart SFP 1G RS-232 module as a terminal server.

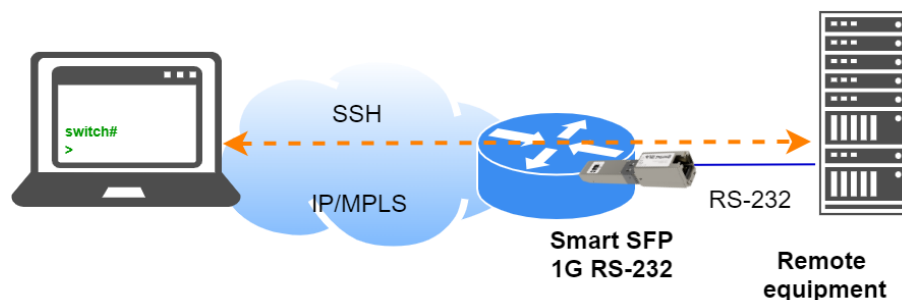


Figure 11.2. An example of using the Smart SFP 1G RS-232 module as a terminal server

Before the RS-232 data transmission configuration, it is necessary to configure the IP addresses in accordance with Section 7.2. After that, follow these steps:

1. Connect to the device via SSH.
2. Log in as admin and get root privileges:

```
admin@smart-sfp:~$ su
```

3. Run minicom by the command:

```
root@smart-sfp:/home/admin# minicom -D /dev/ttyPORT0 -s
```

4. In an open menu select Serial port setup and press Enter.
5. In the serial port settings menu:
 - a. In the Bps/Par/Bits section, configure the data transfer rate, number of data bits, number of stop bits, and parity control.
 - b. Enable or disable Hardware Flow Control.
 - c. Enable or disable Software Flow Control.
6. After this, press Esc twice to open the terminal.

At the end of using the terminal in order to exit minicom:

1. Press Ctrl+A, then Z to open the minicom menu.
2. In the minicom menu press X to exit.
3. Confirm exit by pressing Enter.

11.2. IEC 60870-5-101 data transfer over Ethernet/IP networks

The configuration of IEC 60870-5-101 (hereafter IEC-101) data transmission over Ethernet/IP networks is done by modifying the configuration file `/var/lib/probe/etc/iecmond/iecmond_config.json` and using the `iecmond` utility.

11.2.1. IEC-101 “point-to-two-points” data transfer configuration example

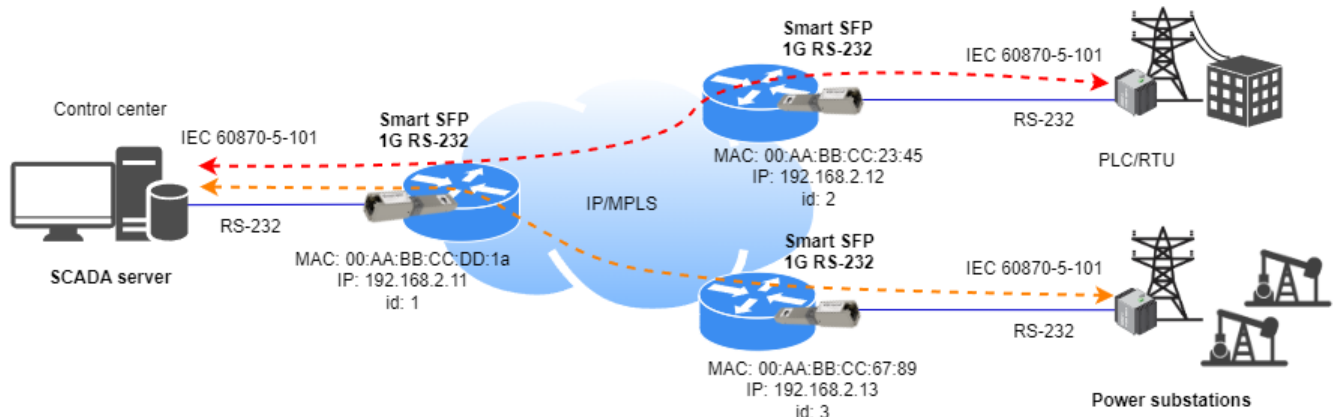


Figure 11.1. IEC-101 “point-to-two-points” data transfer diagram

In this example IEC-101 Master-station is connected to Smart SFP with IP address 192.168.2.11 configured on it. Two Slave-stations are connected to Smart SFPs with IP addresses 192.168.2.12 and 192.168.2.13.

Content of `/var/lib/probe/etc/iecmond/iecmond_config.json` on Smart SFP with IP address 192.168.2.11:

```
{
  "Settings" : {
    "id" : "1",
    "cport" : "61000"
  },
  "Encaps" : {
    "2" : {
      "macd" : "auto",
      "macs" : "auto",
      "ips" : "auto",
      "ipd" : "192.168.2.12",
      "ups" : "60000",
      "upd" : "60001"
    },
    "3" : {
      "macd" : "auto",
      "macs" : "auto",
      "ips" : "auto",
      "ipd" : "192.168.2.13",
      "ups" : "60000",
      "upd" : "60001"
    }
  },
  "EtherFlt" : {
    "ipd" : "auto"
  },
  "RS232" : {
    "baudrate" : "1200",
    "flow_ctl" : "off",
    "link_ctl" : "off",
    "parity" : "off",
    "parity_mode" : "even",
    "stop" : "1",
```

```

    "rx_inv"           : "off",
    "tx_inv"           : "off",
    "dtr_inv"          : "off",
    "dsr_inv"          : "off",
    "dcd_inv"          : "off",
    "rts_inv"          : "off",
    "cts_inv"          : "off",
    "prx_inv"          : "off",
    "ptx_inv"          : "off",
    "rx_los_override"  : "off",
    "rx_los_wd"        : "2",
    "ignore_starterr"  : "off"
},

    "IEC" : {
        "length"           : "2",
        "idle_bits"        : "1",
        "wait_idle_bits"   : "33",
        "freerun_mode_en"  : "off",
        "drop_inval_pkts_en" : "off",
        "drop_inval_start_sym_en" : "off",
        "drop_unspec_sym_en" : "off",
        "ignore_data_start_err_en" : "off",
        "ignore_data_par_err_en" : "off",
        "ignore_data_stop_err_en" : "off",
        "ignore_inval_data_en" : "off",
        "ignore_inval_len_dup_en" : "off",
        "ignore_inval_2_start_en" : "off",
        "ignore_checksum_en" : "off",
        "ignore_end_sym_en" : "off"
    }
}

```

```

{
  "Settings" : {
    "id" : "2",
    "cport" : "61000"
  },
  "Encaps" : {
    "1" : {
      "macd" : "auto",
      "macs" : "auto",
      "ips" : "auto",
      "ipd" : "192.168.2.11",
      "ups" : "60000",
      "upd" : "60001"
    }
  },
  "EtherFlt" : {
    "ipd" : "auto"
  },
  "RS232" : {
    "baudrate" : "1200",
    "flow_ctl" : "off",
    "link_ctl" : "off",
    "parity" : "off",
    "parity_mode" : "even",
    "stop" : "1",
    "rx_inv" : "off",
    "tx_inv" : "off",
    "dtr_inv" : "off",
    "dsr_inv" : "off",
    "dcd_inv" : "off",
    "rts_inv" : "off",
    "cts_inv" : "off",
    "prx_inv" : "off",
    "ptx_inv" : "off",
    "rx_los_override" : "off",
    "rx_los_wd" : "2",
    "ignore_starterr" : "off"
  },
  "IEC" : {
    "length" : "2",
    "idle_bits" : "1",
    "wait_idle_bits" : "33",
    "freerun_mode_en" : "off",
    "drop_inval_pkts_en" : "off",
    "drop_inval_start_sym_en" : "off",
    "drop_unspec_sym_en" : "off",
    "ignore_data_start_err_en" : "off",
    "ignore_data_par_err_en" : "off",
    "ignore_data_stop_err_en" : "off",
    "ignore_inval_data_en" : "off",
    "ignore_inval_len_dup_en" : "off",
    "ignore_inval_2_start_en" : "off",
    "ignore_checksum_en" : "off",
    "ignore_end_sym_en" : "off"
  }
}

```



```

{
  "Settings" : {
    "id" : "3",
    "cport" : "61000"
  },
  "Encaps" : {
    "1" : {
      "macd" : "auto",
      "macs" : "auto",
      "ips" : "auto",
      "ipd" : "192.168.2.11",
      "ups" : "60000",
      "upd" : "60001"
    }
  },
  "EtherFlt" : {
    "ipd" : "auto"
  },
  "RS232" : {
    "baudrate" : "1200",
    "flow_ctl" : "off",
    "link_ctl" : "off",
    "parity" : "off",
    "parity_mode" : "even",
    "stop" : "1",
    "rx_inv" : "off",
    "tx_inv" : "off",
    "dtr_inv" : "off",
    "dsr_inv" : "off",
    "dcd_inv" : "off",
    "rts_inv" : "off",
    "cts_inv" : "off",
    "prx_inv" : "off",
    "ptx_inv" : "off",
    "rx_los_override" : "off",
    "rx_los_wd" : "2",
    "ignore_starterr" : "off"
  },
  "IEC" : {
    "length" : "2",
    "idle_bits" : "1",
    "wait_idle_bits" : "33",
    "freerun_mode_en" : "off",
    "drop_inval_pkts_en" : "off",
    "drop_inval_start_sym_en" : "off",
    "drop_unspec_sym_en" : "off",
    "ignore_data_start_err_en" : "off",
    "ignore_data_par_err_en" : "off",
    "ignore_data_stop_err_en" : "off",
    "ignore_inval_data_en" : "off",
    "ignore_inval_len_dup_en" : "off",
    "ignore_inval_2_start_en" : "off",
    "ignore_checksum_en" : "off",
    "ignore_end_sym_en" : "off"
  }
}

```

11.2.2. IEC-101 data transfer settings

11.2.2.1. Settings section

id

Unique device identifier used for encapsulator/decapsulator operations

Allowed values

Specify the device identifier from the range 0-65535 using format:

```
"id" : "1",
```

cport

TCP port setting for statistics exchange. All devices exchanging data with each other must use the same port.

It is recommended to use ports from the private range 49152-65535.

Allowed values

Specify the TCP port number from the range 1-65535 using format:

```
"cport" : "61000"
```

11.2.2.2. Encaps section

The Encaps section is a configuration dictionary for the encapsulator. The identifiers of the remote Smart SFP 1G RS-232 (id) are the keys in this dictionary.

Up to 32 remote device identifiers can be configured.

For each remote device identifier, the following settings should be specified:

macd

The destination MAC address for the outgoing IEC-101 data packets. The destination MAC address can be set automatically or manually.

When manually configuring the destination MAC address, please note that if the data transmission occurs within the same subnet, the MAC address of the remote Smart SFP 1G RS-232 should be specified. If the devices are in different subnets and routing is used, you should specify the MAC address of the gateway through which the IEC-101 data packets will be transmitted.

Allowed values

Automatic mode for setting the destination MAC address is specified:

```
"macd" : "auto",
```

Or the destination MAC address is specified manually in the format:

```
"macd" : "00:aa:bb:cc:23:45",
```

Default

```
"macd" : "auto",
```

macs

The source MAC address for the outgoing IEC-101 data packets. The source MAC address can be set automatically or manually. When the source MAC address is set automatically, the MAC address of port B (gbe1) will be used.

Allowed values

Automatic mode for setting the source MAC address is specified:

```
"macd" : "auto",
```

Or the source MAC address is specified manually in the format:

```
"macs" : "00:aa:bb:cc:dd:1a",
```

Default

```
"macs" : "auto",
```

ips

The source IP address for the outgoing IEC-101 data packets. The source IP address can be set automatically or manually. When the source IP address is set automatically, the IP address of the port B (gbe1) will be used.

Allowed values

Automatic mode for setting the source IP address is specified:

```
"ips" : "auto",
```

Or the source IP address is specified manually in the format:

```
"ips" : "192.168.2.11",
```

Default

```
"ips" : "auto",
```

ipd

The destination IP address for the outgoing IEC-101 data packets.

Allowed values

Specify the destination IP address using format:

```
"ipd" : "192.168.2.12",
```

ups

The source UDP port for the outgoing IEC-101 data packets.

It is recommended to use ports from the private range 49152-65535.

Allowed values

Specify the source UDP port number from the range 1-65535 using format:

```
"ups" : "60000",
```

upd

The destination UDP port for the outgoing IEC-101 data packets.

It is recommended to use ports from the private range 49152-65535.

Allowed values

Specify the destination UDP port number from the range 1-65535 using format:

```
"upd" : "60001"
```

11.2.2.3. EtherFlt section

ipd

This parameter configures the decapsulator filter. You need to specify the IP address of the Smart SFP 1G RS-232 that will be set as the destination IP address in the incoming IEC-101 data packets from the remote Smart SFP 1G RS-232.

The decapsulator filter can be configured automatically or manually. When configured automatically, the decapsulator filter will use the IP address of the port B (gbe1).

Allowed values

The automatic configuration mode of the decapsulator filter is indicated:

"ipd" : "auto"

Or the decapsulator filter IP address is specified manually in the following format:

"ipd" : "192.168.2.11"

Default

"ipd" : "auto",

11.2.2.4. RS-232 section

baudrate

The data transmission speed through the RS-232 port in bits per second.

Allowed values

Specify the data transmission speed from the range of 300-115200 bps using format:

"baudrate" : "1200",

Default

"baudrate" : "9600",

flow_ctl

Enable or disable the hardware flow control (RTS/CTS control)

Allowed values

Specify the state as "on" or "off".

Default

"flow_ctl" : "off",

link_ctl

Enable or disable the hardware flow control (DTR/DSR/DCD control)

Allowed values

Specify the state as "on" or "off".

Default

"link_ctl" : "off",

parity

Enable or disable the parity control.

Allowed values

Specify the state as “on” or “off”.

Default

"parity" : "off",

parity_mode

Specify the parity control mode as odd or even. This setting affects the operation only when the parity control is enabled.

Allowed values

Specify the mode as “even” or “odd”.

Default

"parity_mode" : "even",

stop

Specify the number of stop-bits in RS-232 frames.

Allowed values

The number should be "1" or "2".

Default

"stop" : "2",

rx_inv

Enable or disable inversion of the RD circuit.

Allowed values

Specify the state as “on” or “off”.

Default

"rx_inv" : "off",

tx_inv

Enable or disable inversion of the TD circuit.

Allowed values

Specify the state as “on” or “off”.

Default

"tx_inv" : "off",

dtr_inv

Enable or disable inversion of the DTR circuit.

Allowed values

Specify the state as “on” or “off”.

Default

"dtr_inv" : "off",

dsr_inv

Enable or disable inversion of the DSR circuit.

Allowed values

Specify the state as “on” or “off”.

Default

"dsr_inv" : "off",

dcd_inv

Enable or disable inversion of the DCD circuit.

Allowed values

Specify the state as “on” or “off”.

Default

"dcd_inv" : "off",

rts_inv

Enable or disable inversion of the RTS circuit.

Allowed values

Specify the state as “on” or “off”.

Default

"rts_inv" : "off",

cts_inv

Enable or disable inversion of the CTS circuit.

Allowed values

Specify the state as “on” or “off”.

Default

"cts_inv" : "off",

prx_inv

Enable or disable inversion of the received parity bit.

Allowed values

Specify the state as “on” or “off”.

Default

"prx_inv" : "off",

ptx_inv

Enable or disable inversion of the transmitted parity bit.

Allowed values

Specify the state as “on” or “off”.

Default

```
"ptx_inv" : "off",
```

rx_los_override

Enabling the control of the module's RX_LOS signal by the data loss detecting at the input of the RS-232 port function. If this setting is enabled, the module will send an RX_LOS signal to the host device when data is lost at the RS-232 port input. When data is received again at the input of the RS-232 port, the RX_LOS signal will be disabled.

When using this feature, please be aware that some switches and routers disable the link on the interface that receives an RX_LOS signal. This may result in loss of the module control.

Allowed values

The “on” or “off” status is indicated.

Default

```
"rx_los_override" : "off",
```

rx_los_wd

Configuration of the data loss detecting at the input of the RS-232 port function. This function is used to detect problems with the RS-232/IEC-101 connection or connected RS-232/IEC-101 equipment.

This parameter specifies the time interval in seconds. If no characters (including error ones) are received during this interval, the module assumes that the connection at the RS-232 port input has been lost.

Allowed values

The time interval from the range of 2-255 seconds is specified in the format:

```
"rx_los_wd" : "5",
```

When the value is set to 0, the data loss detecting function at the input of the RS-232 port is disabled:

```
"rx_los_wd" : "0",
```

Default

```
"rx_los_wd" : "2",
```

ignore_starterr

Enabling the option to ignore start bit errors. If this option is disabled, transactions with start bit errors are discarded.

Allowed values

Specify the state as “on” or “off”.

Default

```
"ignore_starterr" : "off"
```

11.2.2.5. IEC section

length

The payload size in fixed-length packets. It is recommended to choose a value equal to 'Link address size' + 1.

When using the 'FREERUN' mode, this parameter configures the size of the packets sent to the remote module Smart SFP 1G RS-232.

Allowed values

Specify the packet size from the range of 1-4095 bytes using format:

"length" : "2",

Default

"length" : "2",

idle_bits

Allowed number of the IDLE bits between bytes inside the packet. Exceeding the specified value results in the immediate termination of the current packet processing.

Allowed values

Specify the number of bits from the range of 0-65535 using format:

"idle_bits" : "1",

Default

"idle_bits" : "1",

wait_idle_bits

The number of consecutive IDLE bits that the receiver must detect in the stream after completing the processing of a packet with an error.

Allowed values

Specify the number of bits from the range of 0-255 using format:

"wait_idle_bits" : "33",

Default

"wait_idle_bits" : "33",

freerun_mode_en

Enabling the FREERUN mode, where data is transmitted without analyzing the IEC 60870-5-101 structure. In this mode, fixed-size packets are sent according to the length setting.

Allowed values

Specify the state as "on" or "off".

Default

"freerun_mode_en" : "off",

drop_inval_pkts_en

Enable the dropping of erroneous packets of variable or fixed length. In this mode, packets that were processed with errors will not be sent to the network.

Allowed values

Specify the state as “on” or “off”.

Default

"drop_inval_pkts_en" : "off",

drop_inval_start_sym_en

Enable the dropping of erroneous single characters received outside the context of processing variable or fixed-length packets. In this mode, single characters that were received with start/stop bit errors or parity bit errors will not be sent to the network.

Allowed values

Specify the state as “on” or “off”.

Default

"drop_inval_start_sym_en" : "off",

drop_unspec_sym_en

Enable the dropping of single unspecified symbols. In this mode, single symbols that are not special symbols (0xE5, 0xA2), start symbols of variable-length (0x68) or fixed-length (0x10) packets, and that have no start/stop bit or parity bit errors during reception, will not be sent to the network.

Allowed values

Specify the state as “on” or “off”.

Default

"drop_unspec_sym_en" : "off",

ignore_data_start_err_en

Enable the ignoring of the start bit errors.

Allowed values

Specify the state as “on” or “off”.

Default

"ignore_data_start_err_en" : "off",

ignore_data_par_err_en

Enable the ignoring of the parity bit errors.

Allowed values

Specify the state as “on” or “off”.

Default

"ignore_data_par_err_en" : "off",

ignore_data_stop_err_en

Enable the ignoring of the stop bit errors.

Allowed values

Specify the state as “on” or “off”.

Default

```
"ignore_data_stop_err_en" : "off",
```

ignore_inval_data_en

Enable the ignoring of the start, stop or parity bit errors.

Allowed values

Specify the state as “on” or “off”.

Default

```
"ignore_inval_data_en" : "off",
```

ignore_inval_len_dup_en

Enable the ignoring of the 'L' and 'L repeated' IEC 60870-5-101 fields mismatch during variable-length packet processing.

Allowed values

Specify the state as “on” or “off”.

Default

```
"ignore_inval_len_dup_en" : "off",
```

ignore_inval_2_start_en

Enable the ignoring of the mismatch between the symbol after 'L repeated' field and the start symbol of the variable length packet – 0x68 during variable-length packet processing.

Allowed values

Specify the state as “on” or “off”.

Default

```
"ignore_inval_2_start_en" : "off",
```

ignore_checksum_en

Enable the ignoring of the mismatch between the calculated packet checksum and the received checksum value.

Allowed values

Specify the state as “on” or “off”.

Default

```
"ignore_checksum_en" : "off",
```

ignore_end_sym_en

Enable the ignoring of the mismatch between the last symbol of the packet and the packet end symbol – 0x68.

Allowed values

Specify the state as “on” or “off”.

Default

"ignore_end_sym_en" : "off"

11.2.3. IEC-101 data transmission service management

iecmnd stop

IEC-101 data transmission service stop. After stopping, the service will restart automatically.

Used to apply changes to the configuration file /var/lib/probe/etc/iecmnd/iecmnd_config.json.

Syntax

iecmnd stop

Example

```
root@smart-sfp:/home/admin# iecmnd stop
Stop is done.
```

iecmnd show

Show the IEC-101 data transmission service statistics. The parameters displayed in the statistics are listed in the table below.

Table 11.1. The parameters displayed in the statistics

Title	Description
Internal	Smart SFP internal statistics section.
encapse	Encapsulator statistics section.
flows	Encapsulator per flow statistics section.
ENCAPSE_PKTS	The number of encapsulated packets.
ENCAPSE_BYTES	The number of encapsulated bytes.
common	Encapsulator common statistics section.
DROP_PKTS	The number of dropped packets due to the buffer overflow.
DROP_BYTES	The number of dropped bytes due to the buffer overflow.
RATE_NET_TX	Data transmission rate to the network. Only encapsulated data without the header is measured. Units of measurement: bytes per second.
RATE_RAW_RX	RS-232 data reception rate for encapsulation. Units of measurement: bytes per second.
decapse	Decapsulator statistics section.
flows	Decapsulator per flow statistics section.
DECAPSE_PKTS	The number of decapsulated packets.
OOO_PKTS	The number of out-of-order packets. These are packets whose sequence number does not correspond to the previous packet's number incremented by one.
FTD_PKTS	The number of received frame transfer delay measurement packets. FTD packets are considered valid only if the reception time is greater than or equal to the packet's transmission time. FTD packets may be dropped due to incorrect time settings on the Smart SFP or in cases of low network latency with poor time synchronization accuracy on the Smart SFP. The FTD measurement will be accurate only if the time on the Smart SFP is synchronized using NTP or PTP protocols.

Title	Description
	The value is updated every 2 seconds and shows the number of FTD packets during that period.
NET_FTD_AVG	Average network frame transfer and processing delay. Updated every 2 seconds. Units of measurement – microseconds (us).
NET_FTD_MIN	Minimal network frame transfer and processing delay. Updated every 2 seconds. Units of measurement – microseconds (us).
NET_FTD_MAX	Maximum network frame transfer and processing delay. Updated every 2 seconds. Units of measurement – microseconds (us).
RAW_FTD_AVG	Average raw frame transfer delay between RS-232 interfaces. Updated every 2 seconds. Units of measurement – microseconds (us).
RAW_FTD_MIN	Minimal raw frame transfer delay between RS-232 interfaces. Updated every 2 seconds. Units of measurement – microseconds (us).
RAW_FTD_MAX	Maximum raw frame transfer delay between RS-232 interfaces. Updated every 2 seconds. Units of measurement – microseconds (us).
WRONG_SIZE_PKTS	The number of packets where the actual payload size differs from the size specified in the header.
DECAPSE_BYTES	The number of decapsulated bytes.
common	Decapsulator common statistics section.
WRONG_PKTS	The number of packets having protocol version or payload type mismatch.
DST_MISMATH_PKTS	The number of received packets with destination device identifier not matching the identifier configured on this Smart SFP.
SRC_MISMATH_PKTS	The number of received packets in which the sender device identifier does not match any of the remote device identifiers configured on this Smart SFP.
DROP_PKTS	The number of dropped packets due to the buffer overflow.
DROP_BYTES	The number of dropped bytes due to the buffer overflow.
RATE_RAW_RX	RS-232 data transmission rate after decapsulation. Units of measurement: bytes per second.
RATE_NET_TX	Data reception rate from the network. Only decapsulated data without the header is measured. Units of measurement: bytes per second.
iec_stat	IEC-101 protocol statistics section.
RX_SYM_I_CNT	The number of special symbols type I – 0xE5.
RX_SYM_II_CNT	The number of special symbols type II – 0xA2.
RX_UNSPEC_SYM_CNT	The number of valid but unspecified single symbols.
RX_INVALID_SYM_CNT	The number of invalid single symbols.
RX_FIX_PKT_START_SYM_CNT	The number of fixed-length packet start symbols – 0x10.
RX_VAR_PKT_START_SYM_CNT	The number of variable-length packet start symbols – 0x68.
RX_CORRECT_FIX_PKT_CNT	The number of fixed-length packets without any of the following errors: checksum error; end symbol (0x16) error.
RX_CORRECT_VAR_PKT_CNT	The number of variable-length packets without any of the following errors: 'L' and 'L repeated' IEC 60870-5-101 fields mismatch; second start symbol (0x68) error; checksum error; end symbol (0x16) error.
RX_PKT_INVAL_BYTES	The number of invalid bytes received during fixed- or variable-length packets processing.
RX_TIMEOUT_ERR_CNT	The number of packets for which a timeout occurred during reception.
RX LENGHT_DUP_ERR_CNT	The number of variable-length packets having 'L' and 'L repeated' IEC 60870-5-101 fields mismatch.
RX_START_DUP_ERR_CNT	The number of variable-length packets having second start symbol (0x68) error.

Title	Description
RX_CHECKSUM_ERR_CNT	The number of fixed- or variable-length packets having checksum error.
RX_END_ERR_CNT	The number of fixed- or variable-length packets having end symbol (0x16) error.
RX_PKTS1TO66OCTETS	The number of packets with a size of 1 to 66 bytes. Single characters are not counted.
RX_PKTS67TO132OCTETS	The number of packets with a size of 67 to 132 bytes. Single characters are not counted.
RX_PKTS133TO198OCTETS	The number of packets with a size of 133 to 198 bytes. Single characters are not counted.
RX_PKTS199TO261OCTETS	The number of packets with a size of 199 to 261 bytes. Single characters are not counted.
RX_PKTS262TO4095	The number of packets with a size of 261 to 4095 bytes. Single characters are not counted.
RX_MIN_PKT_SIZE	Minimal valid packet size.
RX_MAX_PKT_SIZE	Maximum valid packet size.
RX_INVALID_PKTS	The number of packets whose processing was interrupted due to errors but were still sent to the network.
RX_TOTAL_PKTS_SEND	The total number of packets sent to encapsulator.
TX_TOTAL_PKTS_SEND	The total number of packets received from decapsulator and transmitted to RS-232 interface.
rs232_stat	RS-232 interface statistics section.
RX_START_ERR_BYTES	The number of received bytes having start-bit error.
RX_PARITY_ERR_BYTES	The number of received bytes having parity-bit error.
RX_STOP_ERR_BYTES	The number of received bytes having stop-bit error.
RX_INVALID_BYTES	The number of received bytes having start-bit, parity-bit, or stop-bit error.
RX_CAPTURED_BYTES	The number of received bytes.
TX_SEND_BYTES	The number of transmitted bytes.
WAS_RX_LOS	Current state of the data loss detecting function at the input of the RS-232 port. Possible states: 1 – there is no data at the RS-232 port input during the rx_loss_wd period 0 – data was received at the RS-232 port input during the rx_loss_wd period, or the function is disabled (“rx_loss_wd”: “0”)
External	External statistics section. It contains statistics from remote Smart SFP 1G RS-232.
id	Device identifier of the remote Smart SFP.
addr	The remote Smart SFP socket to which IEC-101 data is transmitted.
status	Connection status. Possible statuses: - Online – connection established - Reconnecting – connection to the remote Smart SFP lost - RemoteDisable – IEC-101 data transmission service is disabled on the remote Smart SFP.
last_seen	Date and time of the last successful connection check.
stat	Section containing internal statistics, received from this remote Smart SFP.

Syntax

```
iecmdond show
```

Example

```
root@smart-sfp:/home/admin# iecmond show
{
  "Internal": {
    "encapse": {
      "flows": {
        "1": {
          "ENCAPSE_PKTS": 9518,
          "ENCAPSE_BYTES": 47590
        }
      },
      "common": {
        "DROP_PKTS": 0,
        "DROP_BYTES": 0,
        "RATE_RAW_RX": 10,
        "RATE_NET_TX": 10
      }
    },
    "decapse": {
      "flows": {
        "1": {
          "DECAPSE_PKTS": 9518,
          "OOO_PKTS": 0,
          "FTD_PKTS": 4,
          "NET_FTD_AVG": 142,
          "NET_FTD_MIN": 142,
          "NET_FTD_MAX": 142,
          "RAW_FTD_AVG": 41379,
          "RAW_FTD_MIN": 41378,
          "RAW_FTD_MAX": 41381,
          "WRONG_SIZE_PKTS": 0,
          "DECAPSE_BYTES": 47590
        }
      },
      "common": {
        "WRONG_PKTS": 0,
        "DST_MISMATH_PKTS": 0,
        "SRC_MISMATH_PKTS": 0,
        "DROP_PKTS": 0,
        "DROP_BYTES": 0,
        "RATE_RAW_RX": 10,
        "RATE_NET_TX": 10
      }
    },
    "iec_stat": {
      "RX_SYM_I_CNT": 0,
      "RX_SYM_II_CNT": 0,
      "RX_UNSPEC_SYM_CNT": 0,
      "RX_INVALID_SYM_CNT": 0,
      "RX_FIX_PKT_START_SYM_CNT": 9518,
      "RX_VAR_PKT_START_SYM_CNT": 0,
      "RX_CORRECT_FIX_PKT_CNT": 9518,
      "RX_CORRECT_VAR_PKT_CNT": 0,
      "RX_PKT_INVAL_BYTES": 0,
      "RX_TIMEOUT_ERR_CNT": 0,
      "RX LENGHT_DUP_ERR_CNT": 0,
      "RX_START_DUP_ERR_CNT": 0,
      "RX_CHECKSUM_ERR_CNT": 0,
      "RX_END_ERR_CNT": 0,
      "RX_PKTS1T0660CTETS": 9518,
      "RX_PKTS67T01320CTETS": 0,
      "RX_PKTS133T01980CTETS": 0,
      "RX_PKTS199T02610CTETS": 0,
      "RX_PKTS262T04095": 0,
      "RX_MIN_PKT_SIZE": 5,
      "RX_MAX_PKT_SIZE": 5,
    }
  }
}
```

```

    "RX_INVAL_PKTS": 0,
    "RX_TOTAL_PKTS_SEND": 9518,
    "TX_TOTAL_PKTS_SEND": 9518
  },
  "rs232_stat": {
    "RX_START_ERR_BYTES": 0,
    "RX_PARITY_ERR_BYTES": 0,
    "RX_STOP_ERR_BYTES": 0,
    "RX_INVALID_BYTES": 0,
    "RX_CAPTURED_BYTES": 47590,
    "TX_SEND_BYTES": 47590,
    "WAS_RX_LOS": 0
  }
},
"External": {
  "1": {
    "id": 1,
    "addr": "192.168.2.11:60001",
    "status": "Online",
    "last_seen": "Thu, 14 Feb 2019 18:27:18 +0000",
    "stat": {
      "encapse": {
        "flows": {
          "3": {
            "ENCAPSE_PKTS": 9427,
            "ENCAPSE_BYTES": 47135
          },
          "2": {
            "ENCAPSE_PKTS": 9427,
            "ENCAPSE_BYTES": 47135
          }
        },
        "common": {
          "DROP_PKTS": 0,
          "DROP_BYTES": 0,
          "RATE_RAW_RX": 10,
          "RATE_NET_TX": 20
        }
      }
    },
    "decapse": {
      "flows": {
        "3": {
          "DECAPSE_PKTS": 9424,
          "OOO_PKTS": 1,
          "FTD_PKTS": 4,
          "NET_FTD_AVG": 180,
          "NET_FTD_MIN": 180,
          "NET_FTD_MAX": 180,
          "RAW_FTD_AVG": 50953,
          "RAW_FTD_MIN": 50825,
          "RAW_FTD_MAX": 51042,
          "WRONG_SIZE_PKTS": 0,
          "DECAPSE_BYTES": 47120
        },
        "2": {
          "DECAPSE_PKTS": 9426,
          "OOO_PKTS": 0,
          "FTD_PKTS": 0,
          "NET_FTD_AVG": 0,
          "NET_FTD_MIN": 2251799813685,
          "NET_FTD_MAX": 0,
          "RAW_FTD_AVG": 0,
          "RAW_FTD_MIN": 2251799813685,
          "RAW_FTD_MAX": 0,
          "WRONG_SIZE_PKTS": 0,
          "DECAPSE_BYTES": 47130
        }
      }
    }
  }
}

```


iecmdond info

The current configuration of the IEC-101 data transfer service. For parameters configured in auto mode, the current parameter value, which was set automatically, is displayed.

Syntax

iecmdond info

Example

```
root@smart-sfp:/home/admin# iecmdond info
{
  "Settings": {
    "id": "1",
    "cport": "61000"
  },
  "Encaps": {
    "2": {
      "macd": "00:21:ce:60:00:53",
      "macs": "00:21:ce:60:00:1b",
      "ips": "192.168.2.11",
      "ipd": "192.168.2.12",
      "ups": "60000",
      "upd": "60001"
    },
    "3": {
      "macd": "00:21:ce:60:00:5b",
      "macs": "00:21:ce:60:00:1b",
      "ips": "192.168.2.11",
      "ipd": "192.168.2.13",
      "ups": "60000",
      "upd": "60001"
    }
  },
  "RS232": {
    "baudrate": "1200",
    "flow_ctl": "Off",
    "link_ctl": "Off",
    "parity": "Off",
    "parity_mode": "even",
    "stop": "1",
    "rx_inv": "Off",
    "tx_inv": "Off",
    "dtr_inv": "Off",
    "dsr_inv": "Off",
    "dcd_inv": "Off",
    "rts_inv": "Off",
    "cts_inv": "Off",
    "prx_inv": "Off",
    "ptx_inv": "Off",
    "ignore_starterr": "Off"
  },
  "EtherFlt": {
    "ipd": "192.168.2.11"
  },
  "IEC": {
    "length": "2",
    "idle_bits": "1",
    "wait_idle_bits": "33",
    "freerun_mode_en": "Off",
    "drop_inval_pkts_en": "Off",
    "drop_inval_start_sym_en": "Off",
    "drop_unspec_sym_en": "Off",
    "ignore_data_start_err_en": "Off",
    "ignore_data_par_err_en": "Off",
    "ignore_data_stop_err_en": "Off",
  }
}
```

```

        "ignore_inval_data_en": "Off",
        "ignore_inval_len_dup_en": "Off",
        "ignore_inval_2_start_en": "Off",
        "ignore_checksum_en": "Off",
        "ignore_end_sym_en": "Off"
    }
}

```

iecmnd pinmode

Configuration of manual or automatic control mode for CTS/DSR/DCD lines.

Syntax

```
iecmnd pinmode { auto | manual }
```

Parameters

- auto – enable automatic control mode for CTS/DSR/DCD lines
- manual – enable manual control mode for CTS/DSR/DCD lines

Example

```

root@smart-sfp:/home/admin# iecmnd pinmode auto
{
  "status": {
    "RD": 1,
    "TD": 1,
    "RTS": 0,
    "DCD": 1,
    "DTR": 1,
    "CTS": 1,
    "DSR": 1
  },
  "mode": {
    "DCD": "auto",
    "DSR": "auto",
    "CTS": "auto"
  }
}

```

iecmnd pinset

Configuration of the state for each of CTS/DSR/DCD lines in manual control mode.

Syntax

```
iecmnd pinset { CTS | DSR | DCD } { 0 | 1 }
```

Parameters

- CTS – configure state “0” or “1” for CTS line
- DSR – configure state “0” or “1” for DSR line
- DCD – configure state “0” or “1” for DCD line

Example

```

root@smart-sfp:/home/admin# iecmnd pinset CTS 1
{
  "status": {
    "TD": 1,
    "DCD": 0,
    "CTS": 1,
    "RD": 1,
    "RTS": 0,

```

```

        "DSR": 0,
        "DTR": 1
    },
    "mode": {
        "DSR": "manual",
        "DCD": "manual",
        "CTS": "manual"
    }
}

```

iecmnd pinshow

Show the current state of all RS-232 lines and the current control mode for CTS/DSR/DCD lines.

Syntax

```
iecmnd show
```

Example

```

root@smart-sfp:/home/admin# iecmnd pinshow
{
  "status": {
    "TD": 1,
    "CTS": 0,
    "RTS": 0,
    "DCD": 0,
    "RD": 1,
    "DTR": 1,
    "DSR": 0
  },
  "mode": {
    "DSR": "manual",
    "CTS": "manual",
    "DCD": "manual"
  }
}

```

iecmnd loopback

RS-232 loopback configuration.

Syntax

```
iecmnd loopback { show | on | off }
```

Parameters

- show – show the current state of loopback
- on – turn RS-232 loopback on
- off – turn RS-232 loopback off

Example

```

root@smart-sfp:/home/admin# iecmnd loopback on
Loopback status: 1
root@smart-sfp:/home/admin# iecmnd loopback show
Loopback status: 1
root@smart-sfp:/home/admin# iecmnd loopback off
Loopback status: 0

```

11.2.4. IEC-101 data transmission state monitoring using SNMP

The SNNPd server is used to monitor IEC-101 data transmission state and module status. It is enabled by default and is available for connecting an SNMP client. When connecting, the client should use the SNMP v2c protocol and community public.

The SNMPd settings are located in the /etc/snmp/snmpd.conf file. When working with SNMPd settings, refer to the documentation, which is publicly available. After changing the SNMPd settings, you should restart it using the command:

```
root@smart-sfp:/home/admin# systemctl restart snmpd
```

Monitoring using standard MIB files is supported: MIB-II, IF-MIB, LM-SENSORS-MIB, ENTITY-MIB, as well as proprietary OIDs for monitoring IEC-101 transmission.

11.2.4.1. Proprietary OIDs for IEC-101 transmission monitoring

The list of proprietary OIDs is provided in the table:

OID	Displayed parameter	Data type
.1.3.6.1.4.1.63222.2.1.4.2.1	SNMP API IEC/RS232 version	Integer32
.1.3.6.1.4.1.63222.2.1.4.2.2.1.1.1	decapse / common / WRONG_PKTS	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.1.1.2	decapse / common / RATE_RAW_TX	Integer32
.1.3.6.1.4.1.63222.2.1.4.2.2.1.1.3	decapse / common / RATE_NET_RX	Integer32
.1.3.6.1.4.1.63222.2.1.4.2.2.1.1.4	decapse / common / DROP_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.1.1.5	decapse / common / DST_MISMATCH_PKTS	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.1.1.6	decapse / common / SRC_MISMATCH_PKTS	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.1.1.7	decapse / common / DROP_PKTS	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.1.2.1	encapse / common / DROP_PKTS	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.1.2.2	encapse / common / DROP_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.1.2.3	encapse / common / RATE_NET_TX	Integer32
.1.3.6.1.4.1.63222.2.1.4.2.2.1.2.4	encapse / common / RATE_RAW_RX	Integer32
.1.3.6.1.4.1.63222.2.1.4.2.2.2.1	iec_stat / RX_PKT_INVALID_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.2.2	iec_stat / RX_MIN_PKT_SIZE	Integer32
.1.3.6.1.4.1.63222.2.1.4.2.2.2.3	iec_stat / RX_MAX_PKT_SIZE	Integer32
.1.3.6.1.4.1.63222.2.1.4.2.2.2.4	iec_stat / RX_INVALID_PKTS	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.2.5	iec_stat / RX_TOTAL_PKTS_SEND	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.2.6	iec_stat / TX_TOTAL_PKTS_SEND	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.3.1	rs232_stat / RX_START_ERR_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.3.2	rs232_stat / RX_PARITY_ERR_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.3.3	rs232_stat / RX_STOP_ERR_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.3.4	rs232_stat / RX_INVALID_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.3.5	rs232_stat / RX_CAPTURED_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.3.6	rs232_stat / TX_SEND_BYTES	Counter64
.1.3.6.1.4.1.63222.2.1.4.2.2.3.7	rs232_stat / WAS_RX_LOS	Integer32

11.2.5. Backup and restore the configuration of the module with IEC-101 data transmission function

Smart SFP 1G RS-232 with IEC-101 data transmission function has configuration backup and restore function.

config-tool export

Export of the module configuration. To run, it is necessary to log in as root.

Syntax

```
config-tool export { -p backup.tar }
```

Parameters

- without the -p option, the file name “configs_2025-08-14_10.35.54.tar” is generated automatically
- after the -p option, specify the file name or path where the configuration will be saved. When specifying the path, you should specify a directory that exists in memory: config-tool export -p /home/admin/config.tar

Example

```
root@smart-sfp:/home/admin# config-tool export -p config.tar
=====
Show process journal /home/admin/config.tar.log
-----
2025-09-14 15:42:17,583 INFO : Journal
2025-09-14 15:42:17,594 INFO : .....
2025-09-14 15:42:17,595 INFO : Files to export:
2025-09-14 15:42:17,596 INFO : /etc/cron.d/
2025-09-14 15:42:17,597 INFO : /etc/cron.d/.placeholder
2025-09-14 15:42:17,598 INFO : /etc/cron.daily/
2025-09-14 15:42:17,598 INFO : /etc/cron.daily/.placeholder
2025-09-14 15:42:17,599 INFO : /etc/cron.daily/apt-compat
2025-09-14 15:42:17,600 INFO : /etc/cron.daily/bsdmainutils
2025-09-14 15:42:17,601 INFO : /etc/cron.daily/dpkg
2025-09-14 15:42:17,602 INFO : /etc/cron.daily/lighttpd
2025-09-14 15:42:17,603 INFO : /etc/cron.daily/logrotate
2025-09-14 15:42:17,604 INFO : /etc/cron.daily/ntp
2025-09-14 15:42:17,605 INFO : /etc/cron.daily/passwd
2025-09-14 15:42:17,605 INFO : /etc/cron.hourly/
2025-09-14 15:42:17,606 INFO : /etc/cron.hourly/.placeholder
2025-09-14 15:42:17,607 INFO : /etc/cron.monthly/
2025-09-14 15:42:17,608 INFO : /etc/cron.monthly/.placeholder
2025-09-14 15:42:17,609 INFO : /etc/cron.weekly/
2025-09-14 15:42:17,610 INFO : /etc/cron.weekly/.placeholder
2025-09-14 15:42:17,611 INFO : /etc/crontab
2025-09-14 15:42:17,612 INFO : /etc/group
2025-09-14 15:42:17,614 INFO : /etc/hostname
2025-09-14 15:42:17,615 INFO : /etc/hosts
2025-09-14 15:42:17,621 INFO : /etc/ntp.conf
2025-09-14 15:42:17,623 INFO : /etc/passwd
2025-09-14 15:42:17,623 INFO : /etc/shadow
2025-09-14 15:42:17,624 INFO : /etc/snmp/snmpd.conf
2025-09-14 15:42:17,625 INFO : /etc/ssh/ssh_config
2025-09-14 15:42:17,626 INFO : /etc/ssh/ssh_config.d
2025-09-14 15:42:17,627 INFO : /etc/ssh/sshd_config
2025-09-14 15:42:17,628 INFO : /etc/ssh/sshd_config.d
2025-09-14 15:42:17,628 INFO : /var/etu
2025-09-14 15:42:17,629 INFO : /var/lib/ntp/ntp.drift
2025-09-14 15:42:17,630 INFO : /var/lib/probe/etc/iecmnd/iecmnd_config.json
2025-09-14 15:42:17,638 INFO : /var/lib/probe/etc/network
2025-09-14 15:42:17,639 INFO : /var/lib/probe/etc/network/interfaces
2025-09-14 15:42:17,641 INFO : /var/lib/probe/etc/network/interfaces.d
2025-09-14 15:42:17,642 INFO : /var/lib/probe/etc/network/interfaces.d/gbe
2025-09-14 15:42:17,643 INFO : /var/lib/probe/etc/network/interfaces.d/usb
2025-09-14 15:42:17,644 INFO : Total: 37
2025-09-14 15:42:17,645 INFO : .....
2025-09-14 15:42:17,646 INFO : Check files...
2025-09-14 15:42:17,664 WARNING : Does not exist: /etc/ssh/ssh_config.d
2025-09-14 15:42:17,666 WARNING : Does not exist: /etc/ssh/sshd_config.d
```

```

2025-09-14 15:42:17,668 WARNING : Does not exist: /var/lib/ntp/ntp.drift
2025-09-14 15:42:17,673 INFO : Done
2025-09-14 15:42:17,837 INFO : .....
2025-09-14 15:42:17,839 INFO : Archive content:
2025-09-14 15:42:17,839 INFO :      MODE      UID:GID      USER:GROUP      SIZE      PATH
2025-09-14 15:42:17,841 INFO :      40755 0:0      root:root        0      etc/cron.d/
2025-09-14 15:42:17,842 INFO :      100644 0:0      root:root        102     etc/cron.d/.placeholder
2025-09-14 15:42:17,843 INFO :      40755 0:0      root:root        0      etc/cron.daily/
2025-09-14 15:42:17,844 INFO :      100644 0:0      root:root        102
etc/cron.daily/.placeholder
2025-09-14 15:42:17,846 INFO :      100755 0:0      root:root        1478    etc/cron.daily/apt-compat
2025-09-14 15:42:17,847 INFO :      100755 0:0      root:root        355
etc/cron.daily/bsdmainutils
2025-09-14 15:42:17,848 INFO :      100755 0:0      root:root        1187    etc/cron.daily/dpkg
2025-09-14 15:42:17,849 INFO :      100755 0:0      root:root        338     etc/cron.daily/lighttpd
2025-09-14 15:42:17,850 INFO :      100755 0:0      root:root        377     etc/cron.daily/logrotate
2025-09-14 15:42:17,851 INFO :      100755 0:0      root:root        1403    etc/cron.daily/ntp
2025-09-14 15:42:17,852 INFO :      100755 0:0      root:root        249     etc/cron.daily/passwd
2025-09-14 15:42:17,853 INFO :      40755 0:0      root:root        0      etc/cron.hourly/
2025-09-14 15:42:17,854 INFO :      100644 0:0      root:root        102
etc/cron.hourly/.placeholder
2025-09-14 15:42:17,855 INFO :      40755 0:0      root:root        0      etc/cron.monthly/
2025-09-14 15:42:17,856 INFO :      100644 0:0      root:root        102
etc/cron.monthly/.placeholder
2025-09-14 15:42:17,857 INFO :      40755 0:0      root:root        0      etc/cron.weekly/
2025-09-14 15:42:17,858 INFO :      100644 0:0      root:root        102
etc/cron.weekly/.placeholder
2025-09-14 15:42:17,859 INFO :      100644 0:0      root:root        1042    etc/crontab
2025-09-14 15:42:17,861 INFO :      100644 0:0      root:root        732     etc/group
2025-09-14 15:42:17,862 INFO :      100644 0:0      root:root        10      etc/hostname
2025-09-14 15:42:17,863 INFO :      100644 0:0      root:root        109     etc/hosts
2025-09-14 15:42:17,864 INFO :      100777 0:0      root:root        2172    etc/ntp.conf
2025-09-14 15:42:17,865 INFO :      100644 0:0      root:root        1505    etc/passwd
2025-09-14 15:42:17,867 INFO :      100640 0:42     root:shadow      1121    etc/shadow
2025-09-14 15:42:17,868 INFO :      100600 0:0      root:root        7160    etc/snmp/snmpd.conf
2025-09-14 15:42:17,869 INFO :      100644 0:0      root:root        1580    etc/ssh/ssh_config
2025-09-14 15:42:17,870 INFO :      100644 0:0      root:root        3250    etc/ssh/sshd_config
2025-09-14 15:42:17,871 INFO :      40755 0:0      root:root        0      var/etu
2025-09-14 15:42:17,872 INFO :      100755 0:0      root:root        1681
var/lib/probe/etc/iecmnd/iecmnd_config.json
2025-09-14 15:42:17,873 INFO :      40755 0:0      root:root        0      var/lib/probe/etc/network
2025-09-14 15:42:17,874 INFO :      100644 0:0      root:root        228
var/lib/probe/etc/network/interfaces
2025-09-14 15:42:17,875 INFO :      40755 0:0      root:root        0
var/lib/probe/etc/network/interfaces.d
2025-09-14 15:42:17,876 INFO :      100777 0:0      root:root        146
var/lib/probe/etc/network/interfaces.d/gbe
2025-09-14 15:42:17,877 INFO :      100644 0:0      root:root        110
var/lib/probe/etc/network/interfaces.d/usb
2025-09-14 15:42:17,878 INFO :      100600 0:0      root:root        2607    METADATA.ini
2025-09-14 15:42:17,879 INFO :
2025-09-14 15:42:17,880 INFO : Total elements count: 35
2025-09-14 15:42:17,881 INFO : .....
2025-09-14 15:42:17,882 INFO : Exported into: /home/admin/config.tar

=====
.....
34 file(s) processed.
Export process successfully done.
.....

```

config-tool import

Import of the module configuration. To run, it is necessary to log in as root. After completing the configuration import, it is necessary to reboot the module using the reboot command.

Syntax

```
config-tool import -a backup.tar
```

Parameters

- after the -a option, specify the file name or path where the file with the backup copy of the configuration is located

Example

```
root@smart-sfp:/home/admin# config-tool import -a config.tar
.....
Root file system is read only. Remount will be performed.
[ 3198.081487] EXT4-fs (mmcblk0p2): re-mounted 9a3a0d9a-9119-43a0-a70f-5abbe74ea57e r/w. Quota mode:
none.
Remount done.
=====
Show process journal /home/admin/config.tar.log
-----
2025-09-14 15:50:15,936 INFO : Journal
2025-09-14 15:50:16,225 WARNING : .....
2025-09-14 15:50:16,227 WARNING : Pre-import actions:
2025-09-14 15:50:16,228 WARNING : App services are not running. Will not be stopped.
2025-09-14 15:50:16,308 INFO :
2025-09-14 15:50:16,309 INFO : .....
2025-09-14 15:50:16,310 INFO : Device information:
2025-09-14 15:50:16,312 INFO :

v      : 1
sn     : 00380
pn     : IEC101
vend   : SmartSFP
prod   : SmartSFP
hwrev  : 1
hwrev1 : 1
mac3   : 00:11:22:33:44:53
mac4   : 00:11:22:33:44:54

2025-09-14 15:50:16,313 INFO : .....
2025-09-14 15:50:16,320 INFO : .....
2025-09-14 15:50:16,321 INFO : Files in archive:
2025-09-14 15:50:16,322 INFO : /etc/cron.d
2025-09-14 15:50:16,325 INFO : /etc/cron.d/.placeholder
2025-09-14 15:50:16,327 INFO : /etc/cron.daily
2025-09-14 15:50:16,329 INFO : /etc/cron.daily/.placeholder
2025-09-14 15:50:16,331 INFO : /etc/cron.daily/apt-compat
2025-09-14 15:50:16,333 INFO : /etc/cron.daily/bsdmainutils
2025-09-14 15:50:16,335 INFO : /etc/cron.daily/dpkg
2025-09-14 15:50:16,337 INFO : /etc/cron.daily/lighttpd
2025-09-14 15:50:16,339 INFO : /etc/cron.daily/logrotate
2025-09-14 15:50:16,342 INFO : /etc/cron.daily/ntp
2025-09-14 15:50:16,344 INFO : /etc/cron.daily/passwd
2025-09-14 15:50:16,346 INFO : /etc/cron.hourly
2025-09-14 15:50:16,348 INFO : /etc/cron.hourly/.placeholder
2025-09-14 15:50:16,350 INFO : /etc/cron.monthly
2025-09-14 15:50:16,352 INFO : /etc/cron.monthly/.placeholder
2025-09-14 15:50:16,354 INFO : /etc/cron.weekly
2025-09-14 15:50:16,357 INFO : /etc/cron.weekly/.placeholder
2025-09-14 15:50:16,359 INFO : /etc/crontab
2025-09-14 15:50:16,361 INFO : /etc/group
2025-09-14 15:50:16,363 INFO : /etc/hostname
2025-09-14 15:50:16,366 INFO : /etc/hosts
2025-09-14 15:50:16,368 INFO : /etc/ntp.conf
2025-09-14 15:50:16,370 INFO : /etc/passwd
2025-09-14 15:50:16,372 INFO : /etc/shadow
2025-09-14 15:50:16,374 INFO : /etc/snmp/snmpd.conf
2025-09-14 15:50:16,376 INFO : /etc/ssh/ssh_config
2025-09-14 15:50:16,379 INFO : /etc/ssh/sshd_config
2025-09-14 15:50:16,381 INFO : /var/etu
2025-09-14 15:50:16,383 INFO : /var/lib/probe/etc/iecmnd/iecmnd_config.json
2025-09-14 15:50:16,385 INFO : /var/lib/probe/etc/network
2025-09-14 15:50:16,387 INFO : /var/lib/probe/etc/network/interfaces
2025-09-14 15:50:16,389 INFO : /var/lib/probe/etc/network/interfaces.d
2025-09-14 15:50:16,391 INFO : /var/lib/probe/etc/network/interfaces.d/gbe
2025-09-14 15:50:16,393 INFO : /var/lib/probe/etc/network/interfaces.d/usb
2025-09-14 15:50:16,396 INFO :
2025-09-14 15:50:16,397 INFO : .....
2025-09-14 15:50:16,398 INFO : Create backup archive: /home/admin/config_backup.tar
2025-09-14 15:50:16,410 INFO : .....
2025-09-14 15:50:16,412 INFO : Files to export:
2025-09-14 15:50:16,413 INFO : /etc/cron.d
2025-09-14 15:50:16,413 INFO : /etc/cron.d/.placeholder
2025-09-14 15:50:16,414 INFO : /etc/cron.daily
2025-09-14 15:50:16,416 INFO : /etc/cron.daily/.placeholder
```

```

2025-09-14 15:50:16,417 INFO : /etc/cron.daily/apt-compat
2025-09-14 15:50:16,417 INFO : /etc/cron.daily/bsdmainutils
2025-09-14 15:50:16,418 INFO : /etc/cron.daily/dpkg
2025-09-14 15:50:16,419 INFO : /etc/cron.daily/lighttpd
2025-09-14 15:50:16,420 INFO : /etc/cron.daily/logrotate
2025-09-14 15:50:16,421 INFO : /etc/cron.daily/ntp
2025-09-14 15:50:16,422 INFO : /etc/cron.daily/passwd
2025-09-14 15:50:16,423 INFO : /etc/cron.hourly
2025-09-14 15:50:16,424 INFO : /etc/cron.hourly/.placeholder
2025-09-14 15:50:16,424 INFO : /etc/cron.monthly
2025-09-14 15:50:16,425 INFO : /etc/cron.monthly/.placeholder
2025-09-14 15:50:16,426 INFO : /etc/cron.weekly
2025-09-14 15:50:16,427 INFO : /etc/cron.weekly/.placeholder
2025-09-14 15:50:16,428 INFO : /etc/crontab
2025-09-14 15:50:16,429 INFO : /etc/group
2025-09-14 15:50:16,430 INFO : /etc/hostname
2025-09-14 15:50:16,431 INFO : /etc/hosts
2025-09-14 15:50:16,432 INFO : /etc/ntp.conf
2025-09-14 15:50:16,433 INFO : /etc/passwd
2025-09-14 15:50:16,434 INFO : /etc/shadow
2025-09-14 15:50:16,435 INFO : /etc/snmp/snmpd.conf
2025-09-14 15:50:16,435 INFO : /etc/ssh/ssh_config
2025-09-14 15:50:16,436 INFO : /etc/ssh/sshd_config
2025-09-14 15:50:16,437 INFO : /var/etu
2025-09-14 15:50:16,438 INFO : /var/lib/probe/etc/iecmnd/iecmnd_config.json
2025-09-14 15:50:16,439 INFO : /var/lib/probe/etc/network
2025-09-14 15:50:16,440 INFO : /var/lib/probe/etc/network/interfaces
2025-09-14 15:50:16,441 INFO : /var/lib/probe/etc/network/interfaces.d
2025-09-14 15:50:16,442 INFO : /var/lib/probe/etc/network/interfaces.d/gbe
2025-09-14 15:50:16,443 INFO : /var/lib/probe/etc/network/interfaces.d/usb
2025-09-14 15:50:16,444 INFO : Total: 34
2025-09-14 15:50:16,445 INFO : .....
2025-09-14 15:50:16,446 INFO : Check files...
2025-09-14 15:50:16,470 INFO : Done
2025-09-14 15:50:16,638 INFO : .....
2025-09-14 15:50:16,639 INFO : Archive content:
2025-09-14 15:50:16,640 INFO :      MODE      UID:GID    USER:GROUP      SIZE      PATH
2025-09-14 15:50:16,642 INFO :      40755 0:0          root:root         0          etc/cron.d
2025-09-14 15:50:16,643 INFO :      100644 0:0          root:root        102         etc/cron.d/.placeholder
2025-09-14 15:50:16,644 INFO :      40755 0:0          root:root         0          etc/cron.daily
2025-09-14 15:50:16,645 INFO :      100644 0:0          root:root        102
etc/cron.daily/.placeholder
2025-09-14 15:50:16,646 INFO :      100755 0:0          root:root       1478         etc/cron.daily/apt-compat
2025-09-14 15:50:16,647 INFO :      100755 0:0          root:root        355
etc/cron.daily/bsdmainutils
2025-09-14 15:50:16,648 INFO :      100755 0:0          root:root       1187         etc/cron.daily/dpkg
2025-09-14 15:50:16,649 INFO :      100755 0:0          root:root        338         etc/cron.daily/lighttpd
2025-09-14 15:50:16,650 INFO :      100755 0:0          root:root        377         etc/cron.daily/logrotate
2025-09-14 15:50:16,652 INFO :      100755 0:0          root:root       1403         etc/cron.daily/ntp
2025-09-14 15:50:16,653 INFO :      100755 0:0          root:root        249         etc/cron.daily/passwd
2025-09-14 15:50:16,654 INFO :      40755 0:0          root:root         0          etc/cron.hourly
2025-09-14 15:50:16,655 INFO :      100644 0:0          root:root        102
etc/cron.hourly/.placeholder
2025-09-14 15:50:16,656 INFO :      40755 0:0          root:root         0          etc/cron.monthly
2025-09-14 15:50:16,657 INFO :      100644 0:0          root:root        102
etc/cron.monthly/.placeholder
2025-09-14 15:50:16,659 INFO :      40755 0:0          root:root         0          etc/cron.weekly
2025-09-14 15:50:16,660 INFO :      100644 0:0          root:root        102
etc/cron.weekly/.placeholder
2025-09-14 15:50:16,661 INFO :      100644 0:0          root:root       1042         etc/crontab
2025-09-14 15:50:16,662 INFO :      100644 0:0          root:root        732         etc/group
2025-09-14 15:50:16,663 INFO :      100644 0:0          root:root         10         etc/hostname
2025-09-14 15:50:16,664 INFO :      100644 0:0          root:root         109         etc/hosts
2025-09-14 15:50:16,665 INFO :      100777 0:0          root:root       2172         etc/ntp.conf
2025-09-14 15:50:16,666 INFO :      100644 0:0          root:root       1505         etc/passwd
2025-09-14 15:50:16,667 INFO :      100640 0:42          root:shadow     1121         etc/shadow
2025-09-14 15:50:16,669 INFO :      100600 0:0          root:root       7160         etc/snmp/snmpd.conf
2025-09-14 15:50:16,670 INFO :      100644 0:0          root:root       1580         etc/ssh/ssh_config
2025-09-14 15:50:16,670 INFO :      100644 0:0          root:root      3250         etc/ssh/sshd_config
2025-09-14 15:50:16,672 INFO :      40755 0:0          root:root         0          var/etu
2025-09-14 15:50:16,673 INFO :      100755 0:0          root:root      1681
var/lib/probe/etc/iecmnd/iecmnd_config.json
2025-09-14 15:50:16,674 INFO :      40755 0:0          root:root         0          var/lib/probe/etc/network
2025-09-14 15:50:16,675 INFO :      100644 0:0          root:root        228
var/lib/probe/etc/network/interfaces
2025-09-14 15:50:16,676 INFO :      40755 0:0          root:root         0
var/lib/probe/etc/network/interfaces.d

```



```

2025-09-14 15:50:16,677 INFO : 100777 0:0          root:root          146
var/lib/probe/etc/network/interfaces.d/gbe
2025-09-14 15:50:16,678 INFO : 100644 0:0          root:root          110
var/lib/probe/etc/network/interfaces.d/usb
2025-09-14 15:50:16,679 INFO : 100600 0:0          root:root          2607      METADATA.ini
2025-09-14 15:50:16,680 INFO :
2025-09-14 15:50:16,682 INFO : Total elements count: 35
2025-09-14 15:50:16,683 INFO : .....
2025-09-14 15:50:16,684 INFO : Exported into: /home/admin/config_backup.tar
2025-09-14 15:50:16,685 INFO : .....
2025-09-14 15:50:16,686 INFO : Unpack archive /home/admin/config.tar into filesystem:
2025-09-14 15:50:16,694 INFO : Unpack /etc/cron.d
2025-09-14 15:50:16,703 INFO : Unpack /etc/cron.d/.placeholder
2025-09-14 15:50:16,723 INFO : Unpack /etc/cron.daily
2025-09-14 15:50:16,726 INFO : Unpack /etc/cron.daily/.placeholder
2025-09-14 15:50:16,750 INFO : Unpack /etc/cron.daily/apt-compat
2025-09-14 15:50:16,769 INFO : Unpack /etc/cron.daily/bsdmainutils
2025-09-14 15:50:16,783 INFO : Unpack /etc/cron.daily/dpkg
2025-09-14 15:50:16,812 INFO : Unpack /etc/cron.daily/lighttpd
2025-09-14 15:50:16,818 INFO : Unpack /etc/cron.daily/logrotate
2025-09-14 15:50:16,823 INFO : Unpack /etc/cron.daily/ntp
2025-09-14 15:50:16,832 INFO : Unpack /etc/cron.daily/passwd
2025-09-14 15:50:16,843 INFO : Unpack /etc/cron.hourly
2025-09-14 15:50:16,845 INFO : Unpack /etc/cron.hourly/.placeholder
2025-09-14 15:50:16,863 INFO : Unpack /etc/cron.monthly
2025-09-14 15:50:16,866 INFO : Unpack /etc/cron.monthly/.placeholder
2025-09-14 15:50:16,882 INFO : Unpack /etc/cron.weekly
2025-09-14 15:50:16,885 INFO : Unpack /etc/cron.weekly/.placeholder
2025-09-14 15:50:16,889 INFO : Unpack /etc/crontab
2025-09-14 15:50:16,902 INFO : Unpack /etc/group
2025-09-14 15:50:16,907 INFO : Unpack /etc/hostname
2025-09-14 15:50:16,915 INFO : Unpack /etc/hosts
2025-09-14 15:50:16,932 INFO : Unpack /etc/ntp.conf
2025-09-14 15:50:16,937 INFO : Unpack /etc/passwd
2025-09-14 15:50:16,942 INFO : Unpack /etc/shadow
2025-09-14 15:50:16,947 INFO : Unpack /etc/snmp/snmpd.conf
2025-09-14 15:50:16,952 INFO : Unpack /etc/ssh/ssh_config
2025-09-14 15:50:16,957 INFO : Unpack /etc/ssh/sshd_config
2025-09-14 15:50:16,963 INFO : Unpack /var/etu
2025-09-14 15:50:16,967 INFO : Unpack /var/lib/probe/etc/iecmnd/iecmnd_config.json
2025-09-14 15:50:16,973 INFO : Unpack /var/lib/probe/etc/network
2025-09-14 15:50:16,976 INFO : Unpack /var/lib/probe/etc/network/interfaces
2025-09-14 15:50:16,980 INFO : Unpack /var/lib/probe/etc/network/interfaces.d
2025-09-14 15:50:16,983 INFO : Unpack /var/lib/probe/etc/network/interfaces.d/gbe
2025-09-14 15:50:16,988 INFO : Unpack /var/lib/probe/etc/network/interfaces.d/usb
2025-09-14 15:50:16,994 INFO :
2025-09-14 15:50:16,995 INFO : .....
2025-09-14 15:50:17,023 INFO :
2025-09-14 15:50:17,024 INFO : Files checksums verification was successful.
2025-09-14 15:50:17,025 INFO :
2025-09-14 15:50:17,026 INFO : .....
2025-09-14 15:50:17,027 INFO : Device information:
2025-09-14 15:50:17,028 INFO :

v      : 1
sn     : 00380
pn     : IEC101
vend   : SmartSFP
prod   : SmartSFP
hwrev  : 1
hwrev1 : 1
mac3   : 00:11:22:33:44:53
mac4   : 00:11:22:33:44:54

2025-09-14 15:50:17,029 INFO : .....

=====
[ 3200.361822] EXT4-fs (mmcblk0p2): re-mounted 9a3a0d9a-9119-43a0-a70f-5abbe74ea57e ro. Quota mode:
none.
Remount root file system back to read only done.
.....
34 file(s) processed.
Import process successfully done. You should reboot the device.
.....

```

config-tool show

Information about the backup copy of the module configuration.

Syntax

```
config-tool show -a backup.tar
```

Parameters

- after the -a option, specify the file name or path where the file with the backup copy of the configuration is located

Example

```
root@smart-sfp:/home/admin# config-tool show -a config.tar

.....
Device information:

v      : 1
sn     : 00380
pn     : IEC101
vend   : SmartSFP
prod   : SmartSFP
hwrev  : 1
hwrev1 : 1
mac3   : 00:11:22:33:44:53
mac4   : 00:11:22:33:44:54

.....
Archive content:

MODE   UID:GID   USER:GROUP   SIZE   PATH
755 0:0     root:root     0      etc/cron.d
644 0:0     root:root     102    etc/cron.d/.placeholder
755 0:0     root:root     0      etc/cron.daily
644 0:0     root:root     102    etc/cron.daily/.placeholder
755 0:0     root:root     1478   etc/cron.daily/apt-compat
755 0:0     root:root     355    etc/cron.daily/bsdmainutils
755 0:0     root:root     1187   etc/cron.daily/dpkg
755 0:0     root:root     338    etc/cron.daily/lighttpd
755 0:0     root:root     377    etc/cron.daily/logrotate
755 0:0     root:root     1403   etc/cron.daily/ntp
755 0:0     root:root     249    etc/cron.daily/passwd
755 0:0     root:root     0      etc/cron.hourly
644 0:0     root:root     102    etc/cron.hourly/.placeholder
755 0:0     root:root     0      etc/cron.monthly
644 0:0     root:root     102    etc/cron.monthly/.placeholder
755 0:0     root:root     0      etc/cron.weekly
644 0:0     root:root     102    etc/cron.weekly/.placeholder
644 0:0     root:root     1042   etc/crontab
644 0:0     root:root     732    etc/group
644 0:0     root:root     10     etc/hostname
644 0:0     root:root     109    etc/hosts
777 0:0     root:root     2172   etc/ntp.conf
644 0:0     root:root     1505   etc/passwd
640 0:42    root:shadow   1121   etc/shadow
600 0:0     root:root     7160   etc/snmp/snmpd.conf
644 0:0     root:root     1580   etc/ssh/ssh_config
644 0:0     root:root     3250   etc/ssh/sshd_config
755 0:0     root:root     0      var/etu
755 0:0     root:root     1681   var/lib/probe/etc/iecmnd/iecmnd_config.json
755 0:0     root:root     0      var/lib/probe/etc/network
644 0:0     root:root     228    var/lib/probe/etc/network/interfaces
755 0:0     root:root     0      var/lib/probe/etc/network/interfaces.d
777 0:0     root:root     146    var/lib/probe/etc/network/interfaces.d/gbe
644 0:0     root:root     110    var/lib/probe/etc/network/interfaces.d/usb
600 0:0     root:root     2607   METADATA.ini

Total elements count: 35

.....
Files checksums:
```

```

49674d9b1bd433cdb58f30497f6c441dc823de549160eef5c1acec4c20786810 : /etc/cron.d/.placeholder
49674d9b1bd433cdb58f30497f6c441dc823de549160eef5c1acec4c20786810 : /etc/cron.daily/.placeholder
a069113148fc81e3c72f76249f77bff234d933d1be88909a896d672ea81bfaed : /etc/cron.daily/apt-compat
a68002bf1dc9f42a150087b00437448a46f7cae6755ecddca70a6d3c9d20a14b : /etc/cron.daily/bsdmainutils
172557932b09e6b22519653ce4bb4397767c96c270ac9271c8e92df9bba009bf : /etc/cron.daily/dpkg
e314ef938fc5740139aaa117d6669971b9c99cfea7a2e5346f31b067819dd156 : /etc/cron.daily/lighttpd
12b36ff7068d3932f428e6eba07cbc9b9b2f7f7d37756d86ce13ddfcc6cd875f : /etc/cron.daily/logrotate
51e13dde67bc066de7cae41c409594f0885266578c3f753f49c857f33a5f0ab : /etc/cron.daily/ntp
777a9112ee093d8683645b031eb6cfeb9ce77274f40575c48ff2054ea24114d1 : /etc/cron.daily/passwd
49674d9b1bd433cdb58f30497f6c441dc823de549160eef5c1acec4c20786810 : /etc/cron.hourly/.placeholder
49674d9b1bd433cdb58f30497f6c441dc823de549160eef5c1acec4c20786810 : /etc/cron.monthly/.placeholder
49674d9b1bd433cdb58f30497f6c441dc823de549160eef5c1acec4c20786810 : /etc/cron.weekly/.placeholder
c5c9241274bee45e7e60d8b247a15bd5f69bf821b813215194156fd60fc4afc9 : /etc/crontab
1c6d04ba96129572832050878cbb834e4ccb422846d60dc57f32306253ada4d7 : /etc/group
f23f94fe28c3d96ccbabe90f5fec3c17c22e9c446b0dcc2cd7c93e1fc73d3ebd : /etc/hostname
9adb29b748caaeec3f9086e01443e5520e7d9c00ea6a5bacd73249c4c504628a : /etc/hosts
ec79886fd6a6171ac5ec180a0b2eea7ba500cce2e0ec3cedce18f7fb1baecaf7 : /etc/ntp.conf
d22cb8a99ef4020dae4b99144e0ef681e370a34515e287cbbf0d728a8d527a72 : /etc/passwd
5c86563fa85509d542029e125f21c126a2683233dbd9020a79124a5eb5a0be9c : /etc/shadow
264df406adf7df259505c5e229a58d1ce2494505b79d0401492954bf812386e7 : /etc/snmp/snmpd.conf
a39fbc57dc2ef8a473f078d1f6a35f725809400df67070b8852e8ed725047df2 : /etc/ssh/ssh_config
29eb564dc9c1b4d06cab04558838ec5aea4f77a790ae2ea398d1aa2b5779c1c8 : /etc/ssh/sshd_config
3f502f964ec9989273f8b6556ad82a0675796d2ddce1b2fd68665e5002e8a21e :
/var/lib/probe/etc/iecmnd/iecmnd_config.json
ad64a9914887209095aade17a66d90da7e0f83e8623c25a96079fc91ca8829f1 :
/var/lib/probe/etc/network/interfaces
b03bf5bc63c78490f5f2271a9bd9436ea6b093e51df27556aaed0ea378d15314 :
/var/lib/probe/etc/network/interfaces.d/gbe
d88d3fea42980def114bba73ead26251ea6d3fb8c26ce03b075f4970e0fa4953 :
/var/lib/probe/etc/network/interfaces.d/usb
.....

```

12. Factory reset

Note. After performing this procedure, all settings and changes in the file system (for example, changes in configuration files) and all installed packages will be lost.

To reset Smart SFP to the factory settings:

1. Eject Smart SFP from the host equipment (see Section 6.1).
2. Set the DIP switch to the “Recovery mode” position (see Table 3.10).
3. Insert Smart SFP into the host device. The factory reset procedure will start automatically. It takes about 15 minutes.
4. Eject Smart SFP and set the DIP switch to the “Regular mode” position (see Table 3.10).

13. Software upgrade

13.1. Software upgrade of Smart SFP with installed version 1.0.101 or higher

This software upgrade instruction is intended only for software versions 1.0.101 and higher. If the software version number is lower than 1.0.101, then you have to upgrade Smart SFP to version 1.0.104 according to chapter 13.2.

13.1.1. Necessary files

To perform a remote software update, you need to contact technical support to obtain an archive with the necessary files:

- a. update.tar.gz
- b. u-boot-env.bin
- c. updater.sh
- d. complete_update.sh
- e. config_file

13.1.2. Keeping settings during software upgrade

It is possible to perform software upgrade and keep necessary settings and files. Files and directories that should be kept on Smart SFP without removing or setting them to default must be listed in file config_file. By default, config_file has following list of files and directories in it:

```
/etc/passwd
/etc/shadow
/etc/group
/etc/network/interfaces
/etc/network/interfaces.d/
/etc/ssh/
/var/lib/ntp/ntp.drift
/var/et/
/etc/cron.d/
/etc/cron.daily/
/etc/cron.hourly/
/etc/cron.monthly/
/etc/cron.weekly/
/etc/crontab
```

By using default config_file the following settings will be kept after the software upgrade procedure: users (including passwords, home directory, access rights, interpreter after login), network settings, SSH keys and settings, crontab schedule and all of the setting made in command line in run-klish mode.

If you don't want some of these settings to be kept, you have to remove corresponding lines from config_file. For example, if you don't want to keep users, you have to remove lines:

```
/etc/passwd
/etc/shadow
/etc/group
```

In this case default users will be created after software upgrade.

If you want to keep any other files or directories, you should add them to config_file:

```
/etc/passwd
/etc/shadow
/etc/group
/etc/network/interfaces
/etc/network/interfaces.d/
/etc/ssh/
/var/lib/ntpd/ntp.drift
/var/etu/
/etc/cron.d/
/etc/cron.daily/
/etc/cron.hourly/
/etc/cron.monthly/
/etc/cron.weekly/
/etc/crontab
/user_directory/
/var/user_directory/
/user_file
```

For correct software upgrade you must not add directories / and /var in config_file. If you want to keep user files in these directories, you should add paths to exact objects you want to keep, for example: /user_directory/, /var/user_directory/, /user_file.

13.1.3. Upgrade procedure

1. Extract files from the archive received from technical support.
2. Modify the config_file according to chapter 14.1.3 if it is needed.
3. Upload the resulting set of files to Smart SFP into /home/user directory using SFTP or SCP protocols.
You should use “user” account for this.
4. Connect to Smart SFP via SSH and “user” account.
5. Switch to “root” using the command su.
6. Change the present working directory to /home/user, if it is not it already:

```
root@smart-sfp:~# cd /home/user
root@smart-sfp:/home/user#
```

7. Run commands:

```
root@smart-sfp:/home/user# chmod +x updater.sh
root@smart-sfp:/home/user# chmod +x complete_update.sh
```

8. Start the upgrade procedure:

```
root@smart-sfp:/home/user# ./updater.sh -a ./update.tar.gz -c ./config_file -e ./u-boot-
env.bin -d 1
```

To perform software upgrade and reset all settings and files to the factory default state, you should start the software upgrade procedure without config_file in the command:

```
root@smart-sfp:/home/user# ./updater.sh -a ./update.tar.gz -e ./u-boot-env.bin -d 1
```

In this case you have to confirm software upgrade with factory reset by pressing Y, then Enter.

```
You do not provide config file (-c). Are you sure?
Print yes/y or no/n
y
```

All settings, including network settings will be reset. Use default IP addresses to connect to Smart SFP after software upgrade.

9. The software upgrade procedure will take around 10 minutes. The message will appear after it finishes:

```
Please reboot system and then run complete_update.sh script
```

10. After that reboot Smart SFP:

```
root@smart-sfp:/home/user# reboot
```

Important notice: Next steps must be done fast. After reboot the 86 second watchdog timer will be started. If you will not run `complete_update.sh` script within this time, rollback procedure will start: Smart SFP will reboot again, and will return to the state before upgrade. This rollback function provides reliability in cases of upgrade failure, lost remote access due to accidental factory reset, or configuration incompatibility with new software version.

11. Connect again to Smart SFP via SSH and “user” account.

12. Switch to “root” using the command `su`.

13. Change the present working directory to `/home/user`, if it is not it already:

```
root@smart-sfp:~# cd /home/user
root@smart-sfp:/home/user#
```

14. Run command to finish the upgrade procedure:

```
root@smart-sfp:/home/user# ./complete_update.sh 1
Update completed
```

13.2. Upgrade procedure from software versions below 1.0.101 to version 1.0.104

This instruction describes the upgrade procedure only from software versions below 1.0.101 to version 1.0.104.

Note: After software upgrade from versions below 1.0.101 to version 1.0.104 only network settings will be kept on device. Any other settings or user files will be removed.

13.2.1. Necessary files

To perform a remote software update, you need to contact technical support to obtain an archive with the necessary files:

- a. `.bin.zip` archive containing software image
- b. Archive with service software files

Additionally, you need to install an SFTP server on your computer. If you are using a Windows computer, you can use CoreFTP Server: <http://www.coreftp.com/server/download/mini-sftp-server.x64.exe>

13.2.2. Upgrade procedure

1. Extract service software files from the archive.
2. Upload service software files to Smart SFP into `/home/user` directory using SFTP or SCP protocols. You should use “user” account for this.
3. Connect to Smart SFP via SSH and “user” account.
4. Switch to “root” using command:

```
user@smart-sfp:~# su -
Password:
root@smart-sfp:~#
```

5. Move service software files to the home directory of root:

```
root@smart-sfp:~# mv /home/user/initramfs.gz.u-boot /home/user/uImage .
root@smart-sfp:~# mv /home/user/uboot.env /home/user/prepare.sh /home/user/stcmtk-
grif.dtb .
```

6. Run commands:

```
root@smart-sfp:~# /sbin/reg16 123 0x0
root@smart-sfp:~# /sbin/reg16 128 0x0
root@smart-sfp:~# chmod +x prepare.sh
```

7. Run prepare.sh script:

```
root@smart-sfp:~# ./prepare.sh
```

After the successful script run, the following message will appear in the console:

```
Check nesessary files... ok
Copy new Service RAMFS... ok
Copy network settings... ok
Configure bootloader... ok
Done!
```

8. Reboot Smart SFP after that:

```
root@smart-sfp:~# /sbin/reboot
```

9. After the reboot, connect to Smart SFP via SSH, but this time using “root” account.

10. Start SFTP server on your computer.

If you're using CoreFTP Server, you should setup username and password for connections to the SFTP server, and also you should setup Root path: it should be path to the directory containing .bin.zip archive with software image in it.

11. Mount the server directory containing .bin.zip archive with software image in it to the Smart SFP directory by running this command:

```
root@localhost:~# sshfs username@192.168.2.105:/ /mnt/
The authenticity of host '192.168.2.105 (192.168.2.105)' can't be established.
RSA key fingerprint is SHA256:RTroFhhdIdAgSC7K/QyAq8Aum5aJqMAc1NE7p5sWjk.
Are you sure you want to continue connecting (yes/no)? yes
Password:
root@localhost:~#
```

Write IP address of your SFTP server instead of 192.168.2.105, and username that you setup on SFTP server instead of “username”. If connection confirmation request appears, then type yes and press Enter. Then type in the password and press Enter.

If following error occurs:

```
fuse: mountpoint is not empty
fuse: if you are sure this is safe, use the 'nonempty' mount option
```

Use this command:

```
root@localhost:~# sshfs -o nonempty username@192.168.2.105:/ /mnt/
Password:
root@localhost:~#
```

12. Start the software upgrade procedure:


```
root@localhost:~# unzip -p /mnt/software.bin.zip *.bin | dd of=/dev/mmcblk0 bs=4M
oflag=dsync status=progress
3848536064 bytes (3.8 GB, 3.6 GiB) copied, 856 s, 4.5 MB/s
0+32195 records in
0+32195 records out
3850371072 bytes (3.9 GB, 3.6 GiB) copied, 856.885 s, 4.5 MB/s
root@localhost:~#
```

Write the name of .bin.zip archive with software image in it instead of software.bin.zip.

13. Run the command:

```
root@localhost:~# fusermount -u /mnt
```

14. If you need to keep the network settings to have remote access to Smart SFP after upgrade, enter the following commands:

```
root@localhost:~# mount /dev/mmcblk0p2 /mnt/
root@localhost:~# mount /dev/mmcblk0p5 /mnt/var/
root@localhost:~# cp /etc/network/interfaces.d/gbe
/mnt/var/lib/probe/etc/network/interfaces.d/gbe
root@localhost:~# umount /mnt/var/
root@localhost:~# umount /mnt/
```

If you want to have Smart SFP with default configuration, you should skip this step.

15. Reboot the Smart SFP:

```
root@localhost:~# /sbin/reboot
```

14. EEPROM memory reprogramming

Reprogramming the A0h (0x50) page of the Smart SFP EEPROM is possible in two ways:

1. Using typical EEPROM programmers for SFP modules.
2. Without using an EEPROM programmer

Reprogramming the A2h (0x51) page of the Smart SFP EEPROM is possible without using an EEPROM programmer.

14.1. Reprogramming the A0h (0x50) EEPROM page using a programmer

By default, the A0h (0x50) EEPROM page in the Smart SFP is write-protected. To remove this protection, you need:

1. Connect to the Smart SFP as admin and then switch to the root account:

```
admin@smart-sfp:~$ su
```

2. Remount the file system for writing:

```
mount -o remount,rw /
```

3. Open the /etc/rc.local file in the “nano” text editor:

```
nano /etc/rc.local
```

Add lines:

```
#!/bin/sh -e
echo 0 > /sys/devices/platform/cpu-gpio/host-eeeprom-write-protect/value
exit 0
```

Save changes and close the file by pressing Ctrl+X, then Y, then Enter.

4. Run command:

```
chmod +x /etc/rc.local
```

5. Reboot Smart SFP:

```
/sbin/reboot
```

After starting Smart SFP, EEPROM recording will not be available immediately, but by the time the Linux OS is loaded (10-20 sec).

6. Connect to the Smart SFP and check that the “host-eeeprom-write-protect” value is 0:

```
cat /sys/devices/platform/cpu-gpio/host-eeeprom-write-protect/value
```

7. Now, you can unplug the Smart SFP from network equipment, plug it into regular SFP programmer, and start reprogramming as a regular SFP module.

At the end of programming, write protection of the EEPROM memory should be returned:

1. Connect to the Smart SFP as admin and then switch to the root account:

```
admin@smart-sfp:~$ su
```

2. Remount the file system for writing:

```
mount -o rw,remount /
```

3. Open the /etc/rc.local file in the “nano” text editor:

```
nano /etc/rc.local
```

Delete the line:

```
echo 0 > /sys/devices/platform/cpu-gpio/host-eeeprom-write-protect/value
```

Save changes and close the file by pressing Ctrl+X, then Y, then Enter.

4. Reboot Smart SFP:

```
/sbin/reboot
```

14.2. Reprogramming the A0h (0x50) EEPROM page without using a programmer

To change the A0h (0x50) EEPROM page without a programmer, you should:

1. Copy the image of the A0h (0x50) EEPROM page for installation to the /home/admin directory. To do this, you can use the “scp” utility or the WinScp program with graphical interface:

```
scp /Users/user/Downloads/image.img admin@192.168.1.1:
```

2. Connect to the Smart SFP via SSH under the admin account and get root privileges:

```
admin@smart-sfp:~$ su -
```

3. Remount the file system with write access:

```
mount -o rw,remount /
```

4. Run the commands:

```
echo 0 > /sys/devices/platform/cpu-gpio/host-eeeprom-write-protect/value
echo 24c02 0x50 > /sys/bus/i2c/devices/i2c-0/new_device
```

5. Change the directory to /home/admin/ where the image.img file is located:

```
cd /home/admin/
```

6. Back up the original contents of the A0h (0x50) EEPROM page:

```
/home/admin# cat /sys/bus/i2c/devices/0-0050/eeeprom > backup.img
```

7. Write the contents of the image to the page A0h (0x50) of the module's EEPROM:

```
/home/admin# cat image.img > /sys/bus/i2c/devices/0-0050/eeeprom
```

8. Check if the content of the EEPROM page A0h (0x50) have changed:

```
sfp-eeeprom-parser -s /sys/bus/i2c/devices/0-0050/eeeprom
```

9. Reboot Smart SFP:

```
/sbin/reboot
```

10. If the module is not detected after rebooting and no connection is established on port B, remove the module from the switch and reinstall it after 10 seconds.

14.3. Reprogramming the A2h (0x51) EEPROM page

To change the A2h (0x51) EEPROM page, you should:

1. Connect to the Smart SFP via SSH under the admin account and get root privileges:

```
admin@smart-sfp:~$ su -
```

2. Remount the file system for writing:

```
mount -o remount,rw /
```

3. Change the directory to /home/admin/:

```
cd /home/admin/
```

4. Back up the original contents of the A2h (0x51) EEPROM page:

```
/home/admin# cp /var/lib/probe/ddmi/probe-ddmi-config.ini probe-ddmi-config.ini.bak
```

5. Open the /var/lib/probe/ddmi/probe-ddmi-config.ini file in the “nano” text editor:

```
nano /var/lib/probe/ddmi/probe-ddmi-config.ini
```

The file contains the required byte values in the format “byte number = hexadecimal value”. For example:

```
1=0x55  
2=0x00  
3=0x5A  
.....
```

When editing a file, you need to save its structure and change only the byte values.

Byte 95 is responsible for the A2h page checksum (0x51). In order for the checksum to be calculated automatically, store the “CRC” value for byte 95 in this file and do not change it:

```
.....  
95=CRC  
.....
```

After making changes, you need to save the changes and close the file by pressing Ctrl+X, then Y, then Enter.

6. To apply the changes, reboot the module:

```
/sbin/reboot
```

7. If after rebooting the module the network equipment displays old information about the module, then you should remove the module from the switch and reinstall it after 10 seconds.

15. Maintenance

A fully assembled and well operated device does not need any maintenance.

16. Examples of typical settings

This section provides examples of typical settings for Smart SFP service functions. Available settings may vary depending on the Smart SFP modification and installed feature set.

16.1. Loopback

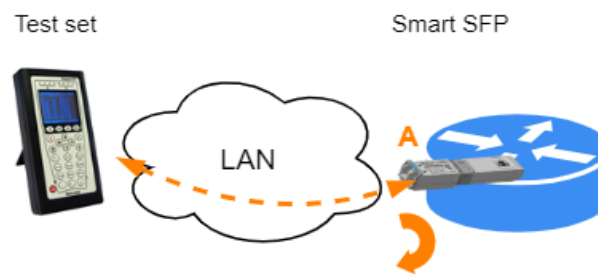


Figure 16.1. An example of using the loopback function

This example describes a loopback configuration on a Smart SFP line interface (port A) on layer 3 of the OSI model with a duration of 1 minute.

Note. The loopback function only works for packets with a destination MAC address in the Ethernet header equal to the MAC address of the port on which loopback is enabled. The other packets are transmitted via Smart SFP without any changes.

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klish
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# loopback config profile0
```

2. Using the “topology” command, bind port A to the logical interface 0, specifying the active status of this topology:

```
smart-sfp(admin)(config-loopback[profile0])# topology iface 0 port a status on
ok
```

3. Set the layer of the OSI model at which the loopback will work:

```
smart-sfp(admin)(config-loopback[profile0])# type layer3
ok
```

4. Set the loopback duration, for example 1 minute:

```
smart-sfp(admin)(config-loopback[profile0])# duration time 00:01:00:00
ok
```

5. Exit the profile configuration mode and enable loopback:

```
smart-sfp(admin)(config-loopback[profile0])# up
smart-sfp(admin)(config)# loopback start profile0
ok
```

6. After a minute, the loopback will be disabled; you can also stop it with the command:

```
smart-sfp(admin)(config)# loopback stop profile0
ok
```

16.2. Loopback by filter

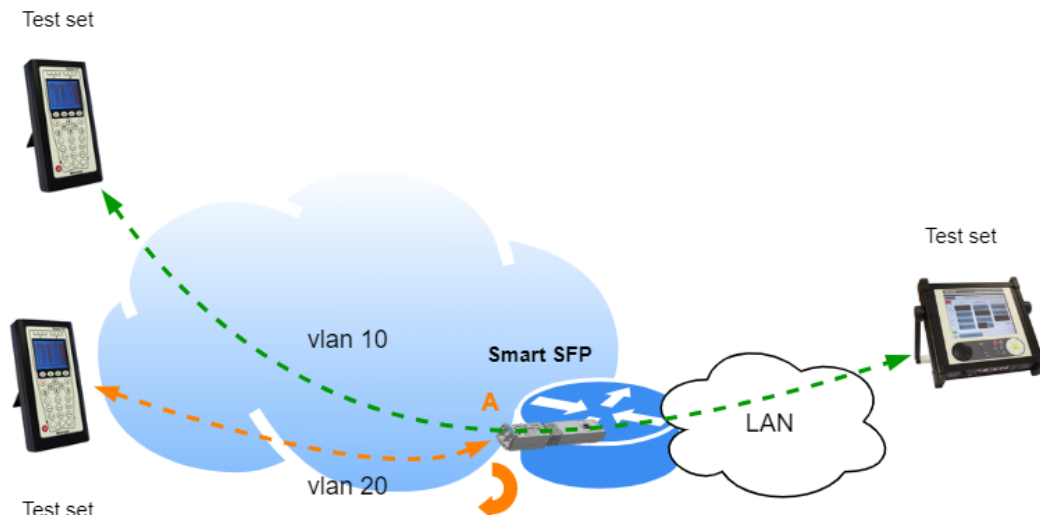


Figure 16.2. An example of using the loopback by the filter function

This example describes a loopback by filter configuration (filter function) on a Smart SFP line interface (port A). The loopback function will only work for packets with VLAN ID = 20.

Note. The loopback function only works for packets with a destination MAC address in the Ethernet header equal to the MAC address of the port on which loopback is enabled. The other packets are transmitted via Smart SFP without any changes.

1. Run the special shell and switch to the profile configuration mode:

```
admins@smart-sfp:~$ run-klsh
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# filter config profile0
```

2. Using the “topology” command, bind port A to the logical interface 0, specifying the active status of this topology:

```
smart-sfp(admin)( config-filter[profile0])# topology iface 0 port a status on
ok
```

3. Configure filter #1 to filter packets with VLAN ID = 20:

```
smart-sfp(admin)( config-filter[profile0])# sel num 1 vlan 20
ok
```

4. Specify the “loopback” action for packets matching this filter:

```
smart-sfp(admin)( config-filter[profile0])# sel num 1 route loopback
ok
```

5. Exit the profile configuration mode and enable loopback:

```
smart-sfp(admin)(config-filter[profile0])# up
smart-sfp(admin)(config)# filter start profile0
ok
```

To enable loopback by filters function, you should also enable loopback with the correct port, the layer of the OSI model, and duration after starting the filter function. For setting example, see section 16.1.

16.3. TWAMP Light (reflector)

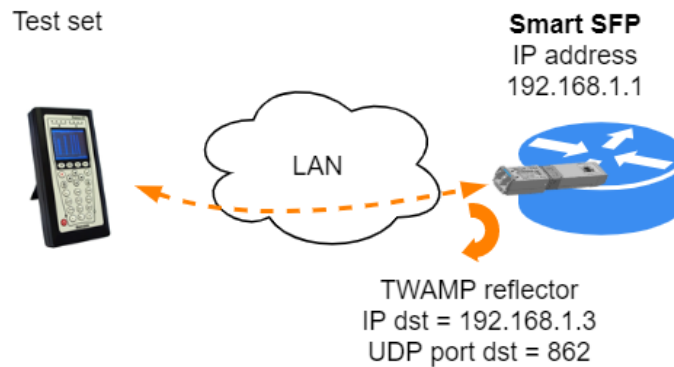


Figure 16.3. An example of using the TWAMP reflector function

This example describes the configuration of the TWAMP reflector on the line interface (port A). The function only works for packets with a destination IP address 192.168.1.3 and destination UDP port number 862. The other packets will be transmitted via Smart SFP without any changes. The filter function is used to select TWAMP test packets.

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klsh
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# filter config profile0
```

2. Using the “topology” command, bind port A to the logical interface 0, specifying the active status of this topology:

```
smart-sfp(admin)( config-filter[profile0])# topology iface 0 port a status on
ok
```

3. Configure filter #2 to filter packets destination IP address 192.168.1.3 and destination UDP port number 862:

```
smart-sfp(admin)(config-filter[profile0])# sel num 2 ip dst 192.168.1.3
ok
smart-sfp(admin)(config-filter[profile0])# sel num 2 port dst 862
ok
```

4. Specify the “twamp” action for packets matching this filter:

```
smart-sfp(admin)( config-filter[profile0])# sel num 2 route twamp
ok
```

5. Exit the profile configuration mode and enable loopback:

```
smart-sfp(admin)(config-filter[profile0])# up
smart-sfp(admin)(config)# filter start profile0
ok
```

To enable the TWAMP reflector function, you should also enable loopback with the correct port, the layer of the OSI model, and duration after starting the filter function. For setting example, see section 16.1.

16.4. Traffic capturing and mirroring

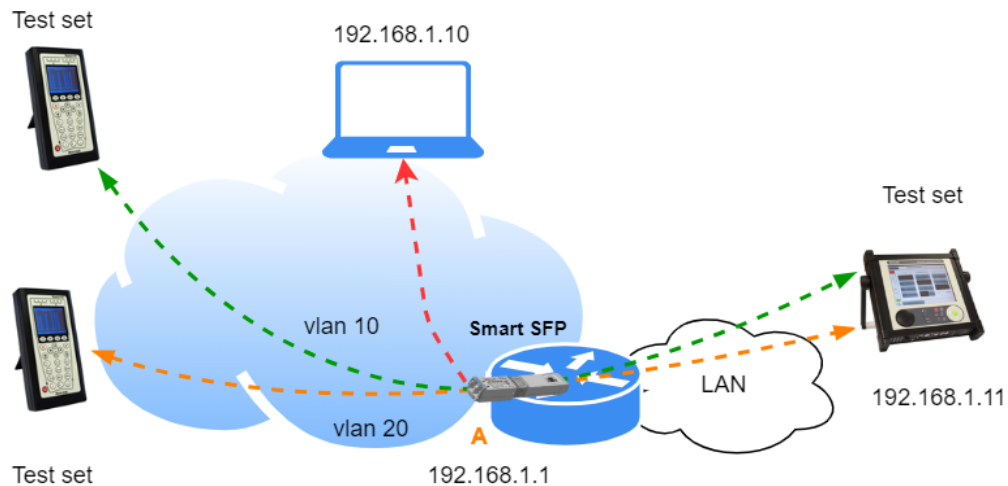


Figure 16.4. An example of using traffic capturing and mirroring function

This example describes the packet capturing and mirroring function configuration on the line interface (port A) for packets with a destination IP address 192.168.1.11 and VLAN ID = 20. The captured packets are sent to the device with the IP address 192.168.1.10.

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klish
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# erspan config profile0
```

2. Use the topology command to configure packet capture on port A:

```
smart-sfp(admin)(config-erspan[profile0])# topology iface capture1 status off
ok
smart-sfp(admin)(config-erspan[profile0])# topology iface capture0 port a
ok
smart-sfp(admin)(config-erspan[profile0])# topology iface capture0 status on
ok
```

3. Use the “topology” command to configure sending captured packets via port A:

```
smart-sfp(admin)(config-erspan[profile0])# topology iface erspan0 port a
ok
smart-sfp(admin)(config-erspan[profile0])# topology iface erspan0 status on
ok
```

4. Configure filter #0 to capture only receiving (rx) on the line interface packets with VLAN ID = 20 and destination IP address 192.168.1.11:

```
smart-sfp(admin)(config-erspan[profile0])# filter num 0 erspan 0
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 capture 0
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 direction rx
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 protocol mode off
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 ip src mode off
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 ip dst mode on
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 ip dst 192.168.1.11
ok
```



```

smart-sfp(admin)(config-erspan[profile0])# filter num 0 vlan id mode on
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 vlan id 20
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 port src mode off
ok
smart-sfp(admin)(config-erspan[profile0])# filter num 0 port dst mode off
ok

```

5. Specify the parameters of the ERSPAN packet header, in which the captured traffic will be encapsulated, to be sent to a remote device with the IP address 192.168.1.10:

```

smart-sfp(root)(config-erspan[profile0])# erspan 0 session_id 1
ok
smart-sfp(root)(config-erspan[profile0])# erspan 0 hw_id 1
ok
smart-sfp(root)(config-erspan[profile0])# erspan 0 mtu 128
ok
Vsmart-sfp(root)(config-erspan[profile0])# erspan 0 header src ip 192.168.1.1
ok
Vsmart-sfp(root)(config-erspan[profile0])# erspan 0 header dst ip 192.168.1.10
ok
smart-sfp(root)(config-erspan[profile0])# erspan 0 header dst mac auto
ok

```

6. To start the traffic capturing and mirroring function, exit the profile configuration mode and start the traffic capturing and mirroring service using the command:

```

smart-sfp(admin)(config-erspan[profile0])# up
smart-sfp(admin)(config)# erspan start profile0
ok

```

16.5. Loaned IP

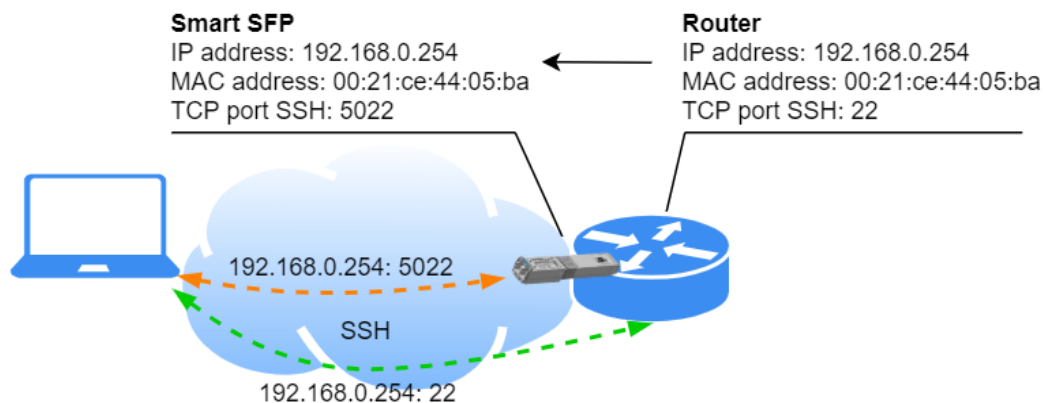


Figure 16.5. An example of using loaned IP function

This example describes the configuration of the loaned IP address function, which allows you to use the router's IP address for Smart SFP management, but using a different TCP port.

The example uses the following settings:

```

Loaned IP address: 192.168.0.254
Loaned MAC address: 00:21:ce:44:05:ba
TCP port for SSH service: 5022

```

Attention! It is important to follow these steps to ensure that the remote connection to the Smart SFP is not lost.

16.5.1. Configuring loaned IP settings

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klish
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# loaned-ip config profile0
```

2. Specify the IP address loaned from the router:

```
smart-sfp(admin)( config-loaned-ip[profile0])# ip 192.168.0.254
ok
```

3. Use the “range” command to configure the TCP/UDP port range, in this example from 5001 to 5022, when access to which, Smart SFP will intercept incoming traffic:

```
smart-sfp(admin)( config-loaned-ip[profile0])# range 5001 5022
ok
```

4. Specify that the loaned IP address will be bound to the line interface (port A):

```
smart-sfp(admin)( config-loaned-ip[profile0])# topology iface ext port a status on
ok
```

5. Exit to Linux shell:

```
smart-sfp(admin)(config-loaned-ip[profile0])# up
smart-sfp(admin)(config)# up
smart-sfp(admin)# up
admin@smart-sfp:~$
```

6. Switch to the root account:

```
admin@smart-sfp:~$ su
```

16.5.2. Configuring network and system parameters and applying all changes

1. Remount file system with write permissions:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

2. Specify the IP address and MAC address loaned from the router in the gbe0 network interface settings (port A) by editing the /etc/network/interfaces.d/gbe file as follows, leaving the gbe1 interface settings unchanged:

```
root@smart-sfp:~# nano /etc/network/interfaces.d/gbe
#SFP Port
auto gbe0
iface gbe0 inet static
pre-up ifconfig gbe0 hw ether 00:21:ce:44:05:ba
address 192.168.0.254
netmask 255.255.255.0

#Host port
auto gbe1
iface gbe1 inet manual
        address 192.168.2.1
        netmask 255.255.255.0
```

3. Save changes and close the file by pressing Ctrl+X, then Y.
4. Create a file (script) that will allow you to reassign the TCP port for managing Smart SFP over SSH and apply the previously made settings of the network parameters and the loaned IP:

```
nano loanedip_start.sh
```

5. Add the following lines to the file:

```
#!/bin/bash

if [ -z "$1" ]; then
    echo "Error! New TCP-port for SSH service should be specified as a script parameter,
i.e. loanedip_start.sh 5022, but it is empty. Please, specify it."
    echo "Terminating..."
    exit -1
fi

#SSH service TCP port change
echo "--- SSH service TCP port changing to $1 and restarting..."
sed -i "/Port /c\Port $1" /etc/ssh/sshd_config
/sbin/service ssh restart
sleep 1

#Network restart
echo "--- Network interfaces restarting..."
/sbin/ifdown gbe0 && /sbin/ifup gbe0
sleep 1

#loaned-ip service start
echo "--- Loaned IP service starting..."
ubus call loanedip.profile0 start > /dev/null 2>&1
sleep 1

echo "--- Remount filesystem to RO mode..."
mount -o ro,remount /

echo "--- Done!"
```

6. Save changes and close the file by pressing Ctrl+X, then Y.

7. Make the file executable:

```
chmod +x loanedip_start.sh
```

8. Run the script, specifying the new TCP port number for Smart SFP management over SSH as a parameter. The TCP port must be in the port range specified earlier when configuring the loaned IP address with the “range” command:

```
./loanedip_start.sh 5022
```

Attention! After running the loanedip_start.sh script, the SSH connection will be lost. You should reconnect to the Smart SFP over SSH using the TCP port entered when running the script.

16.6. Time synchronization from NTP server

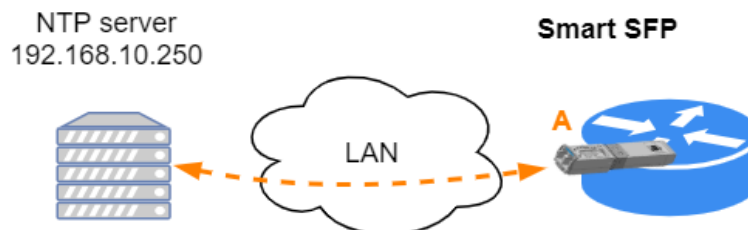


Figure 16.6. An example of using the time synchronization function

Configure the synchronization of the Smart SFP internal clock from the NTP server. Synchronization will be done through the line interface (port A). To configure NTP synchronization correctly, you need to activate the “timesync” function only via run-klish. If you activate the “timesync” function and the “ntp” function (“service ntp start” command) at the same time, the “timesync” function will not work.

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klish
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# timesync config profile0
```

2. Configure endless synchronization with the NTP server:

```
smart-sfp(admin)( config-synctime[profile0])# duration endless
ok
```

3. Specify the synchronization protocol – NTP:

```
smart-sfp(admin)( config-synctime[profile0])# type ntp
ok
```

4. Configure slave mode:

```
smart-sfp(admin)( config-synctime[profile0])# mode slave
ok
```

5. Specify the IP address of the NTP server:

```
smart-sfp(admin)( config-synctime[profile0])# ntp count 1
ok
smart-sfp(admin)(config-synctime[profile0])# ntp host 0 192.168.10.250
ok
```

6. Specify that the synchronization will be performed via port A:

```
smart-sfp(admin)( config-synctime[profile0])# topology iface port a
ok
```

7. To start the synchronization function, exit the profile configuration mode and run the “timesync start profile0” command:

```
smart-sfp(admin)(config-synctime[profile0])# up
smart-sfp(admin)(config)# timesync start profile0
ok
```

16.7. Time synchronization from PTP server

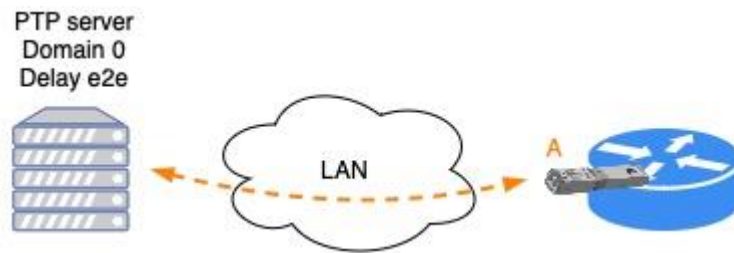


Figure 16.7. An example of using the time synchronization function

By default, non-transit multicast traffic is blocked on the Smart SFP. For correct time synchronization from PTP server it is necessary to enable the “cpu-multicast” function to transfer multicast traffic to the CPU. You can do this using the network service. The configuration is required for two ports (A and B):

```
smart-sfp(admin)(config)# network config porta

Enter `apply` to apply it
smart-sfp(admin)(config-network [porta])# cpu-multicast status on
ok
smart-sfp(admin)(config-network [porta])# up
smart-sfp(admin)(config)# network config portb

Enter `apply` to apply it
smart-sfp(admin)(config-network [portb])# cpu-multicast status on
ok
```

This is an example of setting up the synchronization of the Smart SFP internal clock from a PTP server. Synchronization will be performed through the line interface (port A).

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klsh
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# timesync config profile0
```

2. Configure endless synchronization with the PTP server:

```
smart-sfp(admin)(config-synctime[profile0])# duration endless
ok
```

3. Specify the synchronization protocol – PTP:

```
smart-sfp(admin)(config-synctime[profile0])# type ptp
ok
```

4. Configure slave mode:

```
smart-sfp(admin)( config-synctime[profile0])# mode slave
ok
```

5. Specify the domain number and delay determination mechanism for the PTP server:

```
smart-sfp(admin)(config-synctime[profile0])# ptp delay e2e
ok

smart-sfp(admin)(config-synctime[profile0])# ptp domain 0
ok
```

6. Specify that the synchronization will be performed via port A:

```
smart-sfp(admin)( config-synctime[profile0])# topology iface port a
ok
```

7. To start the synchronization function, exit the profile configuration mode and run the “timesync start profile0” command:

```
smart-sfp(admin)(config-synctime[profile0])# up
smart-sfp(admin)(config)# timesync start profile0
ok
```

16.8. Filter statistics

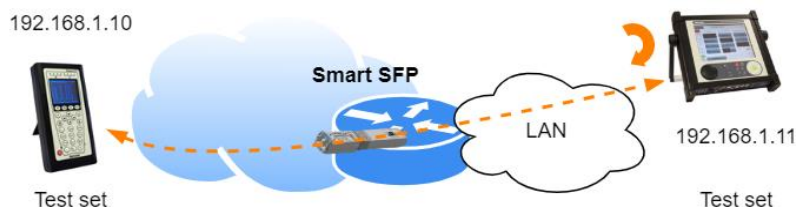


Figure 16.8. An example of using the filter statistics function

This example describes the configuration of the filter statistics function on the line interface (port A). The function only works for TCP packets with a source IP address 192.168.1.10. All packets will be transmitted via Smart SFP without any changes.

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klish
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# fstatistic config profile0
```

2. Configure filter #0 to gather statistics about TCP packets received on the line interface (port A) with source IP address 192.168.1.10:

```
smart-sfp(admin)( config- fstatistic[profile0])# filter num 0 protocol tcp
ok
smart-sfp(admin)( config- fstatistic[profile0])# filter num 0 protocol mode on
ok
smart-sfp(admin)( config- fstatistic[profile0])# filter num 0 ip src 192.168.1.10
ok
smart-sfp(admin)( config- fstatistic[profile0])# filter num 0 on iface a ip src mode on
ok
```

3. To start gathering filter statistics, you need to exit the profile configuration mode and run the “fstatistic start” command:

```
smart-sfp(admin)(config-fstatistic[profile0])# up
smart-sfp(admin)(config)# fstatistic start profile0
ok
```

16.9. Y.1564

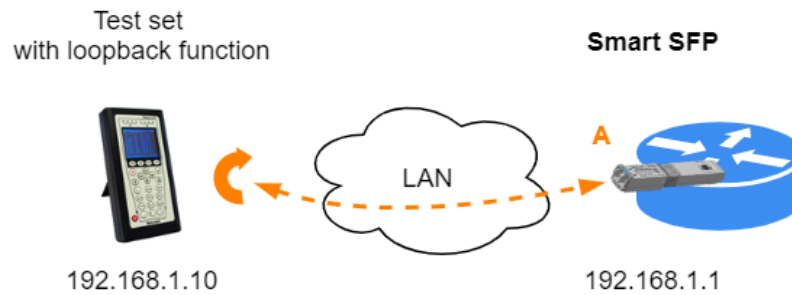


Figure 16.9. An example of using the Y.1564 function

This example describes the configuration of the Y.1564 function on the line interface (port A). Smart SFP performs the functions of a test traffic generator and analyzer. As a loopback at the remote device, additional equipment is used to swap the source and destination's MAC addresses and IP addresses in received packets (to perform the loopback function at the third layer of the OSI model).

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klish
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# y1564 config profile0
```

2. Configure generation and reception of test traffic on port A:

```
smart-sfp(admin)(config-y1564[profile0])# topology iface tx port a
ok
smart-sfp(admin)(config-y1564[profile0])# topology iface rx port a
ok
```

3. Set the number of steps for the CIR test:

```
smart-sfp(admin)( config- y1564[profile0])# step 4
ok
```

4. Enable the performance test and set its duration (30 sec):

```
smart-sfp(admin)(config-y1564[profile0])# perf on
ok
smart-sfp(admin)( config- y1564[profile0])# perf duration useconds 00:00:30:0000
ok
```

5. Set the number of services:

```
smart-sfp(admin)( config- y1564[profile0])# count 1
ok
```

6. Switch to service #0 configuration mode:

```
smart-sfp(admin)(config-y1564[profile0])# service 0
* Enter `show` to view the configuration
```

7. Set the test frame size for this service:

```
smart-sfp(admin)(config-y1564[service0])# frame size 1024
ok
```

8. Configure the test frames header. In this example, Smart SFP will automatically gets the MAC address of the remote device:

```
smart-sfp(admin)(config-y1564[service0])# header src ip 192.168.1.1
ok
```

```

smart-sfp(admin)(config-y1564[service0])# header src mac 00:21:ce:44:05:42
ok
smart-sfp(admin)(config-y1564[service0])# header dst ip 192.168.1.10
ok
smart-sfp(admin)(config-y1564[service0])# header dst mac auto
ok

```

9. Set the rate:

```

smart-sfp(admin)(config-y1564[service0])# rate cir 10 mbps layer3
ok
smart-sfp(admin)(config-y1564[service0])# rate eir 1 mbps layer3
ok
smart-sfp(admin)(config-y1564[service0])# rate tp 1 mbps layer3
ok
smart-sfp(admin)(config-y1564[service0])# up

```

10. Select the configuration tests to be performed:

```

smart-sfp(admin)(config-y1564[profile0])# cfg cir on
ok
smart-sfp(admin)(config-y1564[profile0])# cfg eir on
ok
smart-sfp(admin)(config-y1564[profile0])# cfg tp on
ok

```

11. Set the configuration test duration:

```

smart-sfp(admin)(config-y1564[profile0])# cfg duration useconds 00:00:30:0000
ok

```

12. To start the test, exit the profile configuration mode and start it using the “y1564 start profile0” command:

```

smart-sfp(admin)(config-y1564[profile0])# up
smart-sfp(admin)(config)# y1564 start profile0
ok

```

16.10. Timestamp insertion



Figure 16.10 An example of using timestamp insertion function

This example shows how to configure timestamp insertion function on the line interface (port A) of the Smart SFP. Packets with sequences will be generated by the device with IP address 192.168.1.10. The “data” field of the generated packets contains, for example, the following data: “AABBCCDD00112233FFEEDDCC99887766XXXXXXXXXXXXXXXXXX”, where:

- “AABBCCDD00112233” – the first template to be replaced
- “FFEEDDCC99887766” – the second template to be replaced
- “XXXXXXXXXXXXXXXXXX” – other data in the packet

Timestamps will be inserted into packets, transiting through Smart SFP, instead of templates. A support equipment is used as a loop on a remote device. This equipment is capable to swap MAC addresses, IP addresses of the sender and receiver, UDP ports of the sender and receiver in incoming packets, i.e. perform the loopback function at the fourth layer of the OSI model.

For correct timestamp value, it is necessary to configure the Smart SFP internal clock synchronization from PTP/NTP server in accordance with the instructions in section 16.6.

1. Run the special shell and switch to the profile configuration mode:

```
admin@smart-sfp:~$ run-klsh
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# tsins config profile0
```

2. Use the “tspoint” command to select port A to insert the timestamp. Smart SFP will process incoming (rx) transit traffic in the direction from port A to port B.

```
smart-sfp(admin)(config-tsins[profile0])# tspan iface 0 ingress
ok
```

Smart SFP will process outgoing (tx) transit traffic in the direction from port A to port B.

```
smart-sfp(admin)(config-tsins[profile0])# tspan iface 1 egress
ok
```

3. Configure endless timestamp insertion in transit traffic:

```
smart-sfp(admin)(config-tsins[profile0])# duration endless
ok
```

4. Set the OSI model layer for the timestamp insertion function:

```
smart-sfp(admin)(config-tsins[profile0])# level iface 0 udp
ok
smart-sfp(admin)(config-tsins[profile0])# level iface 1 udp
ok
```

5. Set the value of the sequences that will change to timestamps:

```
smart-sfp(admin)(config-tsins[profile0])# pattern iface 1 AABCCDD00112233
ok
smart-sfp(admin)(config-tsins[profile0])# pattern iface 0 FFEEDDCC99887766
ok
```

6. Enable directions for the timestamp insertion:

```
smart-sfp(admin)(config-tsins[profile0])# status iface 0 on
ok
smart-sfp(admin)(config-tsins[profile0])# status iface 1 on
ok
```

7. Exit the profile configuration mode and enable the timestamp insertion function:

```
smart-sfp(admin)(config-tsins[profile0])# up
smart-sfp(admin)(config)# tsins start profile0
ok
```

8. The timestamp insertion function can be stopped with the command:

```
smart-sfp(admin)(config)# tsins stop profile0
ok
```

When working with the timestamp insertion function, it is necessary to consider:

1. Sequences should not contain identical bytes.

- Sequences can only be specified in hexadecimal format. Packets containing sequences in other formats, for example, ASCII, cannot be processed correctly by the device.

16.11. Configuring VLAN subinterfaces

VLAN subinterface settings are specified in the file `/etc/network/interfaces.d/gbe`. The information about created VLANs added to the file `/etc/network/interfaces.d/gbe` is retained after reboot.

This example shows how to configure a VLAN subinterface with VLAN ID = 10, IP address 192.168.10.1 on the gbe0 interface and a VLAN subinterface with VLAN ID = 20, IP address 192.168.20.1 on the gbe1 interface.

- Connect to the device via SSH and specify the IP address of the gbe0 (port A) or gbe1 (port B) interface.
- Log in as admin and get root privileges:

```
admin@smart-sfp:~$ su
```

- Remount the Smart SFP file system with write permissions:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

- Open the file `/etc/network/interfaces.d/gbe` for editing:

```
root@smart-sfp:~# nano /etc/network/interfaces.d/gbe
```

To configure a VLAN subinterface with VLAN ID = 10, IP address 192.168.10.1 on the gbe0 interface, add the following lines to the end of the file:

```
#VLAN 10
auto gbe0.10
allow-hotplug gbe0.10
iface gbe0.10 inet static
    address 192.168.10.1
    netmask 255.255.255.0
```

To configure a VLAN subinterface with VLAN ID = 20, IP address 192.168.20.1 on the gbe1 interface, add the following lines to the end of the file:

```
#VLAN 20
auto gbe1.20
allow-hotplug gbe1.20
iface gbe1.20 inet static
    address 192.168.20.1
    netmask 255.255.255.0
```

The settings of the gbe0 and gbe1 interfaces do not need to be changed.

- To apply the settings, reboot the Smart SFP with the “reboot” command or enter the command below to apply the settings without rebooting:

```
root@smart-sfp:~# systemctl restart networking.service
```

16.12. Speed calculating at different levels

Below are the rules for recalculating the payload according to a specific layer (L1 Mbps, L2 Mbps, L3 Mbps, L4 Mbps, or fps):

- Physical layer (Mbps L1): the Ethernet frame size (including CRC), preamble, and interframe gap are taken into account. For example, for a 64-byte frame at the L1, the following would be taken into account: 64 bytes (Ethernet frame) + 8 bytes (preamble) + 12 bytes (interframe gap) = 84 bytes.
- Data link layer (Mbps L2): only the Ethernet frame size (including CRC) is taken into account. For example, for a 64-byte frame at the L2, the following would be taken into account: 64 bytes (Ethernet frame with CRC).
- Network layer (Mbps L3): the size of the Ethernet frame without CRC, Ethernet header, VLAN and MPLS tags is taken into account. For example, for a 64-byte frame at the L3, the following will be taken into account: 64 bytes (Ethernet frame) - 4 bytes (CRC) - 14 bytes (Ethernet header) = 46 bytes.
- Transport layer (Mbps L4): the size of the Ethernet frame without CRC, Ethernet and IP header, VLAN and MPLS labels is taken into account. For example, for a 64-byte frame at the L4, the following will be taken into account: 64 bytes (Ethernet frame) - 4 bytes (CRC) - 14 bytes (Ethernet header) - 20 bytes (IP header) = 26 bytes.

The payload is calculated using the following formulas:

1. At the L2 layer:

$$\frac{\text{max speed} \times \text{number of bytes at L2 layer}}{\text{number of bytes at L1 layer}}$$

For example, for a 64-byte frame:

$$\frac{1000 \times 64}{84} = 761,904 \text{ Mbps}$$

2. At the L3 layer:

$$\frac{\text{max speed} \times \text{number of bytes at L3 layer}}{\text{number of bytes at L1 layer}}$$

For example, for a 64-byte frame:

$$\frac{1000 \times 46}{84} = 547,619 \text{ Mbps}$$

3. At the L4 layer:

$$\frac{\text{max speed} \times \text{number of bytes at L4 layer}}{\text{number of bytes at L1 layer}}$$

For example, for a 64-byte frame:

$$\frac{1000 \times 26}{84} = 309,524 \text{ Mbps}$$

16.13. Configuring time synchronization from GNSS

This example shows how to configure GNSS time synchronization for the Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232 modules.

Before configuring time synchronization, you should connect a GNSS receiver/antenna to the module and make sure that there is a connection to the satellites:

1. Connect to the device via SSH.
2. Log in as admin and get root privileges:

```
admin@smart-sfp:~$ su
```

3. Make sure that data is being received from the receiver. To do this, run the command:

```
root@smart-sfp:/home/admin# minicom -D /dev/ttyPORT0 -b 9600
```

After executing this command, a text similar to this should be displayed:

```
Welcome to minicom 2.7.1

OPTIONS: I18n
Compiled on May 6 2018, 10:36:56.
Port /dev/ttyPORT0, 10:12:18

Press CTRL-A Z for help on special keys

$GNRMC,,V,,,,,,,,,N*4D
$GNVTG,,,,,,,,,N*2E
$GNGGA,,,,,0,00,99.99,,,,,*56
$GNGSA,A,1,,,,,,,,,99.99,99.99,99.99*2E
$GNGSA,A,1,,,,,,,,,99.99,99.99,99.99*2E
$GPGSV,1,1,00*79
$GLGSV,1,1,00*65
$GNGLL,,,,,V,N*7A
$GNRMC,,V,,,,,,,,,N*4D
$GNVTG,,,,,,,,,N*2E
$GNGGA,,,,,0,00,99.99,,,,,*56
$GNGSA,A,1,,,,,,,,,99.99,99.99,99.99*2E
$GNGSA,A,1,,,,,,,,,99.99,99.99,99.99*2E
```

If the text isn't displayed, check the receiver's connection. If the text still isn't displayed, please contact technical support.

If text output occurs, then exit minicom by pressing Ctrl+A, then X, then Enter.

4. Next, you need to check the connection to the satellites. To do this, run the gpsd daemon:

```
root@smart-sfp:/home/admin# gpsd -n -s 9600 /dev/ttyPORT0
```

5. Start the NMEA message monitor:

```
root@smart-sfp:/home/admin# cgps -s
```

The output of this command will display the current connection status to satellites. For the GNSS time synchronization to work, the Status should be set to 3D FIX.

To exit the NMEA message monitor, press Ctrl+C.

6. If it is necessary to disable the use of any systems, you need to run the command:

- a. To disable SBAS:

```
root@smart-sfp:/home/admin# ubxtool -d SBAS
```

b. To disable GPS

```
root@smart-sfp:/home/admin# ubxtool -d GPS
```

c. To disable GLONASS

```
root@smart-sfp:/home/admin# ubxtool -d GLONASS
```

If you need to disable several systems, you should specify -d option with the correct system name multiple times, for example:

```
root@smart-sfp:/home/admin# ubxtool -d GPS -d SBAS
```

After this, repeat step 5 to ensure that after making changes to the receiver settings, Status is set to 3D FIX.

7. Mount the file system for writing:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

8. Configure automatic application of receiver settings. To do this, open the /etc/rc.local file in the editor:

```
root@smart-sfp:/home/admin# nano /etc/rc.local
```

Add lines:

```
#!/bin/bash
gpsd -n -N -s 9600 /dev/ttyPORT0 &>/dev/null &
GPSDPID=$!
ubxtool -d SBAS
gpsctl --nmea
kill $GPSDPID
exit 0
```

Edit the line with the ubxtool command according to the settings selected in step 6. If there is no need to disable any of the systems, you can skip this line.

Save changes by pressing Ctrl+X, then Y, then Enter.

Run commands:

```
root@smart-sfp:/home/admin# chmod +x /etc/rc.local
root@smart-sfp:/home/admin# systemctl enable rc-local
```

To synchronize time with GNSS, you need to configure two services:

- synchronization of the hardware clock from GNSS
- synchronization of the system clock from the hardware clock

Leap-seconds.list file is required for the GNSS hardware clock synchronization service to work. This file is valid for a limited period and needs to be updated when it expires. The current leap-seconds.list file can be downloaded from the link: <https://hpiers.obspm.fr/iers/bul/bulc/ntp/leap-seconds.list>

1. The downloaded file must be uploaded using the SFTP or SCP protocols to the /home/admin directory of the device.
2. Connect to the device via SSH.
3. Log in as admin and get root privileges:

```
admin@smart-sfp:~$ su
```

4. Mount the file system for writing:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

5. Move the leap-seconds.list file:

```
root@smart-sfp:/home/admin# mv leap-seconds.list /etc/ts2phc/leap-seconds.list
```

6. Open GNSS hardware clock synchronization settings file in the “nano” text editor /etc/config/ts2phc.conf:

```
root@smart-sfp:/home/admin# nano /etc/config/ts2phc.conf
```

Add lines:

```
[global]
first_step_threshold 0.000000001
free_running 0
leapfile /etc/ts2phc/leap-seconds.list
logging_level 6
max_frequency 0
step_threshold 0.0000005
servo_num_offset_values 10
servo_offset_threshold 50
use_syslog 0
clock_servo linreg
ts2phc.tod_source nmea
ts2phc.nmea_serialport /dev/ttyPORT0
ts2phc.nmea_baudrate 9600
ts2phc.extts_polarity rising
```

Save changes by pressing Ctrl+X, then Y, then Enter.

7. Open GNSS hardware clock synchronization service file in the “nano” text editor /usr/lib/systemd/system/ts2phc.service:

```
root@smart-sfp:/home/admin# nano /usr/lib/systemd/system/ts2phc.service
```

Add lines:

```
[Unit]
Description=Start ts2phc to sync physical clock from GNSS
After=network.target rc-local.service

[Service]
ExecStart=/usr/sbin/ts2phc -c /dev/ptp0 -f /etc/config/ts2phc.conf -s nmea -mql 6
ExecStop=/bin/kill -s QUIT $MAINPID

[Install]
WantedBy=multi-user.target
```

Save changes by pressing Ctrl+X, then Y, then Enter.

8. Enable automatic start of the hardware clock synchronization service from GNSS:

```
root@smart-sfp:/home/admin# systemctl enable ts2phc
```

9. Configure synchronization of the system clock from the hardware clock. To do this, open the /usr/lib/systemd/system/phc2sys.service file in an editor:

```
root@smart-sfp:/home/admin# nano /usr/lib/systemd/system/phc2sys.service
```

Add lines:

```
[Unit]
Description=Start phc2sys to sync system clock from physical clock
After=ts2phc.service rc-local.service

[Service]
ExecStart=/usr/sbin/phc2sys -r -s /dev/ptp0 -O 0 -S 0.2
ExecStop=/bin/kill -s QUIT $MAINPID

[Install]
WantedBy=multi-user.target
```

Save changes by pressing Ctrl+X, then Y, then Enter.

10. Enable automatic start of the hardware clock synchronization service from GNSS:

```
root@smart-sfp:/home/admin# systemctl enable phc2sys
```

11. To apply the settings, reboot the device using the command:

```
reboot
```

16.14. PTP server configuring with synchronization from GNSS

This example shows how to configure a PTP server with time synchronization from GNSS for the Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232 modules. To use an NTP server with GNSS synchronization, the “timesync” should be disabled (see Section 10.2.14).

Before setting up, time synchronization from GNSS should be configured in accordance with section 16.13.

1. Connect to the device via SSH.
2. Log in as admin and get root privileges:

```
admin@smart-sfp:~$ su
```

3. Allow multicast packets receiving:

```
root@smart-sfp:/home/admin# run-klish
smart-sfp(admin)# configure terminal
smart-sfp(admin)(config)# nfilter config B
* Enter `apply` to apply it
smart-sfp(admin)(config-nfilter[B])# cpu-multicast status on
ok
smart-sfp(admin)(config-nfilter[B])# exit
root@smart-sfp:/home/admin#
```

4. Mount the file system for writing:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

5. Open the PTP server settings file /etc/config/ptp4l.conf in the editor:

```
root@smart-sfp:/home/admin# nano /etc/config/ptp4l.conf
```

Add lines:

```
[global]
domainNumber 0
timeSource 20
clockClass 6
serverOnly 1
time_stamping hardware
twoStepFlag 1

[gbe1]
delay_mechanism E2E
hybrid_e2e 1
network_transport UDPv4
logAnnounceInterval -1
logSyncInterval -2
logMinDelayReqInterval -2
```

Save changes by pressing Ctrl+X, then Y, then Enter.

6. Open the PTP server service file /usr/lib/systemd/system/ptp4l.service in the editor:

```
root@smart-sfp:/home/admin# nano /usr/lib/systemd/system/ptp4l.service
```

Add lines:

```
[Unit]
Description=Start ptp4l to enable PTP time sync
After=network.target ts2phc.service

[Service]
ExecStart=/usr/sbin/ptp4l -l 7 -m -f /etc/config/ptp4l.conf
ExecStop=/bin/kill -s QUIT $MAINPID

[Install]
WantedBy=multi-user.target
```

Save changes by pressing Ctrl+X, then Y, then Enter.

7. Enable automatic start of the PTP server:

```
root@smart-sfp:/home/admin# systemctl enable ptp4l
```

8. To apply the settings, reboot the device using the command:

```
reboot
```

16.15. NTP server configuring with synchronization from GNSS

This example shows how to configure an NTP server with time synchronization from GNSS for the Smart SFP 1G GNSS and Smart SFP 1G GNSS-RS232 modules. To use an NTP server with GNSS synchronization, the “timesync” should be disabled (see Section 10.2.14).

Before setting up, time synchronization from GNSS should be configured in accordance with section 16.13.

1. Connect to the device via SSH.
2. Log in as admin and get root privileges:

```
admin@smart-sfp:~$ su
```

3. Mount the file system for writing:

```
root@smart-sfp:/home/admin# mount -o remount,rw /
```

4. Open the NTP server settings file /etc/ntp.conf in the editor:

```
root@smart-sfp:/home/admin# nano /etc/ntp.conf
```

Add the # symbol before the lines:

```
#pool 0.debian.pool.ntp.org iburst
#pool 1.debian.pool.ntp.org iburst
#pool 2.debian.pool.ntp.org iburst
#pool 3.debian.pool.ntp.org iburst
```

After these lines, add the lines:

```
server 127.127.1.0 minpoll 4 maxpoll 4 prefer
fudge 127.127.1.0 stratum
```

Save changes by pressing Ctrl+X, then Y, then Enter.

5. Enable automatic start of the GNSS hardware clock synchronization service:

```
root@smart-sfp:/home/admin# systemctl enable ntp
```

6. To apply the settings, reboot the device using the command:

```
reboot
```


Appendix 1. Pins of the electrical connector of SFP module

Table 16.1. SFP pin description

Pin	Symbol	Name/Description
1	V _{EET}	Transmitter Ground (Common with Receiver Ground)
2	T _{FAULT}	Transmitter Fault.
3	T _{DIS}	Transmitter Disable. Laser output disabled on high or open.
4	MOD_DEF(2)	Module Definition 2. Data line for Serial ID.
5	MOD_DEF(1)	Module Definition 1. Clock line for Serial ID.
6	MOD_DEF(0)	Module Definition 0. Grounded within the module.
7	Rate Select	No connection required
8	LOS	Loss of Signal indication. Logic 0 indicates normal operation.
9	V _{EER}	Receiver Ground (Common with Transmitter Ground)
10	V _{EER}	Receiver Ground (Common with Transmitter Ground)
11	V _{EER}	Receiver Ground (Common with Transmitter Ground)
12	RD-	Receiver Inverted DATA out. AC Coupled
13	RD+	Receiver Non-inverted DATA out. AC Coupled
14	V _{EER}	Receiver Ground (Common with Transmitter Ground)
15	V _{CCR}	Receiver Power Supply
16	V _{CCT}	Transmitter Power Supply
17	V _{EET}	Transmitter Ground (Common with Receiver Ground)
18	TD+	Transmitter Non-Inverted DATA in. AC Coupled.
19	TD-	Transmitter Inverted DATA in. AC Coupled.
20	V _{EET}	Transmitter Ground (Common with Receiver Ground)

Appendix 2. Pins of the 8P8C connector of Smart SFP 1G RS-232

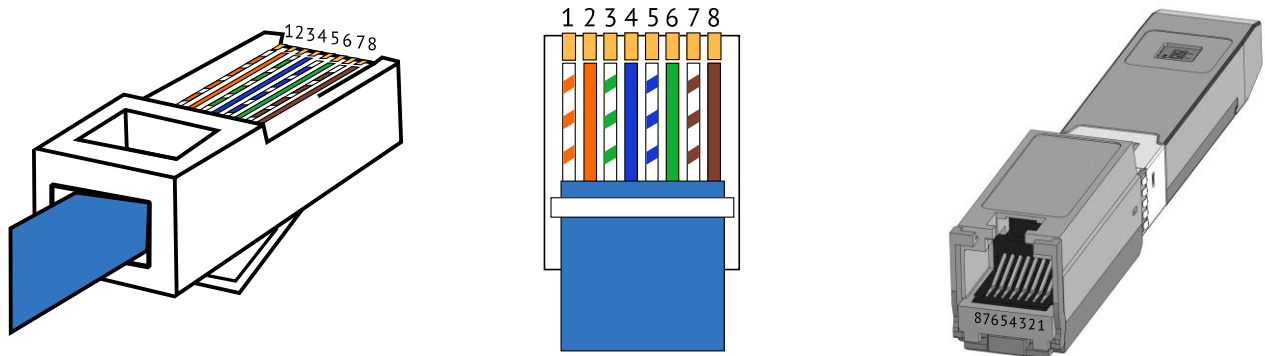


Figure 16.11. 8P8C connector of Smart SFP 1G RS-232

Table 16.2. Pins of the 8P8C connector of Smart SFP 1G RS-232 with feature set supporting one RS-232 port with DSR, DCD, DTR, CTS, RTS circuits in accordance with EIA/TIA 561

Pin number	Circuit
1	DSR OUT
2	DCD OUT
3	DTR IN
4	GND
5	RD OUT
6	TD IN
7	CTS OUT
8	RTS IN

Table 16.3. Pins of the 8P8C connector of Smart SFP 1G RS-232 with feature set supporting three RS-232 port without DSR, DCD, DTR, CTS, RTS circuits

Pin number	Circuit
1	RD OUT port #2
2	Not connected
3	TD IN port #2
4	GND
5	RD OUT port #0
6	TD IN port #0
7	RD OUT port #1
8	TD IN port #1

Appendix 3. Pins of the 8P8C connector of Smart SFP 1G GNSS

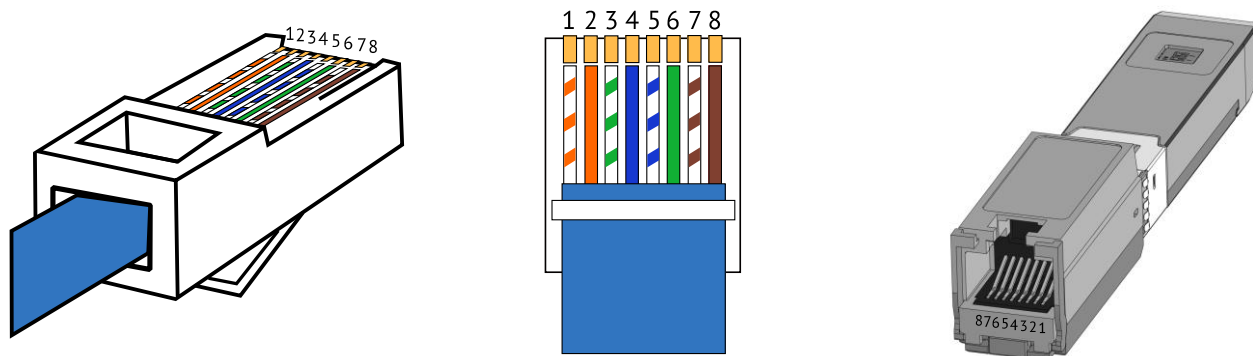


Figure 16.12. 8P8C connector of Smart SFP 1G GNSS

Table 16.4. Pins of the 8P8C connector of Smart SFP 1G GNSS

Pin number	Circuit
1	Not connected
2	+5V
3	Not connected
4	GND
5	RD OUT
6	TD IN
7	Not connected
8	1PPS IN

Appendix 4. Smart SFP Dimensions

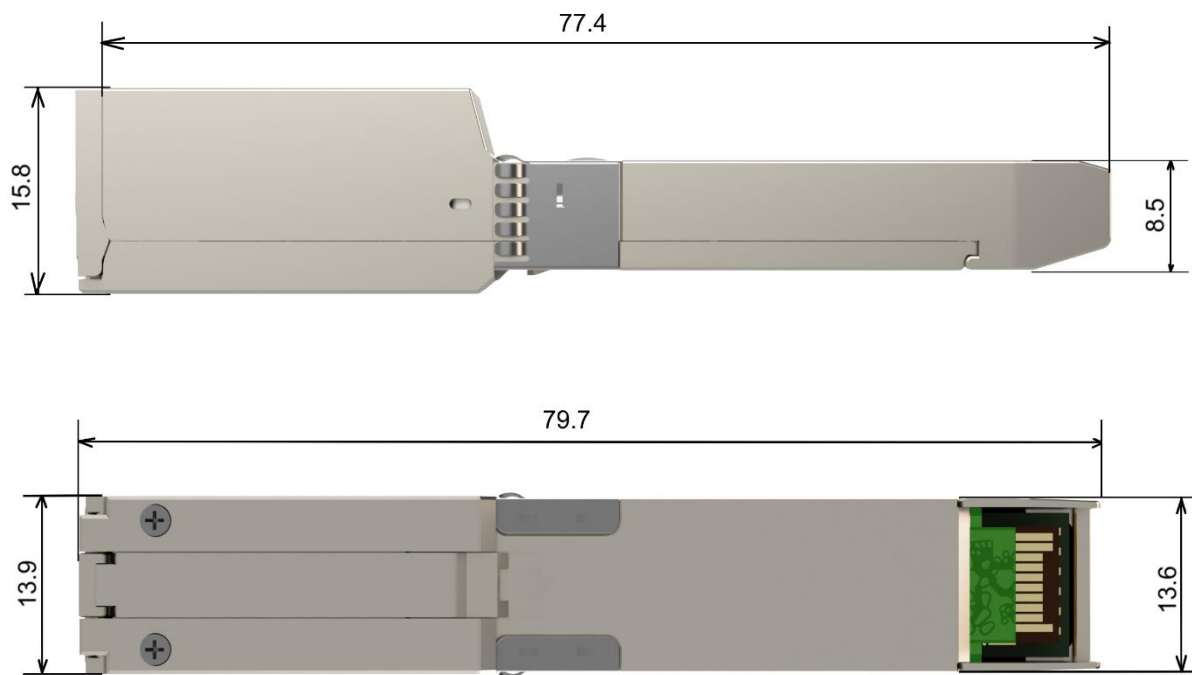


Figure 16.13. Dimensions of Smart SFP 1G copper, Smart SFP 1G RS-232, Smart SFP 1G GNSS, Smart SFP 1G USB